

System Architecture & Clean Code

Expanded Edition 2026

LearnForge

CHAPTER 01 From Coder to Architect

CHAPTER 01 From Coder to Architect

Software engineering is about writing code that works. Software architecture is about deciding what code to write, why, and managing the long-term cost of change. WHAT YOU WILL LEARN • The conceptual shift from local implementation to holistic design. • How to balance immediate delivery velocity against structural decay. • Strategies for evaluating architectural trade-offs explicitly. What It Means Moving from a software developer to a software architect requires shifting your primary frame of reference. A developer focuses on algorithm efficiency, framework mechanics, and clean class interfaces within a module. An architect views systems as interconnected units governed by business realities, operational costs,

network boundaries, and organizational team structures. Why It Matters Local optimizations can create global catastrophes. Writing ultra-performant code inside a service is useless if that service creates a synchronous network bottleneck across three other sub-systems. Architecture ensures that team efforts scale horizontally without incurring exponential complexity overhead. ARCHITECT'S RULE

Architecture is the decisions that are hard to change later. If a choice can be refactored in an afternoon, it is an implementation detail, not an architectural decision. Practical

Example: Managing System Boundaries

Consider a developer tasked with integrating a payment gateway. The developer writes code that calls the external Stripe API directly inside the core HTTP controller layer. LearnForge •

Engineering Handbook • 2026 4

LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE // POOR IMPLEMENTATION:

Mixed concerns in Controller import

```
{ Request, Response } from 'express';
```

```
import Stripe from 'stripe'; const stripe  
= new
```

```
Stripe(process.env.STRIPE_SECRET_KEY!,
```

```
{ apiVersion: '2023-10-16' }); export
```

```
async function handleCheckout(req:
```

```
Request, res: Response) { const
```

```
{ amount, currency } = req.body; //
```

Direct tightly-coupled call to third-party

SDK const paymentIntent = await

```
stripe.paymentIntents.create({ amount,
```

```
currency, }); // Directly writing payment
```

status inside user controller context

```
await db.query('INSERT INTO payments
```

```
VALUES ($1, $2)', [paymentIntent.id,
```

```
amount]); res.json({ success: true,
```

```
clientSecret:
```

```
paymentIntent.client_secret }); } An
```

architect abstracts this integration behind an internal domain boundary interface, insulating the application core from external SDK vendor changes or multi-provider requirements. //

ARCHITECTURAL APPROACH: Decoupled Gateway Interface export interface

IPaymentProcessor

{ processPayment(amount: number, currency: string): Promise; } export

class StripePaymentAdapter

implements IPaymentProcessor

{ constructor(private client: Stripe) {}

async processPayment(amount:

number, currency: string): Promise

{ const response = await

this.client.paymentIntents.create({ amount, currency }); return { transactionId:

response.id, status: response.status

=== 'succeeded' ? 'SUCCESS' :

'PENDING' }; } } COMMON MISTAKE

Confusing structural abstraction with

over-engineering. Abstraction must serve clear failure isolation or change control boundaries, not theoretical future requirements. PRODUCTION TIP Document architectural trade-offs using Architecture Decision Records (ADRs) immediately when decisions are made. Never rely on team memory. KEY TAKEAWAYS • Seniority is marked by thinking in systems, failure modes, and operational costs. • Every architecture is a trade-off; there are no silver bullets, only costs and consequences. • Isolate core business domain code from volatile external infrastructure dependencies. QUICK REVIEW LearnForge • Engineering Handbook • 2026 5 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE 1. What differentiates an implementation detail from an architectural decision? 2. Why does direct third-party SDK leakage

across controller layers increase long-term maintenance cost? ARCHITECT'S CHECKLIST

- ☐ Have I isolated external vendor API calls behind explicit domain interfaces?
- ☐ Are all core architecture choices documented in explicit ADR files?

LearnForge • Engineering Handbook • 2026 6 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

CHAPTER 02 The Foundations of Good Architecture

CHAPTER 02 The Foundations of Good Architecture

Good architecture minimizes the lifetime cost of building, maintaining, operating, and extending a software system. WHAT YOU WILL LEARN

- The core architectural quality attributes (ilities).**
- How to balance competing quality requirements.**
- How structural simplicity protects software lifespan.**

What It Means

System architecture consists of structural patterns chosen

to satisfy system non-functional properties (quality attributes). A sound architecture respects the core attributes: Maintainability, Reliability, Scalability, and Security. Why It Matters

Software development costs are heavily skewed toward maintenance (often 70-80% of total lifecycle expenditure). Systems constructed without sound structural foundations experience structural entropy—where every new feature addition exponentially increases defect probability and cycle times.

Maintainability Reliability Scalability Security Trade-offs Quality Attribute Optimizing For Extreme Sub-millisecond low latency, aggressive Performance caching High Extensibility LearnForge • Engineering Handbook • 2026 7

LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE Plugin frameworks, strict interfaces Strict Consistency

Distributed ACID transactions Trade-off / Cost Higher operational complexity, stale data risk Higher initial abstraction cost, steeper learning curve Lower system availability, increased latency

ARCHITECT'S RULE Simplicity is a prerequisite for reliability. Complex systems fail in complex, unforecastable ways. **KEY TAKEAWAYS** • Architecture quality is evaluated by system maintainability over time, not initial launch speed. • Optimizing one quality attribute usually degrades another; explicitly manage these trade-offs.

QUICK REVIEW 1. Why does optimizing for maximum extensibility often harm code readability? 2. What is structural entropy in long-lived software systems?

ARCHITECT'S CHECKLIST ☐ Are all foundational non-functional properties identified and prioritized? ☐ Has the architecture chosen simplicity over

complex abstractions where possible?

LearnForge • Engineering Handbook •

2026 8 LEARNFORGE SYSTEM

ARCHITECTURE & CLEAN CODE CHAPTER

03 Requirements Before Architecture

CHAPTER 03 Requirements Before

Architecture Designing systems without concrete functional and non- functional requirements guarantees

overengineering or critical production

failure. WHAT YOU WILL LEARN • How to extract explicit Architectural

Characteristics from vague user stories.

• Quantifying SLA, SLO, and SLA targets effectively. • Estimating throughput, memory, and storage boundaries

upfront. What It Means Before drawing architecture diagrams, architects

translate business goals into explicit, quantifiable engineering requirements.

Requirements split into Functional

Requirements (what the system does)

and **Non-Functional Requirements / Quality Attributes**** (how well the system performs it). Why It Matters**

Designing for "high scale" without exact numbers leads to massive financial waste or incorrect infrastructure choices. A system needing 100 requests per second (RPS) requires a radically different design than one processing 100,000 RPS. Quantifying Systems:

Back-of-the-Envelope Estimation

Consider an application with 10 million daily active users (DAU). Each user makes 20 API requests per day. • Total Daily Requests: $10,000,000 \times 20 = 200,000,000$ requests/day. • Average Requests Per Second (RPS): $200,000,000 / 86,400 \approx 2,315$ RPS. • Peak Traffic (3x Average Rule): $2,315 \times 3 \approx 7,000$ Peak RPS. COMMON MISTAKE Designing for average load instead of peak load with

a safety headroom buffer. KEY

TAKEAWAYS • Always translate business requests into concrete numbers: RPS, payload size, storage growth, and

availability percentages. • Architect for peak loads, not daily averages. QUICK

REVIEW 1. What is the difference

between an SLA, an SLO, and an SLI? 2.

How do you convert Daily Active Users

into Peak RPS? ARCHITECT'S CHECKLIST

LearnForge • Engineering Handbook •

2026 9 LEARNFORGE SYSTEM

ARCHITECTURE & CLEAN CODE ☐ Have

maximum peak throughput metrics

been calculated and verified with

business stakeholders? ☐ Are data

retention limits and storage growth

expectations explicitly defined?

LearnForge • Engineering Handbook •

2026 10 LEARNFORGE SYSTEM

ARCHITECTURE & CLEAN CODE CHAPTER

04 Scalability CHAPTER 04 Scalability

Scalability is the structural ability of a system to handle increased load by adding resources, without breaking performance guarantees. WHAT YOU WILL LEARN • Vertical vs. Horizontal scaling trade-offs. • State management in horizontally scaled app tiers. • Database scale-out patterns: Read replicas vs. Sharding. What It Means A system is scalable if adding hardware resources linearly increases its capacity to process load. **Vertical scaling (scaling up) means adding CPU/RAM to a single node. Horizontal scaling (scaling out) means adding more distinct compute nodes to a pool. Load Balancer API Node 1 API Node 2 API Node 3 Stateless Application Tier To enable horizontal scaling at the API level, application nodes must be completely stateless. Session states must be offloaded to distributed data**

stores like Redis. // STATEFUL ANTI-PATTERN (Blocks Horizontal Scaling)

```
const activeSessions = new Map(); // In-memory local state!  
app.post('/login', (req, res) => { // Session locked to this specific node process instance  
activeSessions.set(sessionToken, user); })); // STATELESS ARCHITECTURAL PATTERN (Enables Horizontal Scaling)
```

```
import { createClient } from 'redis';  
const redis = createClient({ url: process.env.REDIS_URL });  
app.post('/login', async (req, res) => { // Shared state offloaded to centralized cache accessible by all nodes await  
redis.set(`session:${sessionToken}`, JSON.stringify(user), { EX: 86400 }); }));
```

ARCHITECT'S RULE LearnForge • Engineering Handbook • 2026 11

LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE Never hold mutable operational state in node application

memory if you intend to scale horizontally. KEY TAKEAWAYS • Horizontal scaling requires stateless application layers. • Database scaling is significantly harder than compute scaling; use read-replicas before introducing sharding complexity. QUICK REVIEW 1. Why does in-memory local session storage break horizontal autoscaling? 2. What is the primary constraint of vertical scaling? ARCHITECT'S CHECKLIST ☐ Is the API/ Application service tier completely stateless? ☐ Are session tokens validated via stateless JWTs or backed by a shared Redis cluster? LearnForge • Engineering Handbook • 2026 12 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE CHAPTER 05 Performance, Latency and Throughput CHAPTER 05 Performance, Latency and Throughput Optimizing performance requires

accurate measurement of response time distributions rather than arithmetic averages. WHAT YOU WILL LEARN • Differentiating Latency, Throughput, and Response Time. • Why averages lie: Understanding Percentiles (p95, p99, p99.9). • Eliminating blocking I/O operations in high- throughput systems. What It Means Latency is the time taken for an individual operation to complete. Throughput is the volume of operations completed within a specific window (e.g., requests per second). The Flaw of Averages Measuring average latency masks severe tail latency issues. If 99 requests take 10ms and 1 request takes 10,000ms, the average response time is ~109ms. However, 1% of your user base experiences an intolerable 10second delay. Architects evaluate systems using percentiles: p50 (median), p95, and p99. // NON-BLOCKING

CONCURRENCY PATTERN (Node.js) //

**Bad: Sequential async processing
creates accumulative tail latency async
function**

```
processOrdersSequential(orderIds:  
string[]) { const results = []; for (const  
id of orderIds) { // Total time = sum of  
all individual latencies const res = await  
processOrder(id); results.push(res); }  
return results; }
```

**Good: Concurrent
processing caps total time near max
single request latency async function**

```
processOrdersConcurrent(orderIds:  
string[]) { // Executes requests in  
parallel across Event Loop cycles return  
Promise.all(orderIds.map(id =>  
processOrder(id))); }
```

**PRODUCTION TIP
Configure client timeouts and circuit
breakers on all external HTTP or
database calls to prevent tail latency
cascades. LearnForge • Engineering**

Handbook • 2026 13 LEARNFORGE

SYSTEM ARCHITECTURE & CLEAN CODE

KEY TAKEAWAYS • Never monitor

performance using arithmetic average latency; track p95 and p99 percentiles.

• Concurrency allows bounding execution times to the slowest single operation instead of summing latencies.

QUICK REVIEW 1. Why are latency percentiles more informative than mean response times? 2. What is the impact of unbounded tail latencies in microservice dependency chains?

ARCHITECT'S CHECKLIST ☐ Are dashboards tracking p95 and p99 response time percentiles explicitly? ☐

Do all downstream network calls enforce strict timeout configurations?

Chapter 06 Monolith, Modular Monolith and Microservices Architecture should evolve from business and operational constraints, not from the assumption that distributed systems are

automatically superior. What You Will Learn

- How monolithic, modular-monolithic, and microservice architectures differ structurally.**
- When a modular monolith is a better choice than immediate service decomposition.**
- How network boundaries introduce latency, failure, deployment, and operational costs.**
- How to identify service boundaries from business capabilities rather than technical layers.**
- How to migrate incrementally from a monolith without creating a distributed monolith.**

1. The Architectural Choice Is a Trade-off

One of the most common architectural mistakes is treating “monolith” and “microservices” as competing technologies rather than different ways of organizing system boundaries. A monolith deploys multiple capabilities as one application unit. Microservices divide capabilities

into independently deployable services. A modular monolith sits between these approaches: it can be deployed as one application while maintaining strong internal boundaries between business modules.

LearnForge • Engineering Handbook • 2026 14 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

The correct choice depends on factors such as domain complexity, team structure, deployment independence, reliability requirements, expected scale, organizational maturity, and operational capacity.

Architect's Rule

Do not introduce a network boundary merely to make a diagram look distributed. A boundary should solve a real organizational, deployment, scaling, reliability, or domain-isolation problem.

2. The Traditional Monolith

In a monolithic architecture, the application's major capabilities run

within a common deployable unit. The system may contain modules for authentication, payments, catalog management, reporting, and other business functions, while still being deployed as one application. Figure 6.1 — A monolith can contain multiple business capabilities while remaining one deployable application. Strengths •

< cat > ~/ebook_premium/chapters/ chapter-06.html <<'EOF' Chapter 06 Monolith, Modular Monolith and Microservices Architecture should evolve from business and operational constraints, not from the assumption that distributed systems are automatically superior. What You Will Learn ◦ How monolithic, modular-monolithic, and microservice architectures differ. ◦ When a modular monolith is preferable to immediate service decomposition. ◦ How network

boundaries introduce latency, failure, and operational cost. ◦ How to identify service boundaries from business capabilities. ◦ How to migrate incrementally without creating a distributed monolith. LearnForge • Engineering Handbook • 2026 15

LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE 1. The Architectural Choice Is a Trade-off One of the most common architectural mistakes is treating “monolith” and “microservices” as competing technologies rather than different ways of organizing system boundaries. A monolith deploys multiple capabilities as one application unit. Microservices divide capabilities into independently deployable services. A modular monolith sits between these approaches: it can be deployed as one application while maintaining strong internal boundaries between business

modules. The correct choice depends on domain complexity, team structure, deployment independence, reliability requirements, expected scale, and operational maturity. Architect's Rule Do not introduce a network boundary merely to make a

diagram look distributed. A boundary should solve a real organizational, deployment, scaling, reliability, or domain- isolation problem.

2. The Traditional Monolith

In a monolithic architecture, major business capabilities run inside a common deployable unit. Authentication, courses, payments, reporting, and administration may all exist inside the same application.

Figure 6.1 — A monolith can contain multiple business capabilities while remaining one deployable application.

Strengths

- Simple deployment and local development.
- Low network overhead between modules.
- Simple transactions across related data.
- Lower infrastructure and operational complexity.
- Easy debugging when the system is relatively small.

Risks

- Weak internal boundaries can create tightly coupled code.

LearnForge • Engineering Handbook • 2026

16 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

- Large deployments can become slower and riskier.
- Independent scaling of individual workloads becomes difficult.
- A failure in one area can affect the whole application.

Common Mistake Assuming that every monolith is badly designed. A well-structured monolith can be easier to operate and evolve than a poorly designed microservice system.

3. Modular Monolith

A modular monolith keeps one deployment unit while enforcing explicit boundaries between business capabilities. For example, an education platform might contain separate modules for identity, courses, payments, and notifications. Each module owns its internal business rules and exposes a small public interface to other modules.

```
export interface CourseCatalog {
  getPublishedCourse(courseId: string): Promise;
}
export class EnrollmentService {
  constructor(
    private readonly catalog: CourseCatalog,
    private readonly enrollmentRepository: EnrollmentRepository
  ) {}
  async enroll(userId: string, courseId: string) {
    const course = await this.catalog.getPublishedCourse(courseId);
    if (!course) {
      throw new Error('Course is not available');
    }
    return this.enrollmentRepository.create({
      userId,
      courseId
    });
  }
}
```

The important architectural property is not the TypeScript syntax itself. The important property is

that the enrollment domain depends on an explicit business contract rather than reaching directly into another module's database implementation. LearnForge • Engineering Handbook • 2026 17 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

4. Microservices Microservices take modular boundaries one step further by making services independently deployable processes. Communication usually occurs through HTTP, gRPC, or asynchronous messaging. Figure 6.2 — Microservices introduce explicit process and network boundaries.

5. The Cost of a Network Boundary Moving functionality from an in-process call to a remote service introduces new failure modes. The caller must now consider network latency, timeouts, retries, partial failure, authentication, service discovery, observability, and version compatibility.

Concern	In-Process Module	Remote Service
Latency	Very low	Network dependent
Failure	Usually local	Partial failures possible
Deployment	Shared	Independent
Scaling	Usually application-wide	Service-specific
Operations	Lower complexity	Higher complexity
Transactions	Relatively simple	Distributed transaction challenges

6. Choosing the Right Architecture A practical decision process starts with the simplest architecture that can satisfy known requirements. LearnForge • Engineering Handbook • 2026 18 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

Situation	Recommended Starting Point
Small team and evolving product	Modular monolith
Strong transactional requirements	Monolith or modular monolith
Clearly independent high-scale workloads	Selective service decomposition
Large organization with autonomous teams	Potentially microservices
Unclear domain boundaries	Modular monolith first

Production Tip Microservices should usually be extracted because a boundary has become valuable, not because a framework makes service creation easy.

7. Migration Without a Distributed Monolith When an existing monolith has reached a point where independent deployment or scaling is valuable, migration should be incremental.

1. Identify a stable business capability.
2. Define a clear ownership boundary.
3. Separate the module internally before extracting it.
4. Introduce an explicit API or event contract.
5. Move the capability behind the new boundary.
6. Measure latency, failure rate, operational cost, and deployment impact.

Architect's Rule First create a modular boundary. Then, only when justified, turn that boundary into a network boundary.

Key Takeaways

- A monolith is not inherently bad architecture.
- A modular monolith provides strong boundaries without network complexity.

LearnForge • Engineering Handbook • 2026 19 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

- Microservices provide independent deployment and scaling at a significant operational cost.
- Business capabilities are usually stronger service-boundary candidates than technical layers.
- Architecture should evolve according to measurable constraints.

Quick Review

1. Why can a modular monolith be preferable to microservices for a new product?
2. What new failure modes appear when an in-process call becomes a network call?
3. Why should business capabilities influence service boundaries?

Architect's Checklist

- Are business modules clearly separated?
- Are module dependencies explicit?
- Is a network boundary solving a measurable problem?
- Have latency and failure modes

been evaluated before extracting a service? LearnForge • Engineering Handbook • 2026 20 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE Chapter 07

Microservice Trade-offs

Microservices can improve independent scaling and deployment, but every distributed boundary introduces new costs in latency, reliability, observability, security, and operational complexity. What You Will Learn • How to evaluate the real benefits and costs of microservices. • Why distributed systems require explicit failure-handling strategies. • How synchronous and asynchronous communication affect architecture. • How service ownership and data ownership reduce coupling. • How to avoid the distributed-monolith anti-pattern.

1. Why Microservices Are Attractive Microservices are attractive because they allow different parts of a system to evolve independently. A team can deploy one service without necessarily redeploying the entire platform. A heavily used service can also be scaled independently from services with lower traffic. For a large organization, these properties can be extremely valuable. However, independence is not free. The architecture exchanges some in-process simplicity for distributed-system complexity. Architect's Rule Every independent service creates an independent operational responsibility. If the organization cannot operate the additional boundaries reliably, the architecture may be more complex than the business requires.
2. Independent Deployment One of the strongest arguments for microservices is deployment independence. A payment service can release a new implementation without requiring a simultaneous release of the course catalog.

LearnForge • Engineering Handbook • 2026 21 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

Figure 7.1 — Microservices can provide independent deployment and scaling, but each boundary creates additional operational responsibility.

3. The Distributed-System Tax When two modules communicate inside one process, a function call is normally fast and predictable. When those modules become separate services, the call must cross a network. The network introduces uncertainty. Packets can be delayed, connections can fail, services can become overloaded, and dependencies can become temporarily unavailable.

In-Process Call	Distributed Call
Usually microseconds or less	Network latency is variable
Failure is usually local	Timeouts and partial failures are possible
Simple stack traces	Distributed tracing may be required
Simple memory model	Serialization and transport required
Usually one deployment	Independent deployment possible

Common Mistake Treating a remote service call as if it were an ordinary local function call. A remote dependency must always be treated as potentially slow, unavailable, or partially successful.

4. Synchronous Communication Synchronous communication is useful when a caller needs an immediate result. HTTP and gRPC are common choices.

LearnForge • Engineering Handbook • 2026 22 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

```

async function getCourse(courseId: string) {
  const response = await fetch(
    `https://course-service.internal/courses/${courseId}`,
    {
      signal: AbortSignal.timeout(1500)
    }
  );
  if (!response.ok) {
    throw new Error(`Course service returned ${response.status}`);
  }
  return response.json();
}

```

The timeout is important.

Without a bounded timeout, a slow dependency can consume application resources while requests wait indefinitely.

5. Retries Are Not Automatically Safe Retries can improve reliability when failures are temporary. However, repeating an operation can be dangerous when the operation is not idempotent. For example, blindly retrying a payment request may create duplicate charges if the first request succeeded but the response was lost. Architect's Rule Retry only when the operation and failure mode are understood. For state-changing operations, use idempotency keys or another mechanism that makes repeated requests safe.

LearnForge • Engineering Handbook • 2026

```
23 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE
interface PaymentRequest { amount: number; currency: string; idempotencyKey: string; }
async function createPayment(request: PaymentRequest) { return
paymentClient.create({ amount: request.amount, currency: request.currency,
headers: { 'Idempotency-Key': request.idempotencyKey } }); }
```

6. The Distributed Monolith A distributed monolith occurs when an application is technically divided into multiple services but those services remain tightly coupled. For example, if ten services must always be deployed together, communicate synchronously for every request, and share one database schema, the system may have the operational cost of microservices without receiving their independence benefits.

Figure 7.2 — A distributed monolith can preserve tight coupling while adding network and operational complexity.

7. Data Ownership One of the most important principles in service architecture is clear data ownership. A service should normally be responsible for the state belonging to its business capability.

LearnForge • Engineering Handbook • 2026

```
24 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE
Service Primary Responsibility Example
Data Identity Authentication and identity
Users, credentials, roles
Course Learning content Courses, lessons, publishing state
Payment Financial transactions Orders, payment status, invoices
Notification Message delivery Templates, delivery status
Clear ownership reduces accidental coupling. Other services should access another service's business state through a defined API or event contract rather than directly modifying its database tables.
```

8. Asynchronous Communication Asynchronous messaging is useful when the caller does not need an immediate result. Instead of waiting for another service, the application publishes an event and continues.

```
await eventBus.publish({ type: 'COURSE_PUBLISHED', eventId: crypto.randomUUID(), courseId, publishedAt: new Date().toISOString() });
```

Consumers can process the event independently. This can improve resilience and reduce synchronous dependency chains.

Production Tip Use asynchronous communication for work that can tolerate eventual consistency, such as notifications, analytics, search indexing, and many background processing tasks.

9. Operational Complexity A production microservice platform requires more than application code. Teams must manage deployment pipelines, service discovery, secrets, monitoring, distributed tracing, logs, certificates, network policies, capacity planning, and incident response.

LearnForge • Engineering Handbook • 2026

```
25 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE
```

Therefore, the decision to

adopt microservices is partly an organizational decision. Architecture and team capability must evolve together.

10. A Practical Decision Framework

Question If Yes If No Does one capability require independent scaling? Consider extraction Keep it together Does it need independent deployment? Service boundary may help Shared deployment may be simpler Is the domain boundary well understood? cat >> ~/ebook_premium/chapters/chapter-07.html <<'EOF' Domain is still unclear Keep the boundary inside a modular monolith Can the team operate distributed infrastructure? Microservices become more realistic Prefer simpler architecture Is failure isolation important? Consider a separate service Shared process may be sufficient Common Mistake LearnForge • Engineering Handbook • 2026 26 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE Choosing microservices because they are associated with large technology companies. Architecture should follow the system's actual constraints, not another organization's scale or technology stack.

11. Migration Without a Big-Bang Rewrite Moving from a monolith to microservices should normally be incremental. Rewriting the entire platform at once creates a large change surface and makes failures difficult to isolate. A safer approach is to identify one business capability with a clear boundary, establish an API or event contract, move ownership gradually, and measure the operational result.

1. Identify a high-value and relatively independent domain.
2. Define its public contract.
3. Separate its business logic from unrelated modules.
4. Establish explicit data ownership.
5. Introduce observability before extraction.
6. Deploy the new service alongside the existing system.
7. Gradually move traffic and verify behavior.
8. Remove the old implementation only after the new path is stable.

Production Tip The first extraction should be chosen for learning and risk control, not simply because it is technically interesting. A well-observed small migration is often more valuable than a large architectural rewrite.

LearnForge • Engineering Handbook • 2026 27 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

12. Summary Comparison

Characteristic	Monolith	Modular Monolith	Microservices
Deployment	Single unit	Single unit	Independent units
Internal boundaries	Can be weak	Explicit modules	Network/service boundaries
Operational	Low	Low to moderate	High complexity
Independent scaling	Limited	Limited	Strong
Failure isolation	Lower	Moderate	Potentially strong
Network failure	Low	Low	High concerns
Team autonomy	Moderate to high	scale healthy	Small or evolving
Growing systems	with Systems with proven	Best starting point	systems clear domains independent-service needs

13. Final Architectural Principle There is no universally superior deployment model. A small product can achieve excellent reliability and maintainability with a well-structured monolith. A large organization may eventually benefit from independently deployable services. The important architectural skill is recognizing when the constraints have changed enough to justify a new boundary.

Architect's Rule Start with the simplest architecture that satisfies the current requirements, but design internal boundaries so that future extraction remains possible when evidence justifies it.

Key Takeaways • Microservices trade local

simplicity for distributed-system complexity. • Independent deployment and scaling are valuable only when the system actually needs them. LearnForge • Engineering Handbook • 2026 28 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE • Remote calls must be treated as unreliable dependencies with timeouts and explicit failure handling. • Idempotency is essential when retries can repeat state-changing operations. • Clear service and data ownership reduces coupling. • A modular monolith is often an excellent intermediate architecture. • Incremental migration is safer than a big-bang rewrite. Quick Review 1. What additional costs does a network boundary introduce? 2. Why can a distributed monolith be worse than a conventional monolith? 3. When should asynchronous communication be preferred? 4. Why is idempotency important for retried operations? 5. When is a modular monolith a better choice than microservices? Architect's Checklist • Does every service boundary solve a real problem? • Are service responsibilities aligned with business capabilities? • Does every remote call have bounded timeout behavior? • Are retryable operations explicitly designed for idempotency? • Is data ownership clearly defined? • Are distributed dependencies observable through logs, metrics, and tracing? • Can the system operate reliably with the organization's current engineering capacity?Chapter 08 Distributed Systems A distributed system is not simply a collection of computers. It is a system in which independent components communicate across boundaries where delay, failure, concurrency, and incomplete information are unavoidable. LearnForge • Engineering Handbook • 2026 29 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE What You Will Learn • Why distributed systems behave differently from single-process applications. • How latency, partial failure, and unreliable communication affect architecture. • The practical meaning of consistency, availability, and partition tolerance. • Why idempotency, timeouts, retries, and deduplication are essential. • How to design distributed workflows that remain understandable during failure. 1. What Makes a System Distributed? A system becomes distributed when important parts of its computation or state exist in separate processes or machines and must communicate over a network. The boundary may exist between two services, between an application and a database cluster, or between geographically separated infrastructure. The most important architectural consequence is that communication is no longer a simple in- memory operation. The network introduces delay, interruption, duplication, reordering, and uncertainty. Architect's Rule Treat every remote dependency as a failure boundary. A remote call can be slow, unavailable, duplicated, rejected, or successful even when its response never reaches the caller. 2. The Network Is Not Reliable Developers often design distributed systems as if communication were deterministic. Production systems demonstrate the opposite. Packets may be delayed, connections may reset, DNS may fail, a dependency may become overloaded, or a response may arrive after the caller has already timed out. LearnForge • Engineering Handbook • 2026 30 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE Failure Possible Effect Architectural Response

Caller cannot determine whether work Timeout Idempotency and status checking completed Connection failure Request may never reach dependency Bounded retry where safe Deduplication or idempotent Duplicate message Operation may execute twice processing Delayed response Stale result may arrive later Deadlines and request correlation Dependency Latency and error rate increase Backpressure and load shedding overload

3. Partial Failure In a local application, a serious process failure often stops the entire operation. In a distributed system, one component can fail while other components continue operating. This condition is called partial failure. It is one of the defining challenges of distributed architecture because the system may remain alive while an important part of the workflow is unavailable. Figure 8.1 — Partial failure: one dependency can fail while other components remain operational.

4. Timeouts and Deadlines Every remote request should have a bounded time budget. Without a timeout, a slow dependency can consume threads, connections, memory, and other resources until the caller itself becomes unhealthy.

LearnForge • Engineering Handbook • 2026 31 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

```

async function getPayment(paymentId: string) {
  const response = await fetch( `https://payment.internal/payments/${paymentId}`, {
    signal: AbortSignal.timeout(1500) } );
  if (!response.ok) { throw new Error(`Payment service returned ${response.status}`); }
  return response.json(); }

```

A deadline is even more useful when a request crosses several services. The remaining time budget can be propagated so downstream services do not continue expensive work after the original request has already expired. Production Tip Set timeouts based on measured service-level objectives rather than arbitrary large values. Monitor timeout rates and latency distributions in production.

5. Retries and Backoff Temporary failures can sometimes be recovered by retrying. A retry strategy should normally include a maximum attempt count and exponential backoff.

LearnForge • Engineering Handbook • 2026 32 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

```

async function retryWithBackoff( operation: () => Promise, attempts = 3 ) {
  for (let i = 0; i < attempts; i++) {
    try { return await operation(); }
    catch (error) {
      if (i === attempts - 1) { throw error; }
      const delay = 200 * 2 ** i;
      await new Promise(resolve => setTimeout(resolve, delay));
    }
  }
}

```

Retries should not be applied blindly. If many clients retry simultaneously against an overloaded service, the additional traffic can make the outage worse. This effect is commonly called a retry storm. Common Mistake Retrying every error. Permanent failures such as invalid input or authorization failures generally should not be retried.

6. Idempotency An operation is idempotent when repeating the same request produces the same intended state rather than creating additional unintended effects. Idempotency is especially important when a client cannot determine whether a previous request succeeded.

LearnForge • Engineering Handbook • 2026 33 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

```

interface PaymentRequest {
  amount: number;
  currency: string;
  idempotencyKey: string;
}
async function createPayment(request: PaymentRequest) {
  return paymentClient.create({
    amount: request.amount,

```

```

currency:      request.currency,      headers:      {      "Idempotency-Key":
request.idempotencyKey } }); }

```

If the client sends the same idempotency key again, the payment system can recognize that the operation has already been processed and avoid creating a duplicate transaction.

7. Consistency and Availability

Distributed systems frequently require trade-offs between consistency, availability, latency, and partition tolerance.

Requirement	Typical Strategy	Cost
Strong consistency	Higher latency and reduced availability	Synchronous coordination
Eventual consistency	Temporary stale data	Asynchronous replication
High availability	Replication and failover	More infrastructure and coordination

Continue operating despite Partition tolerance

Consistency trade-offs may be required communication failure

Architect's Rule

Do not discuss consistency as an abstract preference. Define which business operations require immediate correctness and which operations can tolerate temporary divergence.

LearnForge • Engineering Handbook • 2026 34

LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE 8. CAP in Practical Terms

The CAP theorem describes a fundamental limitation of distributed data systems during a network partition: a system cannot simultaneously guarantee both strong consistency and full availability for every operation while also tolerating the partition. CAP is frequently oversimplified. It does not mean that every system must choose exactly two permanent properties. Instead, it explains the trade-off that becomes unavoidable when communication between distributed components fails.

Figure 8.2 — CAP highlights the consistency/availability trade-off when network partitions occur.

9. Message Delivery Semantics

Distributed messaging systems commonly provide delivery models such as at-most-once, at-least-once, and effectively-once processing patterns. Understanding these semantics is important because message delivery and business effects are separate concerns.

Model	Meaning	Typical Risk
At-most-once	Message is delivered zero or one time	Messages may be lost
At-least-once	Message is delivered one or more times	Duplicates must be handled
Effectively-once	Business effect is made safe against duplicates	Requires careful application design

In practice, at-least-once delivery combined with idempotent consumers is often easier to reason about than pretending that a distributed transport can guarantee perfect exactly-once business behavior.

10. Deduplication

When duplicate events are possible, consumers can maintain a processed-event record or use an idempotency key to prevent repeated business effects.

LearnForge • Engineering Handbook • 2026 35

LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

```

async function
handleEvent(event: { eventId: string; type: string; }) {
  if (await eventStore.wasProcessed(event.eventId)) {
    return;
  }
  await processBusinessEffect(event);
  await eventStore.markProcessed(event.eventId);
}

```

Common Mistake

Assuming that a message queue automatically prevents duplicate business effects. Transport-level delivery guarantees do not replace application-level idempotency.

11. Backpressure and Load Shedding

When a downstream system cannot process work as quickly as requests arrive, the upstream system must avoid creating an unlimited backlog. Backpressure is the mechanism by which a system

limits or slows incoming work when capacity is constrained. Without backpressure, queues can grow until memory, connections, storage, or worker capacity are exhausted. The result can be a cascading failure in which one overloaded component causes neighboring components to become unhealthy. Architect's Rule A resilient system does not attempt to accept unlimited work. It protects critical resources by controlling concurrency, queue depth, and request admission.

12. Circuit Breakers

A circuit breaker prevents repeated calls to a dependency that is already known to be unhealthy. Instead of continuously sending traffic to the failed service, the caller temporarily stops requests and returns a controlled fallback or error.

LearnForge • Engineering Handbook • 2026 36 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

State	Behavior	Purpose
Closed	Requests flow normally	Normal operation
Open	Requests fail fast	Protect caller and dependency
Half-open	Limited test requests are allowed	Determine whether recovery occurred

Circuit breakers should be combined with meaningful timeouts, monitoring, and carefully designed fallback behavior. A fallback that returns incorrect business information can be worse than an explicit error.

13. Cascading Failure

Cascading failure occurs when the failure of one component increases load or resource consumption in other components, causing additional failures.

Figure 8.3 — Cascading failures can propagate through synchronous dependency chains.

14. Designing for Graceful Degradation

A resilient system does not always need every feature to remain available. Graceful degradation means preserving the most important user and business functions while temporarily reducing secondary functionality.

Dependency Failure	Possible Degradation	Recommendation
engine Unavailable	Show popular items instead	Analytics service Unavailable
Queue events for later processing	Notification provider Unavailable	Store notification for retry
Search index Unavailable	Use database-backed fallback where acceptable	Production Tip

LearnForge • Engineering Handbook • 2026 37 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

Define degraded modes before production incidents occur. During an outage, engineers should know which features can be disabled and which business operations must remain available.

15. Distributed Transactions

Traditional database transactions are powerful because they provide atomic operations within a controlled transactional boundary. Once a business operation crosses multiple independent services, maintaining one global transaction becomes significantly more difficult. Instead of forcing every service into one distributed transaction, many systems use patterns such as sagas, compensating actions, and event-driven workflows.

Figure 8.4 — A saga coordinates a multi-step business workflow using individual local transactions and explicit recovery actions.

16. Observability in Distributed Systems

A distributed system cannot be operated effectively through logs from one application alone. Engineers need correlated telemetry across service boundaries.

- Logs: detailed records of application events.
- Metrics: numerical measurements such as latency and error rate.
- Traces: a view of one request as it crosses multiple services.
- Correlation IDs: identifiers that connect

related operations. { "traceId": "7f31c2a9", "service": "payment-service", "operation": "charge", "durationMs": 184, "status": "success" } The goal is not to collect telemetry for its own sake. The goal is to answer operational questions quickly: What failed? Where did latency increase? Which dependency caused the failure? How many users are affected? LearnForge • Engineering Handbook • 2026 38 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE 17. Practical Distributed-System Checklist • Does every remote dependency have a bounded timeout? • Are retries limited and protected by backoff? • Are state-changing operations idempotent where necessary? • Can duplicate messages be processed safely? • Is backpressure applied to overloaded components? • Are critical dependencies protected from cascading failures? • Are degraded modes explicitly designed? • Is distributed tracing available for important workflows? • Are service-level objectives defined for critical operations? • Can the team determine whether a failed request actually completed? 18. Key Takeaways • Distributed systems introduce uncertainty that does not exist in simple in-process calls. • Partial failure must be treated as a normal architectural condition. • Timeouts, retries, backoff, and idempotency are fundamental reliability tools. • At-least-once delivery requires consumers that can safely handle duplicates. • Backpressure and load shedding protect systems from resource exhaustion. • Graceful degradation allows critical business capabilities to survive dependency failures. • Distributed workflows often require explicit compensation rather than one global transaction. • Observability must cross service boundaries. LearnForge • Engineering Handbook • 2026 39 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE 19. Quick Review 1. Why is a remote function call fundamentally different from an in-process function call? 2. What is partial failure? 3. Why can unrestricted retries make an outage worse? 4. What problem does idempotency solve? 5. What is the purpose of a circuit breaker? 6. Why is backpressure important? 7. When might eventual consistency be acceptable? 8. Why are distributed transactions difficult? 20. Architect's Checklist • Have all remote dependencies been identified? • Does each critical dependency have a timeout and failure policy? • Are retryable and non-retryable failures clearly distinguished? • Are idempotency and deduplication mechanisms documented? • Have cascading-failure scenarios been tested? • Is there a defined degraded mode for critical dependencies? • Are distributed workflows observable end-to-end? • Are recovery and compensation procedures documented? Production Tip The best distributed architecture is not the one with the most services. It is the one whose failure behavior is understandable, measurable, and recoverable. LearnForge • Engineering Handbook • 2026 40 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE Chapter 09 Event-Driven Architecture Event-driven architecture allows systems to communicate through facts and asynchronous messages, reducing direct coupling while improving scalability, resilience, and independent evolution. What You Will Learn • How events differ from commands and ordinary synchronous requests. • How producers, consumers, queues, topics, and brokers work together. • Why asynchronous communication can

improve resilience and scalability. • How to design idempotent consumers and handle duplicate delivery. • How event schemas, ordering, retries, and dead-letter queues affect production systems. • How the Outbox Pattern helps prevent database-to-message consistency problems.

1. What Is Event-Driven Architecture?

Event-driven architecture (EDA) is a style in which components communicate by producing and consuming events. An event represents something that has already happened in the system, such as an order being created, a payment being completed, or a course being published. Instead of requiring the producer to know every component that needs the information, the producer publishes an event and interested consumers can process it independently.

Architect's Rule An event should describe a fact that has occurred. Consumers should decide what that fact means for their own business responsibility rather than forcing the producer to coordinate every downstream action.

2. Events vs Commands

Events and commands are both messages, but their meaning is different. A command asks another component to perform an action. An event announces that an action has already occurred.

LearnForge • Engineering Handbook • 2026 41 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

Concept	Meaning	Example
Command	Request to perform an action	CreateOrder
Event	Fact that an action occurred	OrderCreated
Query	Request for information	GetOrder

Commands are generally directed toward a responsible component. Events can be consumed by multiple independent components.

3. Basic Event Flow

Figure 9.1 — A producer publishes an event while independent consumers process the event for their own responsibilities.

4. Why Use Events?

The primary architectural benefit is decoupling. A producer does not need to know exactly which consumers exist or how they implement their processing. This allows new consumers to be added without changing the original producer. For example, when an order is created, the notification system may send an email, the analytics system may record an event, and the inventory system may reserve stock.

Production Tip Use events when downstream work can happen independently and does not need to block the original request.

5. Queues and Topics

A queue generally distributes work among competing consumers. A topic or publish-subscribe channel allows multiple independent subscribers to receive the same logical event.

LearnForge • Engineering Handbook • 2026 42 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

Pattern	Typical Behavior	Useful For
Queue	One worker usually processes each message	Background jobs
Pub/Sub	Multiple subscribers receive an event	Notifications and integrations
Stream	Ordered sequence of records	Event processing and analytics

6. Asynchronous Processing

Asynchronous processing allows an application to acknowledge a request without waiting for every downstream operation to finish.

```
await eventBus.publish({ type: "ORDER_CREATED", eventId: crypto.randomUUID(), orderId, occurredAt: new Date().toISOString() }); return { status: "accepted", orderId };
```

The consumer can process the event later. This reduces synchronous dependency chains and can improve the responsiveness of the user-facing application.

7. At-Least-Once Delivery

Many production messaging systems provide

at-least-once delivery. This means a message should not be lost under normal failure conditions, but a consumer may receive the same message more than once. Therefore, consumers must be designed to tolerate duplicate delivery. Exactly-once behavior is difficult to guarantee across independent systems, so application-level idempotency remains important. Common Mistake LearnForge • Engineering Handbook • 2026 43 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

Assuming that a message will always be delivered exactly once. A consumer that cannot safely process duplicates can create duplicate payments, notifications, records, or other side effects. 8. Idempotent Consumers An idempotent consumer detects messages that have already been processed and avoids repeating the intended side effect.

```

async function handleOrderCreated(event: OrderCreatedEvent)
{
  const alreadyProcessed = await processedEvents.exists(event.eventId);
  if (alreadyProcessed) {
    return;
  }
  await notificationService.sendOrderConfirmation(event.orderId);
  await processedEvents.record(event.eventId);
}

```

The exact implementation depends on the system. A database table, unique constraint, inbox pattern, or transactional processing mechanism can be used to record processed event identifiers. Architect's Rule Design consumers assuming that duplicate events are possible. Idempotency should be part of the normal architecture, not an emergency patch. 9. Event Ordering Some business workflows depend on events being processed in a meaningful order. For example, an OrderCreated event should normally be known before an OrderCancelled event is applied. Ordering is not automatically guaranteed across distributed consumers. Different partitions, workers, retries, and network conditions can cause events to arrive in an unexpected sequence. LearnForge • Engineering Handbook • 2026 44 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

Problem Possible Cause Response Events arrive out of order Parallel processing Partitioning or sequence numbers Duplicate events At-least-once delivery Idempotent consumer Missing event Producer or broker failure Replay and reconciliation Late event Network or consumer delay Version checks and timestamps Architect's Rule Do not require global ordering unless the business actually needs it. Global ordering can significantly reduce scalability and increase coordination costs. 10. Event Identifiers and Metadata Production events should contain enough metadata to support tracing, deduplication, debugging, and evolution. A globally unique event identifier is particularly useful when the same message can be delivered more than once.

```

interface DomainEvent {
  eventId: string;
  eventType: string;
  occurredAt: string;
  aggregateId: string;
  correlationId: string;
  version: number;
  payload: unknown;
}

```

The correlationId can connect several related operations, while the eventId identifies one specific event. Keeping these concepts separate makes distributed troubleshooting much easier. 11. Event Schemas An event is an API between producers and consumers. Its schema should therefore be treated as a public contract within the organization. LearnForge • Engineering Handbook • 2026 45 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

Changing an event

without considering existing consumers can break production workloads. Producers should prefer backward-compatible changes whenever possible. Change Compatibility Add an optional field Usually backward compatible Remove a required field Potentially breaking Rename an existing field Potentially breaking Change field meaning Highly risky Change data type Potentially breaking Production Tip Treat important event schemas like API contracts. Document ownership, compatibility rules, required fields, optional fields, and versioning policy.

12. Event Versioning

Event schemas evolve as business requirements change. A versioning strategy allows producers and consumers to evolve without requiring an immediate system-wide migration.

```
{ "eventType": "ORDER_CREATED", "version": 2, "eventId": "evt_91af",
  "aggregateId": "order_4821", "occurredAt": "2026-08-22T08:30:00Z", "payload":
  { "orderId": "order_4821", "customerId": "customer_71", "currency": "USD" } }
```

Version numbers should represent meaningful contract changes rather than being increased arbitrarily. Consumers should have a clear migration path when older versions are eventually retired. LearnForge • Engineering Handbook • 2026 46

LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE 13. Dead-Letter Queues

Some messages cannot be successfully processed even after reasonable retry attempts. A dead-letter queue (DLQ) provides a controlled destination for messages that require investigation or later recovery. Figure 9.2 — Messages that repeatedly fail processing can be isolated in a dead-letter queue for investigation and controlled recovery. A DLQ should not become a permanent garbage container. Operations teams need monitoring, alerting, ownership, and a documented process for inspecting and reprocessing failed messages.

14. Retry Strategy for Consumers

Consumer retries should distinguish between transient failures and permanent failures. Temporary database connectivity problems may justify a retry, while malformed event data may require immediate isolation.

```
async function processMessage(message: Message) { try { await handle(message); await
  message.ack(); } catch (error) { if (isRetryable(error)) { await
  message.retryWithBackoff(); } else { await
  message.moveToDeadLetterQueue(); } } }
```

Common Mistake Retrying malformed messages indefinitely. A permanently invalid message can consume worker capacity and delay healthy messages.

15. The Outbox Pattern

A common consistency problem occurs when an application updates a database and publishes an event as two separate operations. LearnForge • Engineering Handbook • 2026 47

LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

For example, the database transaction may succeed while the message publication fails. The business state then says that an order was created, but downstream consumers never receive the corresponding event. Figure 9.3 — The transactional outbox stores business state and the event record in the same database transaction before a publisher sends the event. The Outbox Pattern stores the event in the same database transaction as the business change. A separate publisher then reads pending outbox records and publishes them to the message broker. Architect's Rule When database state and event publication

must remain consistent, consider the transactional outbox instead of pretending that two independent operations form one atomic transaction.

16. Eventual Consistency

Asynchronous systems often introduce eventual consistency. Different components may temporarily observe different states while events are being processed. This is not necessarily a defect. It is an architectural trade-off that can provide better availability, scalability, and independence.

Situation

Eventual Consistency Acceptable?

Analytics dashboards Usually yes

Search indexing Usually yes

Email notifications Usually yes

Critical account balance Often requires stronger guarantees

Payment authorization Usually requires explicit consistency controls

LearnForge • Engineering Handbook • 2026 48

LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

17. Event Replay

One important advantage of durable event streams is the possibility of replaying historical events. A new consumer can rebuild derived state by processing events from an earlier point in the stream. Replay requires careful design. Consumers must be deterministic where possible, idempotent, and able to handle historical event versions.

Production Tip

Define retention and replay policies before relying on events as a source of historical truth. Test rebuilding important projections from stored events.

18. Event-Driven Observability

Asynchronous systems can be harder to debug because the original request may finish before downstream processing occurs. A production system therefore needs enough telemetry to connect the original operation with the events and consumer actions that follow it.

- Event ID: uniquely identifies one event.
- Correlation ID: connects related operations across services.
- Consumer metrics: show processing rate, failures, and latency.
- Queue depth: reveals whether work is accumulating.
- Dead-letter metrics: show messages requiring investigation.
- Trace context: connects asynchronous processing to the originating request.

```
{ "eventId": "evt_91af",
  "eventType": "ORDER_CREATED",
  "correlationId": "req_72bc",
  "consumer": "notification-service",
  "attempt": 2,
  "processingMs": 84,
  "status": "success" }
```

The objective is operational clarity. During an incident, engineers should be able to determine whether an event was published, delivered, processed, retried, or moved to a dead-letter queue.

LearnForge • Engineering Handbook • 2026 49

LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

19. Security and Event-Driven Systems

Events frequently contain business information that should not be exposed to every consumer. Event architecture therefore requires explicit access control and data-minimization decisions.

Security Concern

Practical Control

Unauthorized consumers

Topic or queue access policies

Sensitive event data

Minimize payload and protect sensitive fields

Message tampering

Authenticated transport and integrity controls

Compromised consumer

Least-privilege permissions

Replay abuse

Idempotency and authorization checks

Architect's Rule

Do not publish sensitive data simply because a future consumer might need it. Publish the minimum information required by the event contract and apply least-privilege access to consumers.

20. Choosing Events vs Synchronous APIs

Event-driven communication is not a replacement for every API call. Synchronous APIs remain appropriate when the

caller needs an immediate, authoritative response. Use Synchronous API When... Use an Event When... The caller needs an immediate result Work can happen asynchronously The operation requires direct validation Multiple independent consumers need the fact Strong immediate consistency is required Eventual consistency is acceptable The dependency relationship is explicit Loose coupling is valuable The request is interactive The work is background or integration-oriented

LearnForge • Engineering Handbook • 2026 50 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE Many production systems use both approaches. A request may use a synchronous API to create an order and then publish an event for asynchronous notifications, analytics, fulfillment, and other independent workflows.

21. Practical Event-Driven Design A useful design process starts with business facts rather than messaging technology. Identify meaningful events in the domain and then determine which systems need to react to them.

1. Identify the business action or fact.
2. Define the event name and ownership.
3. Define the event schema.
4. Identify producers and consumers.
5. Determine delivery and ordering requirements.
6. Design idempotency and retry behavior.
7. Define dead-letter and recovery procedures.
8. Instrument publication and consumption.
9. Document compatibility and versioning rules.

Figure 9.4 — A practical event-driven design process from business fact to event contract, delivery, consumption, and observability.

22. Production Checklist

- Are events based on meaningful business facts?
- Is ownership of each event clearly defined?
- Are event schemas documented and versioned?
- Can consumers safely process duplicate events?
- Are retryable and permanent failures distinguished?
- Is there a dead-letter strategy for failed messages?

LearnForge • Engineering Handbook • 2026 51 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

- Are ordering requirements explicitly documented?
- Is event publication reliable with respect to database changes?
- Is an Outbox Pattern required for critical workflows?
- Are queue depth, processing latency, and failure rates monitored?
- Are sensitive event fields minimized and protected?
- Can important events be replayed or reconciled when necessary?

23. Key Takeaways

- Events represent facts that have already occurred.
- Commands request actions; events communicate completed facts.
- Asynchronous communication can reduce direct service coupling.
- At-least-once delivery means duplicate processing must be expected.
- Idempotent consumers are fundamental to reliable event processing.
- Event schemas should be treated as contracts.
- Dead-letter queues isolate messages that cannot be processed normally.
- The Outbox Pattern helps keep database changes and event publication consistent.
- Eventual consistency is often a deliberate trade-off rather than a defect.
- Observability must connect event publication with downstream processing.
- Events and synchronous APIs are complementary architectural tools.

24. Quick Review

1. What is the difference between an event and a command?
2. Why can asynchronous communication reduce coupling?
3. What does at-least-once delivery imply for consumers?
4. Why is idempotency important in event-driven systems?
5. What problem does a dead-letter

queue solve? 6. Why should event schemas be treated as contracts? LearnForge • Engineering Handbook • 2026 52 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE 7. What is the purpose of event versioning? 8. How does the Outbox Pattern improve reliability? 9. When is eventual consistency acceptable? 10. When should a synchronous API be preferred over an event? 25. Architect's Checklist • Have business events been identified before selecting messaging technology? • Does every important event have an owner? • Are event contracts documented? • Are consumers designed for duplicates? • Are retries bounded and protected by backoff? • Is there a recovery process for dead-lettered messages? • Are ordering assumptions explicit? • Are database-to-event consistency risks addressed? • Are event flows observable end to end? • Has the team tested consumer failure and replay scenarios? Final Architecture Principle Use event-driven architecture to create meaningful independence, not merely to introduce a message broker. Good event-driven design makes business facts clear, boundaries explicit, failures manageable, and system evolution safer. LearnForge • Engineering Handbook • 2026 53 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE Chapter 10 Clean Architecture Clean Architecture is a way of organizing software so that business rules remain independent from frameworks, databases, user interfaces, and other volatile implementation details. Its central goal is not a particular folder structure, but controlled dependency direction and long-term maintainability. What You Will Learn • What Clean Architecture actually solves. • How entities, use cases, interface adapters, and infrastructure relate to one another. • Why the Dependency Rule is the foundation of the architecture. • How dependency inversion protects business logic from technical details. • How to apply Clean Architecture without creating unnecessary abstraction. • How to structure a practical application around use cases and boundaries. 1. What Clean Architecture Means Clean Architecture is an architectural approach that attempts to keep the core business rules of an application independent from external technologies. The business logic should not need to know whether the application uses PostgreSQL, MongoDB, REST, GraphQL, React, Android, a particular web framework, or a specific cloud provider. The purpose is to make important parts of the system easier to understand, test, change, and protect from infrastructure churn. Architect's Rule Business rules should not depend directly on technical details. Technical details should adapt themselves to the business rules. 2. The Core Problem: Dependency Direction The most important idea in Clean Architecture is not the number of folders or the names of layers. It is the direction in which dependencies point. LearnForge • Engineering Handbook • 2026 54 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE If business logic directly imports a database library, HTTP framework, or external SDK, a technical decision has become part of the business core. Replacing that technology then becomes more expensive. Design Dependency Direction Typical Result Business → Database Core depends on infrastructure Higher coupling Business → HTTP framework Core depends on delivery mechanism Harder testing Infrastructure →

Business abstraction Implementation depends inward Lower coupling UI → Use Case Presentation depends on application logic Clear boundary

3. The Dependency Rule

The Dependency Rule states that source-code dependencies should point toward the architectural center. Inner layers should not know about the details of outer layers. Figure 10.1 — Clean Architecture organizes dependencies toward the business core rather than allowing infrastructure to control the core.

4. The Four Major Areas

A practical Clean Architecture implementation is commonly described using four broad areas: entities, use cases, interface adapters, and frameworks or infrastructure.

Area Responsibility Examples

Area	Responsibility	Examples
Entities	Core business rules	Order, Account, Course
Use Cases	Application-specific business workflows	CreateOrder, PublishCourse
Interface Adapters	Convert data between external and internal representations	gateways
Frameworks or Infrastructure	Technical implementation details	SDKs, LearnForge

• Engineering Handbook • 2026 55

LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

5. Entities

Entities represent important business concepts and rules that are expected to remain meaningful even if the application's delivery mechanism changes. An entity should not need to know how it is stored or how a request reached the application.

```
class Course {
  constructor( readonly id: string, private title: string, private published: boolean = false ) {}
  publish(): void {
    if (this.title.trim().length < 3) {
      throw new Error("A course requires a valid title");
    }
    this.published = true;
  }
  isPublished(): boolean {
    return this.published;
  }
}
```

The entity contains a business rule: a course cannot be published unless it has a meaningful title. Notice that there is no database query, HTTP request, or framework-specific code inside the entity.

Production Tip Keep domain objects focused on business behavior. Avoid turning entities into passive data containers when meaningful invariants belong inside them.

6. Use Cases

Use cases describe application-specific actions. They coordinate business objects and required dependencies to accomplish a meaningful goal. Examples include registering a user, publishing a course, creating an order, processing a refund, or generating an invoice.

• Engineering Handbook • 2026 56

LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

interface CourseRepository

```
{
  findById(id: string): Promise;
  save(course: Course): Promise;
}
```

```
class PublishCourse {
  constructor( private readonly courses: CourseRepository ) {}
  async execute(courseId: string): Promise {
    const course = await this.courses.findById(courseId);
    if (!course) {
      throw new Error("Course not found");
    }
    course.publish();
    await this.courses.save(course);
  }
}
```

The use case depends on a repository abstraction rather than a concrete database implementation. This allows the business workflow to remain independent from the storage technology.

7. Dependency Inversion

Dependency inversion is one of the most powerful techniques used to achieve the Dependency Rule. Instead of making the business layer depend directly on an infrastructure implementation, both sides communicate through an abstraction owned by the appropriate inner layer. Figure 10.2 — Dependency inversion keeps infrastructure replaceable behind a stable application-owned

boundary. 8. Interface Adapters Interface adapters translate between the outside world and the application's internal model. They prevent external representations from spreading throughout the business core. For example, an HTTP controller can translate a JSON request into a use-case input object. A presenter can transform the use-case result into an HTTP response. LearnForge • Engineering Handbook • 2026

57 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE type
 PublishCourseRequest = { courseId: string; }; class CourseController
 { constructor(private readonly publishCourse: PublishCourse) {} async
 handle(request: PublishCourseRequest) { await
 this.publishCourse.execute(request.courseId); return { status: 204 }; } } The
 controller knows about HTTP-oriented concerns, but the use case does not need to
 know that the request came from HTTP. 9. Frameworks Are Details Frameworks are
 useful, but they should not automatically become the center of the architecture. A
 framework can change, be replaced, or become obsolete while the business problem
 remains. Technical Detail Possible Change Protected Core Database PostgreSQL →
 another database Business rules Web framework Express → another framework Use
 cases Transport REST → gRPC Application behavior Queue provider One broker →
 another broker Domain events Common Mistake Building business logic directly
 inside controllers, ORM models, request handlers, or framework callbacks. This
 makes the business rules harder to test and harder to reuse. LearnForge •
 Engineering Handbook • 2026 58 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

10. Repository Abstractions A repository abstraction represents the storage operations required by the application. The abstraction should express what the business needs rather than exposing every capability of a particular database.

```
interface UserRepository { findById(id: string): Promise; findByEmail(email: string): Promise; save(user: User): Promise; }
```

A PostgreSQL implementation can satisfy this contract without forcing the application layer to know SQL details.

```
class PostgresUserRepository implements UserRepository { constructor(private readonly db: Database) {} async findById(id: string): Promise { const row = await this.db.queryOne( "SELECT * FROM users WHERE id = $1", [id] ); return row ? User.fromPersistence(row) : null; } async save(user: User): Promise { await this.db.execute( "UPDATE users SET name = $1 WHERE id = $2", [user.name, user.id] ); } }
```

11. Testing the Architecture Clean Architecture can make testing easier because important business behavior can be tested without starting the entire infrastructure stack. A use case can receive a fake repository and execute entirely in memory. This makes tests faster and helps isolate business behavior from database configuration. LearnForge • Engineering Handbook • 2026 59 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

```
class InMemoryCourseRepository implements CourseRepository { private items = new Map(); async findById(id: string) { return this.items.get(id) ?? null; } async save(course: Course) { this.items.set(course.id, course); } }
```

Test Type Primary Target Typical Speed Unit test Entity or isolated use case Very fast Integration test Adapter + real

infrastructure Moderate End-to-end test Complete user workflow Slower 12. Clean Architecture Is Not a Folder Structure A common misunderstanding is to believe that Clean Architecture means creating directories named entities, usecases, controllers, repositories, and infrastructure. Folders can help communicate boundaries, but they do not create architectural quality by themselves. An application can have perfectly named folders while its business logic still depends directly on frameworks and databases. Architect's Rule Architecture is defined by dependency relationships and boundaries, not by directory names. 13. Avoiding Over-Engineering Clean Architecture can be misapplied. Adding interfaces, factories, repositories, mappers, and abstractions for every small operation can create more complexity than it removes. LearnForge • Engineering Handbook • 2026 60 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE The goal is not maximum abstraction. The goal is useful separation around areas that are likely to change or that contain important business rules. Situation Practical Approach Simple CRUD application Prefer straightforward design Complex business rules Protect the domain carefully Frequently changing infrastructure Use stable abstractions at boundaries Highly regulated business logic Prioritize explicit rules and testability 14. Practical Project Structure A possible structure for a medium-sized TypeScript application is: src/ domain/ entities/ value-objects/ application/ use-cases/ ports/ adapters/ http/ persistence/ infrastructure/ database/ messaging/ config/ main.ts The exact directory names are not mandatory. What matters is that the dependency direction remains intentional and the core business logic is protected from external details. LearnForge • Engineering Handbook • 2026 61 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE 15. Dependency Injection Dependency injection means providing a component with the dependencies it needs rather than forcing it to construct those dependencies internally. `const repository = new PostgresCourseRepository(db); const publishCourse = new PublishCourse(repository); const controller = new CourseController(publishCourse);` This composition root is an appropriate place to connect concrete infrastructure implementations to application abstractions. Production Tip Keep object composition near the application's entry point. Business components should receive their dependencies rather than discovering infrastructure globally. 16. Boundaries and Change Good architecture makes expensive changes local whenever possible. If changing the database requires rewriting business rules, the boundary is weak. If changing an HTTP transport only requires replacing an adapter, the boundary is stronger. Change Weak Boundary Strong Boundary Database replacement Business code changes extensively Replace adapter API transport change Use cases change Replace controller/adapter UI replacement Business logic rewritten Use cases remain stable Queue provider change Domain code changes Replace infrastructure adapter 17. Clean Architecture in a Real Workflow Consider an online learning platform where an administrator publishes a course. LearnForge • Engineering Handbook • 2026 62 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE Figure 10.3 — A practical request flow through

adapters, application logic, and infrastructure.

18. Common Failure Modes

Failure Mode Why It Hurts Better Approach

- Rules become tied to Business logic** in controllers
- Move workflow** into use cases
- transport ORM models** used as domain
- Persistence concerns leak** Separate domain and persistence models
- entities inward** when justified
- Interface for every class** Abstraction becomes noise
- Abstract meaningful boundaries** Dependencies become Global service locator
- Prefer explicit dependency injection** hidden Business intent becomes Overly generic repositories
- Use capability-focused interfaces** unclear

Common Mistake

Assuming that more layers automatically mean better architecture. Excessive indirection can make a simple system harder to understand. Architecture should reduce complexity, not merely relocate it.

19. Practical Design Rules

- Keep important business rules independent from infrastructure.
- Make dependency direction explicit.
- Put application workflows inside use cases.
- Keep controllers thin.
- Use abstractions where they protect meaningful boundaries.
- Keep infrastructure implementations outside the business core.

LearnForge • Engineering Handbook • 2026 63

LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

- Prefer explicit dependency injection.
- Test business rules without requiring the full infrastructure stack.
- Avoid abstraction whose only purpose is satisfying a pattern.
- Design boundaries around change and business responsibility.

20. Key Takeaways

- Clean Architecture is fundamentally about dependency direction.
- The business core should remain independent from technical details.
- Entities contain important business rules and invariants.
- Use cases coordinate application-specific workflows.
- Adapters translate between external systems and internal models.
- Dependency inversion makes infrastructure replaceable.
- Architecture should make important changes cheaper and more localized.
- Clean Architecture is not synonymous with a particular folder structure.
- Too much abstraction can be as harmful as too little.

21. Quick Review

1. What is the Dependency Rule?
2. Why should business logic avoid direct database dependencies?
3. What is the difference between an entity and a use case?
4. What role does an interface adapter play?
5. How does dependency inversion reduce coupling?
6. Why should controllers generally remain thin?
7. When can Clean Architecture become over-engineering?
8. How does architecture reduce the cost of change?

22. Architect's Checklist

LearnForge • Engineering Handbook • 2026 64

LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

- Can the core business rules be tested without the database?
- Can the transport mechanism be changed without rewriting use cases?
- Are infrastructure dependencies pointing toward stable abstractions?
- Are application workflows represented explicitly?
- Are business invariants protected by the domain model?
- Are external data formats prevented from leaking into the domain?
- Is dependency injection used where it improves testability and control?
- Have unnecessary abstractions been removed?
- Can the team explain the dependency direction in one diagram?

23. Final Principle

Architecture Principle

The purpose of Clean Architecture is not to make code look sophisticated. Its purpose is to protect the decisions that matter most: business rules,

use cases, and system behavior. Frameworks, databases, transports, and vendors are important implementation details, but they should not own the architecture.

LearnForge • Engineering Handbook • 2026 65 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

Chapter 11 SOLID Principles

SOLID principles provide practical guidelines for organizing object-oriented code so that systems remain understandable, extensible, testable, and resistant to unnecessary change.

What You Will Learn • What each SOLID principle means in practical engineering. • How SOLID reduces coupling and improves maintainability. • How to recognize common violations in real code. • When abstraction helps and when it becomes unnecessary complexity. • How SOLID principles connect with clean architecture and testing.

1. What SOLID Really Means

SOLID is a collection of five object-oriented design principles associated with maintainable software design. The principles are not rigid laws and they do not require every class to have an interface or every application to use complex inheritance. Their real purpose is to help engineers control dependencies, isolate change, keep responsibilities understandable, and make software easier to test and extend.

Principle Core Idea Main Benefit

- SRP One reason to change
- Focused responsibilities
- OCF Extend behavior without unnecessary modification
- Safer evolution
- LSP Subtypes must remain substitutable
- Predictable polymorphism
- ISP Prefer focused interfaces
- Lower dependency burden
- DIP Depend on abstractions at architectural boundaries
- Reduced coupling

Architect's Rule

SOLID is valuable when it reduces the cost of change. Applying SOLID mechanically can create unnecessary abstractions and make simple systems harder to understand.

LearnForge • Engineering Handbook • 2026 66 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

2. Single Responsibility Principle

The Single Responsibility Principle, or SRP, states that a module should have one clear responsibility and one primary reason to change. A common misunderstanding is that SRP means a class must contain only one method. That is not the intention. A class may contain many related operations while still representing one coherent responsibility.

2.1 A Typical Violation

```
class UserService {
    createUser(data: UserData) { // create database record }
    sendWelcomeEmail(user: User) { // send email }
    generatePdfReport(user: User) { // create PDF }
    writeAuditLog(user: User) { // write audit record }
}
```

This class has accumulated several unrelated responsibilities. Changes to email delivery, PDF generation, persistence, or auditing can all force changes to the same class.

LearnForge • Engineering Handbook • 2026 67 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

2.2 A More Focused Design

```
class UserService {
    constructor( private readonly repository: UserRepository ) {}
    createUser(data: UserData) { return this.repository.save(data); }
}
class EmailService {
    sendWelcomeEmail(user: User) { // email responsibility }
}
class AuditService {
    recordUserCreated(user: User) { // audit responsibility }
}
```

Figure 11.1 — SRP encourages focused responsibilities instead of one class becoming the owner of unrelated concerns.

3. Open/Closed Principle

The Open/Closed Principle states that software entities should generally be

open for extension but closed for unnecessary modification. The practical idea is to design stable code so that new behavior can often be introduced through new implementations rather than repeatedly modifying a large conditional structure.

LearnForge • Engineering Handbook • 2026 68 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE 3.1 Conditional Explosion

```
function calculateDiscount( customerType: string, amount: number ) {
  if (customerType === "regular") { return amount * 0.05; }
  if (customerType === "premium") { return amount * 0.15; }
  if (customerType === "enterprise") { return amount * 0.25; }
  return 0; }

```

Adding more customer types requires modifying the same function repeatedly. A strategy- based design can isolate each pricing rule.

LearnForge • Engineering Handbook • 2026 69 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

```
interface DiscountPolicy { calculate(amount: number): number; }
class RegularDiscount implements DiscountPolicy {
  calculate(amount: number) { return amount * 0.05; }
}
class PremiumDiscount implements DiscountPolicy {
  calculate(amount: number) { return amount * 0.15; }
}
class EnterpriseDiscount implements DiscountPolicy {
  calculate(amount: number) { return amount * 0.25; }
}
function calculateDiscount( policy: DiscountPolicy, amount: number ) {
  return policy.calculate(amount); }

```

Production Tip Do not create abstractions for every possible future requirement. Use OCP where variation is already visible or likely enough to justify the added design complexity.

4. Liskov Substitution Principle

The Liskov Substitution Principle, or LSP, states that objects of a subtype should be usable wherever objects of the parent abstraction are expected without breaking the correctness assumptions of the program. Inheritance is not automatically safe merely because two classes appear related. A subtype must preserve the behavioral expectations established by the abstraction.

LearnForge • Engineering Handbook • 2026 70 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE 4.1 The Classic Problem

```
interface Bird { fly(): void; }
class Eagle implements Bird { fly() { console.log("Flying"); } }
class Penguin implements Bird { fly() { throw new Error("Penguins cannot fly"); } }

```

The abstraction incorrectly assumes that every Bird can fly. Penguin is biologically a bird, but the chosen software abstraction does not represent the behavior safely.

4.2 Better Abstraction

```
interface Bird { eat(): void; }
interface FlyingBird extends Bird { fly(): void; }
class Penguin implements Bird {
  eat() { console.log("Eating"); }
}
class Eagle implements FlyingBird {
  eat() { console.log("Eating"); }
  fly() { console.log("Flying"); }
}

```

LearnForge • Engineering Handbook • 2026 71 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

Common Mistake

Using inheritance merely because two concepts are related in the real world. Software abstractions should model the behavior required by the application, not just taxonomy.

5. Interface Segregation Principle

The Interface Segregation Principle states that clients should not be forced to depend on methods they do not use. Large interfaces often create unnecessary coupling. When one method changes, many unrelated implementations may need to change simply because they depend on the same oversized contract.

Large Interface Focused

Interfaces Printer, scanner, fax, storage Printable, Scannable, Faxable, Storable Many unused methods Only required capabilities Higher implementation burden Lower implementation burden LearnForge • Engineering Handbook • 2026 72

LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE interface Printable { print(document: Document): void; } interface Scannable { scan(): Document; } class SimplePrinter implements Printable { print(document: Document) { console.log("Printing"); } } class MultiFunctionPrinter implements Printable, Scannable { print(document: Document) { console.log("Printing"); } scan() { return new Document(); } } Architect's Rule Prefer interfaces that represent meaningful capabilities. A small interface is not automatically better, but a focused contract usually reduces unnecessary dependency.

6. Dependency Inversion Principle

The Dependency Inversion Principle states that high-level policy should not depend directly on low-level implementation details. Both should depend on stable abstractions. This principle is especially important at architectural boundaries such as databases, external APIs, message brokers, payment providers, and file systems.

LearnForge • Engineering Handbook • 2026 73

LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

6.1 Tight Coupling

class OrderService { private database = new MySQLDatabase(); save(order: Order) { this.database.insert(order); } }

The high-level order logic is directly coupled to a specific database implementation.

6.2 Dependency Inversion

interface OrderRepository { save(order: Order): Promise; } class OrderService { constructor(private readonly repository: OrderRepository) {} save(order: Order) { return this.repository.save(order); } } class MySQLOrderRepository implements OrderRepository { async save(order: Order) { // MySQL implementation } }

Figure 11.2 — Dependency inversion places a stable abstraction between high-level business policy and infrastructure details.

7. SOLID and Testability

Well-designed dependencies make unit testing easier because external systems can be replaced with controlled test implementations.

LearnForge • Engineering Handbook • 2026 74

LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

class FakeOrderRepository implements OrderRepository { public orders: Order[] = []; async save(order: Order) { this.orders.push(order); } }

test("saves an order", async () => { const repository = new FakeOrderRepository(); const service = new OrderService(repository); await service.save({ id: "order-1", total: 100 }); expect(repository.orders).toHaveLength(1); });

The test does not require a real database. This makes the test faster, deterministic, and easier to reason about.

8. SOLID and Clean Architecture

SOLID principles and clean architecture complement each other. SOLID helps organize dependencies inside components, while architectural boundaries determine which components should depend on which others.

Concept Relationship to SOLID Entities Focused business responsibilities Use cases Stable high-level policies Repositories Dependency inversion at persistence boundaries Controllers Focused delivery responsibilities Infrastructure Low-level implementations behind abstractions

LearnForge • Engineering Handbook • 2026 75

LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE 9. Composition Over Inheritance SOLID design frequently leads toward composition rather than deep inheritance hierarchies. Composition allows behavior to be assembled from smaller components without forcing every variation into a rigid class hierarchy.

```
interface PaymentGateway { charge(amount: number): Promise; }
class CheckoutService {
  constructor( private readonly payment: PaymentGateway ) {}
  async checkout(amount: number) { await this.payment.charge(amount); } }

```

The checkout service does not need to know whether the payment implementation uses Stripe, another provider, a local simulator, or a test double.

10. Avoiding Abstraction Overload SOLID principles can be misused. Creating an interface, factory, adapter, repository, and strategy for every small function can make a codebase harder to understand than the original implementation.

Useful Abstraction Suspicious Abstraction Protects a real architectural boundary Exists only because "SOLID says so" Encapsulates meaningful variation Hides a trivial function Improves testing or replacement Adds layers without benefit Clarifies ownership Makes navigation harder

Common Mistake LearnForge • Engineering Handbook • 2026 76

LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE Confusing more abstractions with better architecture. Good architecture minimizes unnecessary complexity while protecting important boundaries.

11. A Practical SOLID Refactoring Process

1. Identify the code that changes frequently.
2. Find responsibilities that are unrelated to one another.
3. Identify dependencies on unstable infrastructure.
4. Extract focused responsibilities where the boundary is meaningful.
5. Introduce abstractions only where they provide real flexibility.
6. Write tests around the behavior being preserved.
7. Refactor incrementally rather than rewriting the entire system.

12. SOLID Smell Detection

Possible Code Smell	Potential Response Principle
Huge class with unrelated responsibilities	SRP Split by responsibility
Consider strategy or Growing conditional branches	OCPP polymorphism
Subtype throws unsupported-operation errors	LSP Redesign the abstraction
Implementations depend on unused methods	ISP Split interfaces
Business logic directly creates infrastructure	DIP Invert the dependency objects

13. SOLID in a Real Application Consider an e-commerce checkout system. The checkout use case should coordinate business decisions without being responsible for SQL queries, HTTP transport, email delivery, or payment-provider-specific details.

LearnForge • Engineering Handbook • 2026 77

LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

```
class CheckoutUseCase {
  constructor( private readonly orders: OrderRepository, private readonly payments: PaymentGateway, private readonly notifications: NotificationService ) {}
  async execute(input: CheckoutInput) {
    const order = await this.orders.create(input);
    await this.payments.charge( order.total, order.paymentReference );
    await this.notifications.sendConfirmation(order);
    return order;
  }
}

```

Each dependency has a clear responsibility. The use case coordinates the business workflow while infrastructure implementations remain replaceable.

14. SOLID Trade-offs

Benefit	Possible Cost
Lower coupling	More interfaces
Better testing	More dependency

configuration Easier extension More indirection Clearer responsibilities More classes or modules Replaceable infrastructure Additional architectural discipline The goal is not maximum abstraction. The goal is an appropriate balance between flexibility and simplicity. LearnForge • Engineering Handbook • 2026 78 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE 15. SOLID Design Heuristics Production Heuristics

- Prefer small, coherent responsibilities.
- Keep business rules independent from infrastructure where practical.
- Use interfaces at meaningful boundaries rather than everywhere.
- Prefer composition when inheritance creates behavioral constraints.
- Test behavior before and after significant refactoring.
- Remove abstractions that no longer provide value.

16. SOLID Architecture Diagram Figure 11.2 — SOLID principles work together to improve responsibility boundaries, dependency direction, substitutability, and maintainability. 17. Review Questions

1. What does "one reason to change" mean in SRP?
2. Why can large conditional structures violate OCP?
3. What does LSP require from a subtype?
4. Why can large interfaces violate ISP?
5. What problem does dependency inversion solve?
6. Why can composition be safer than deep inheritance?
7. When can an abstraction become unnecessary complexity?
8. How does SOLID improve testability?

18. Practical Checklist

- Does each major component have a clear responsibility?
- Are frequently changing concerns isolated?
- LearnForge • Engineering Handbook • 2026 79 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE Can new behavior be added without repeatedly modifying stable logic?
- Are subtype contracts behaviorally safe?
- Are interfaces focused on meaningful capabilities?
- Does business logic avoid unnecessary infrastructure coupling?
- Are important dependencies replaceable in tests?
- Are abstractions justified by real requirements?
- Is composition preferred where inheritance would create constraints?
- Has unnecessary abstraction been removed?

19. Key Takeaways

- SRP keeps responsibilities focused and limits unrelated reasons for change.
- OCP encourages safe extension around stable behavior.
- LSP protects the behavioral correctness of polymorphism.
- ISP reduces unnecessary dependency on large contracts.
- DIP separates high-level policy from low-level implementation details.
- SOLID principles are tools for controlling change, not rules for maximizing abstraction.
- Composition often provides safer flexibility than deep inheritance.
- Good SOLID design improves testability, maintainability, and architectural clarity.

20. Architect's Final Checklist

- Are responsibilities clearly separated?
- Are unstable implementation details isolated behind useful boundaries?
- Can important components be tested without unnecessary infrastructure?
- Are interfaces small enough to represent meaningful capabilities?

LearnForge • Engineering Handbook • 2026 80 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

- Do abstractions represent actual variation rather than imagined futures?
- Can implementations be replaced without changing business policy?
- Are inheritance relationships behaviorally valid?
- Does the architecture remain understandable to a new engineer?

LearnForge • Engineering Handbook • 2026 81 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE Chapter 12 DRY, KISS, YAGNI, Cohesion and

Coupling Maintainable software is not created by adding more abstractions. It is created by controlling duplication, unnecessary complexity, premature features, and unhealthy dependencies while keeping related behavior together. What You Will Learn • What DRY actually means beyond simple code duplication. • How KISS helps engineers control accidental complexity. • Why YAGNI protects systems from speculative features. • How cohesion and coupling affect maintainability. • How to recognize harmful duplication and harmful abstraction. • How these principles work together during practical refactoring.

1. Why Simple Code Is Difficult

Software becomes difficult to maintain when every change requires engineers to understand too many unrelated concepts. Complexity accumulates through duplicated rules, unnecessary abstractions, speculative features, unclear responsibilities, and excessive dependencies. DRY, KISS, and YAGNI are lightweight design principles that help control this complexity. Cohesion and coupling provide a deeper architectural vocabulary for understanding why some designs remain easy to change while others become fragile. Architect's Rule Prefer the simplest design that correctly represents the current business requirements while keeping important boundaries clear.

2. DRY — Don't Repeat Yourself

DRY is commonly interpreted as “never write the same code twice.” That is too narrow. The deeper idea is that important knowledge should have a single authoritative representation when duplication would allow those representations to diverge.

LearnForge • Engineering Handbook • 2026 82

LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

Two pieces of code can look similar while representing different business rules. In that situation, forcing them into one abstraction may create coupling rather than eliminate meaningful duplication.

2.1 Obvious Duplication

```
function calculateRegularPrice(price: number) { return price * 1.2; }
function calculatePremiumPrice(price: number) { return price * 1.2; }
```

If the 20 percent calculation represents the same business rule, maintaining it twice creates a risk that one implementation will eventually change while the other remains unchanged.

2.2 Extracting Shared Knowledge

```
function addTax(price: number, taxRate = 0.2) { return price * (1 + taxRate); }
function calculateRegularPrice(price: number) { return addTax(price); }
function calculatePremiumPrice(price: number) { return addTax(price); }
```

The shared tax rule now has one implementation. A change to the tax calculation can be made in one place.

3. Accidental Duplication vs Coincidental Similarity

Not every repeated-looking structure should be abstracted. Two functions may currently contain similar code because they happen to perform similar work, while their business rules are expected to evolve independently.

LearnForge • Engineering Handbook • 2026 83

LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

Situation Recommended Approach

Situation	Recommended Approach
Same business rule repeated	Consider extracting shared logic
Same algorithm but different business meaning	Keep boundaries separate
Similar code with different future evolution	Avoid premature abstraction
Shared validation rule	Centralize when consistency matters

Common Mistake Treating every repeated line as a DRY violation. Removing textual duplication can create conceptual duplication

elsewhere if the abstraction does not represent the same underlying knowledge. 4. DRY at the Architectural Level DRY can also apply to configuration, schemas, business policies, and operational knowledge. For example, maintaining multiple independent definitions of the same API contract can cause clients and servers to drift apart. `type UserStatus = | "active" | "suspended" | "deleted"; interface User { id: string; status: UserStatus; }` The important principle is not simply “one file.” It is that the authoritative meaning of a concept should be clear and unnecessary competing definitions should be avoided. 5. KISS — Keep It Simple KISS encourages engineers to prefer understandable solutions over unnecessarily complicated ones. Simplicity does not mean ignoring real requirements. It means avoiding complexity that does not produce useful business or technical value. LearnForge • Engineering Handbook • 2026 84 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE Simple Design Overengineered Design Direct function call Multiple unnecessary abstraction layers One clear data structure Several wrappers with little value Simple validation Generic framework for one use case Explicit control flow Metaprogramming without clear need 5.1 Example function `isAdult(age: number): boolean { return age >= 18; }` For a simple rule, a direct function is often clearer than introducing a policy engine, factory, registry, and configuration framework. Architect's Rule Complexity should have a reason. If an abstraction cannot be explained in terms of a real requirement, architectural boundary, or recurring variation, question whether it belongs in the system. 6. Essential vs Accidental Complexity Essential complexity comes from the problem itself. If a payment system must support refunds, fraud checks, currency conversion, and regulatory rules, those requirements create unavoidable complexity. Accidental complexity comes from the way the solution is implemented. Unnecessary frameworks, duplicated transformations, excessive indirection, and unclear dependencies can make a system harder than the underlying business problem requires. Figure 12.1 — Good engineering cannot remove essential complexity, but it can reduce accidental complexity. LearnForge • Engineering Handbook • 2026 85 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE 7. YAGNI — You Aren't Gonna Need It YAGNI advises developers not to implement functionality simply because it might be needed in the future. Future requirements are uncertain, and premature features consume development time while increasing maintenance cost. 7.1 Speculative Design

```
class UserService {
  createUser(user: User) { // current requirement }
  // Future features that have no current requirement:
  importFromLegacySystem() // exportToTwentyDifferentFormats() //
  syncWithUnknownPartner() }

```

 Building unused functionality creates code that must be tested, documented, secured, and maintained even though it provides no current value. 7.2 Better Approach

```
class UserService {
  createUser(user: User) { // Implement the requirement that exists today. }
}

```

 When a real requirement appears, the system can be extended based on actual constraints rather than assumptions. Production Tip Design for change, but do not implement imaginary requirements. Stable boundaries are useful; speculative features are usually not. LearnForge •

Engineering Handbook • 2026 86 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

8. The Relationship Between KISS and YAGNI KISS and YAGNI reinforce each other. KISS asks whether the current solution is more complicated than necessary. YAGNI asks whether functionality exists only because someone predicts that it may be needed later. Question Principle Is the implementation more complicated than necessary? KISS Are we building a feature without a current requirement? YAGNI Are we maintaining the same knowledge in multiple places? DRY

9. Cohesion Cohesion describes how strongly the responsibilities within a module belong together. High cohesion generally means that the elements of a module contribute to one focused purpose. A highly cohesive module is easier to understand because engineers can reason about related behavior without navigating through unrelated functionality. High Cohesion Low Cohesion Payment module owns payment rules Payment module also generates reports User module manages identity behavior User module sends unrelated marketing campaigns Notification module manages delivery Notification module modifies billing records

10. Coupling Coupling describes the degree to which one component depends on another. Some coupling is unavoidable, but excessive or unstable coupling makes changes more expensive. The goal is not zero coupling. A useful system requires components to communicate. The goal is controlled coupling with clear ownership and stable contracts. LearnForge • Engineering Handbook • 2026 87 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

Figure 12.2 — High cohesion groups related responsibilities; excessive coupling allows changes to propagate unnecessarily.

11. Cohesion and Coupling Work Together A common design goal is high cohesion with appropriately low coupling. Related responsibilities should live together, while unrelated modules should communicate through explicit and stable boundaries.

```
interface PaymentGateway {
  charge(amount: number): Promise;
}
class CheckoutService {
  constructor( private readonly payment: PaymentGateway ) {}
  async execute(amount: number) {
    await this.payment.charge(amount);
  }
}

```

Checkout behavior is cohesive, while the concrete payment implementation is kept behind a narrow dependency.

12. The Cost of High Coupling When components are tightly coupled, a small change can create a large testing and deployment surface. LearnForge • Engineering Handbook • 2026 88 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

Coupling Problem Typical Consequence Shared mutable state Unexpected side effects Shared database assumptions Hard-to-coordinate changes Large interfaces Unnecessary implementation changes Direct infrastructure dependencies Difficult testing Circular dependencies Hard-to-understand architecture

13. Circular Dependencies Circular dependencies occur when modules depend on one another in a cycle. They can make initialization, testing, deployment, and reasoning about ownership significantly harder.

```
class OrderService {
  constructor( private readonly paymentService: PaymentService ) {}
}
class PaymentService {
  constructor( private readonly orderService: OrderService ) {}
}

```

A better design often introduces a higher-level workflow or event boundary so that neither module must directly own the other.

14.

Refactoring Toward Better Boundaries

1. Identify modules that change together frequently.
2. Identify modules that should change independently.
3. Move strongly related behavior closer together.
4. Remove dependencies that cross unrelated responsibilities.
5. Replace direct infrastructure dependencies with stable contracts where justified.

LearnForge • Engineering Handbook • 2026 89 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

6. Break cycles before introducing more abstractions.
7. Verify behavior with automated tests.

15. Practical Refactoring Example

```
class OrderManager {
  createOrder(order: Order) {
    // database logic
    // payment logic
    // email logic
    // invoice generation
    // analytics logic
  }
}
```

This design has low cohesion because many responsibilities are concentrated in one component.

```
class OrderService {
  constructor(
    private readonly orders: OrderRepository,
    private readonly payments: PaymentGateway,
    private readonly notifications: NotificationService
  ) {}
  async createOrder(input: CreateOrderInput) {
    const order = await this.orders.create(input);
    await this.payments.charge(
      order.total,
      order.paymentReference
    );
    await this.notifications.sendConfirmation(order);
    return order;
  }
}
```

The second design gives the workflow explicit dependencies and allows each infrastructure concern to evolve independently.

LearnForge • Engineering Handbook • 2026 90 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

16. DRY Does Not Mean Centralize Everything Over-centralization can create a single module that becomes a dependency for large parts of the application. Such a module may technically remove duplication while increasing coupling.
- Common Mistake Creating a universal utility module that contains unrelated business rules, formatters, validators, database helpers, and API functions. This often becomes a hidden dependency hub.
17. KISS Does Not Mean Ignoring Architecture Simple code does not mean putting the entire application into one file or removing every abstraction. A sufficiently large system needs boundaries. The KISS principle asks whether those boundaries are justified and understandable. A modular architecture can therefore be simple even when the system itself is large.
18. YAGNI and Future-Proofing There is an important distinction between future-proofing and speculative implementation.

Future-Proofing Speculative Implementation

Clear interface at a real boundary

Framework for hypothetical features

Configuration instead of hard-coded deployment values

Dozens of unused configuration options

Modular structure

Unused plugin architecture

Automated tests

Unused feature implementations

19. A Unified Design Model Figure 12.3 — DRY, KISS, and YAGNI help guide implementation choices while cohesion and coupling describe the resulting structural quality.

LearnForge • Engineering Handbook • 2026 91 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

20. Practical Decision Framework Question Preferred Direction Is the same business knowledge duplicated? Consider DRY refactoring Is the solution harder than the requirement? Apply KISS Is functionality unsupported by a real requirement? Apply YAGNI Are unrelated responsibilities mixed together? Increase cohesion Do changes propagate unnecessarily? Reduce coupling
21. Practical Checklist • Is important business knowledge represented in

one authoritative place? • Have accidental duplication and conceptual duplication been distinguished? • Is the implementation as simple as the requirements allow? • Are speculative features being avoided? • Are related responsibilities grouped together? • Are unrelated responsibilities separated? • Are dependencies explicit and understandable? • Are circular dependencies avoided? • Can modules evolve independently where appropriate? • Have abstractions been justified by real value?

LearnForge • Engineering Handbook • 2026 92 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

22. Review Questions

1. What does DRY mean beyond avoiding repeated lines of code?
2. When can removing duplication increase coupling?
3. What is accidental complexity?
4. How does KISS differ from simply writing less code?
5. What problem does YAGNI address?
6. What is high cohesion?
7. Why is zero coupling not a realistic goal?
8. What problems can circular dependencies create?
9. How can excessive abstraction violate KISS?
10. How should an engineer decide whether to extract shared logic?

23. Key Takeaways

- DRY is about avoiding duplicated knowledge, not eliminating every similar line of code.
- KISS encourages solutions whose complexity is justified by real requirements.
- YAGNI protects teams from implementing uncertain future features prematurely.
- High cohesion keeps related responsibilities together.
- Controlled coupling prevents unnecessary change propagation.
- Good design balances reuse with independence.
- Abstractions should solve real problems rather than satisfy principles mechanically.
- Simple, focused modules are easier to test, understand, and evolve.

24. Architect's Final Checklist

- Does every shared abstraction represent shared knowledge?
- Could this abstraction create more coupling than it removes?
- Is every major feature supported by a real requirement?
- Is accidental complexity being actively reduced?

LearnForge • Engineering Handbook • 2026 93 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

Are module responsibilities coherent?

- Are dependencies crossing boundaries for a clear reason?
- Can a new engineer understand the structure without excessive explanation?
- Does the design remain simple without sacrificing necessary architecture?

LearnForge • Engineering Handbook • 2026 94 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

Chapter 13 API Architecture

A well-designed API is a stable contract between software components. Good API architecture makes systems easier to integrate, evolve, secure, test, observe, and operate without forcing clients to understand internal implementation details.

What You Will Learn

- How to design APIs around business resources and capabilities.
- How HTTP methods and status codes communicate API behavior.
- How to design consistent request, response, and error contracts.
- How versioning protects clients while APIs evolve.
- How pagination, filtering, sorting, and rate limiting improve API usability.
- How authentication, authorization, validation, and idempotency fit into API architecture.
- How observability and documentation make APIs production-ready.

1. What Is an API?

An Application Programming Interface (API) is a defined contract through which one software component communicates with another. The contract describes what can be

requested, what data must be supplied, what response will be returned, and how errors are represented. An API should hide unnecessary implementation details. A client should not need to know how a service stores its data internally or which framework implements the server. Architect's Rule Design the API around the consumer's required business capabilities, not around the internal structure of your database or classes.

2. API as a Contract

The most important property of a production API is predictability. Clients should be able to rely on stable names, data types, semantics, error formats, and documented behavior.

LearnForge • Engineering Handbook • 2026 95 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

Contract Element Example

Why It Matters	Endpoint	GET /courses/123
Defines how a resource is addressed	Request	JSON body or query parameters
Defines required input	Response	JSON representation
Defines returned data	Status code	200, 404, 422
Communicates operation result	Error contract	Structured error object
Allows predictable failure handling		

3. Resource-Oriented API Design

A common approach for HTTP APIs is to model important business entities as resources. URLs identify resources while HTTP methods communicate the intended operation.

Method Example

Typical Meaning	GET /courses/123	POST /courses	PUT /courses/123	PATCH /courses/123	DELETE /courses/123
Read a resource					
Create a resource or initiate an operation					
Replace a resource representation					
Partially modify a resource					
Remove or deactivate a resource					

Figure 13.1 — A well-designed API separates the external contract from internal business and data implementation.

4. Naming Endpoints

Endpoint naming should be consistent and understandable. Resource names normally use nouns rather than verbs because the HTTP method already communicates the action.

LearnForge • Engineering Handbook • 2026 96 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

Avoid

POST /getCourses	GET /courses	POST /createCourse	POST /courses	POST /deleteCourse/123	DELETE /courses/123	POST /updateCourse/123	PATCH /courses/123

Common Mistake

Mixing multiple naming styles in the same API. Inconsistent endpoint conventions increase cognitive load and make client integrations harder to maintain.

5. HTTP Status Codes

Status codes are part of the API contract. They allow clients to distinguish successful operations from different classes of failure without parsing human-readable messages.

LearnForge • Engineering Handbook • 2026 97 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

Status Meaning

Typical Example	200 OK	201 Created	204 No Content	400 Bad Request	401 Unauthorized	403 Forbidden	404 Not Found	409 Conflict	422 Unprocessable Content	429 Too Many Requests	500 Internal Server Error	503 Service Unavailable
Successful read or update												
Created New resource												
Successful deletion with no response body												
Malformed request												
Authentication required or invalid												
Authenticated but not permitted												
Resource does not exist												
State conflict or duplicate operation												
Content Validation failure												
Too Many Requests												
Rate limit exceeded												
Unexpected server failure												
Temporary service or dependency failure												

6. Request Validation

Validation should happen at the API boundary before invalid data reaches business logic. This protects internal components and provides clients with clear

feedback. LearnForge • Engineering Handbook • 2026 98 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE interface CreateCourseRequest { title: string; description: string; instructorId: string; } function validateCourseRequest(input: CreateCourseRequest) { if (!input.title.trim()) { throw new Error("title is required"); } if (!input.instructorId.trim()) { throw new Error("instructorId is required"); } if (input.title.length > 200) { throw new Error("title is too long"); } }

Production applications should normally use a dedicated validation library or framework mechanism rather than relying only on manually written checks. The important architectural principle is that untrusted external input must be validated before entering trusted application logic.

7. Consistent Error Responses

An API should use a predictable error structure. Clients should not need to parse arbitrary text messages to determine what went wrong. { "error": { "code": "COURSE_NOT_FOUND", "message": "The requested course does not exist.", "requestId": "req_8f31a2" } }

The internal error details shown to developers may be much more detailed than the information returned to clients. Sensitive stack traces, database details, credentials, and infrastructure information should never be exposed through public API responses.

LearnForge • Engineering Handbook • 2026 99 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

Production Tip

Give every request a correlation or request identifier. When a client reports a failure, the identifier should allow engineers to locate the corresponding server-side logs and traces.

8. API Evolution

APIs rarely remain unchanged. New fields, new behaviors, new validation rules, and new business capabilities are introduced over time. A good API strategy allows evolution without unnecessarily breaking existing consumers. Backward compatibility should be treated as an architectural concern rather than something discovered only after a client breaks.

Architect's Rule

Prefer additive, backward-compatible changes whenever possible. Removing or changing the meaning of an existing field can be more dangerous than adding a new capability.

9. API Versioning

Versioning provides a mechanism for managing incompatible changes. Common strategies include URL versioning, header-based versioning, and media-type versioning.

Strategy Example Consideration URL /api/v1/courses Simple and highly visible Header API-Version: 2 Keeps URLs stable Media Type Accept: application/vnd.api.v2+json Explicit representation versioning

10. API Design Principle

- APIs are contracts, not merely URLs.
- Consistency is more valuable than cleverness.
- Validate external input at the boundary.
- Use status codes and structured errors consistently.

LearnForge • Engineering Handbook • 2026 100 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

Design for evolution instead of assuming the first version will never change.

11. API Versioning

APIs evolve as business requirements change. A well-designed API must allow the server to improve without unexpectedly breaking existing consumers. Versioning provides a controlled way to introduce incompatible changes.

Strategy Example Advantage Trade-off Multiple URLs must be URL Versioning /api/v1/users Simple and visible maintained Header Accept: application/ Less obvious to Keeps URL stable Versioning

vnd.api.v2+json developers Query Can become /users?version=2 Easy to implement Versioning inconsistent Explicit Media-Type application/ representation More complex tooling Versioning vnd.company.resource+json control Architect's Rule Do not create a new API version merely because an implementation changed. Version when the public contract changes in a way that existing consumers cannot safely tolerate.

12. Backward Compatibility

Backward compatibility means existing clients continue to work after the server evolves. Additive changes are usually safer than destructive changes. For example, adding an optional response field is generally safer than renaming or removing an existing field.

LearnForge • Engineering Handbook • 2026 101 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

Change Compatibility

Change	Compatibility
Add optional response field	Usually safe
Add optional request field	Usually safe
Remove response field	Potentially breaking
Rename a field	Breaking
Change field type	Breaking
Make an optional field required	Breaking

13. Authentication and Authorization

Authentication answers the question "Who are you?" Authorization answers "What are you allowed to do?" These concerns should be explicitly separated in API design.

```
type UserContext = {
  userId: string;
  roles: string[];
};
function canUpdateCourse(user: UserContext): boolean {
  return user.roles.includes("course_admin");
}
```

Authentication mechanisms may include sessions, OAuth 2.0, OpenID Connect, or signed tokens depending on the system. Authorization should be enforced at the server boundary rather than relying on client-side controls. Common Mistake Hiding an administrative button in the frontend and assuming the operation is protected. A malicious client can call the API directly. Authorization must be validated by the server.

LearnForge • Engineering Handbook • 2026 102 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

14. Input Validation

An API should validate untrusted input before it reaches business logic or persistence layers. Validation should check required fields, types, ranges, formats, and business constraints where appropriate.

```
function validateCreateUser(input: unknown) {
  if (!input || typeof input !== "object") {
    throw new Error("Invalid request body");
  }
  const body = input as Record;
  if (typeof body.email !== "string") {
    throw new Error("Email is required");
  }
  if (typeof body.name !== "string" || body.name.length > 100) {
    throw new Error("Invalid name");
  }
}
```

Validation should produce predictable errors. Clients should be able to understand what was rejected without exposing internal implementation details.

15. Error Handling

Consistent error responses make APIs easier to integrate and operate. An error should communicate the category of failure while avoiding sensitive internal information.

LearnForge • Engineering Handbook • 2026 103 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

Status Meaning Typical Use

Status	Meaning	Typical Use
400	Bad Request	Malformed or invalid input
401	Unauthorized	Missing or invalid authentication
403	Forbidden	Authenticated but not permitted
404	Not Found	Requested resource does not exist
409	Conflict	State conflict or duplicate operation
429	Too Many Requests	Rate limit exceeded
500	Internal Server Error	Unexpected server failure

```
{ "error": { "code": "COURSE_NOT_FOUND", "message": "The requested course does not
```

exist", "requestId": "req_7f31c2a9" } } 16. Pagination, Filtering and Sorting Collection endpoints should avoid returning unlimited amounts of data. Pagination protects server resources and gives clients predictable response sizes. Figure 13.2 — Pagination prevents large collections from consuming unbounded API and database resources.

17. Rate Limiting Rate limiting controls how much traffic a client can generate within a defined period. It protects APIs from accidental overload, abusive traffic, and poorly behaved clients. LearnForge • Engineering Handbook • 2026 104

LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE Strategy Concept Fixed Window Count requests within fixed time intervals Sliding Window Measure requests across a moving time range Token Bucket Allow bursts while controlling average rate Leaky Bucket Process requests at a controlled output rate Production Tip Rate limits should be designed around business importance. Login, payment, search, and public endpoints may require different limits and protection strategies.

18. API Idempotency Some API operations may be repeated because clients retry after network timeouts. For operations such as payments, order creation, or provisioning, the API should provide an idempotency mechanism. POST /api/v1/payments Idempotency-Key: 7f31c2a9-82a4-4a12-bb31-91f0a8c2e101 Content-Type: application/json { "amount": 5000, "currency": "USD" } The server stores the result associated with the idempotency key for an appropriate retention period. A repeated request using the same key can return the original result rather than creating another transaction.

19. API Documentation An API contract is only useful when consumers can understand how to use it. Documentation should describe endpoints, authentication, request schemas, responses, errors, limits, examples, and compatibility expectations. LearnForge • Engineering Handbook • 2026 105

LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE OpenAPI is commonly used to describe HTTP APIs in a machine-readable format. The specification can support documentation generation, validation, client generation, and contract testing. openapi: 3.0.3 info: title: Course API version: 1.0.0 paths: /courses/{id}: get: summary: Get a course parameters: - name: id in: path required: true schema: type: string responses: "200": description: Course returned "404": description: Course not found

20. API Observability A production API should expose enough telemetry to answer operational questions. At minimum, teams should monitor request volume, latency, error rates, saturation, and important business outcomes. Signal Example Why It Matters Latency p50, p95, p99 Shows user-facing performance Errors 4xx and 5xx rates Shows failures and client misuse Traffic Requests per second Shows demand Saturation CPU, memory, connections Shows resource pressure Business metrics Successful payments Connects technical health to outcomes

LearnForge • Engineering Handbook • 2026 106

LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE 21. API Security Checklist • Authenticate requests where required. • Authorize every sensitive operation on the server. • Validate and constrain untrusted input. • Use HTTPS for network communication. • Avoid exposing secrets or internal stack traces. • Apply appropriate rate limits. • Protect sensitive endpoints from

abuse. • Log security-relevant events without logging secrets. 22. API Design Decision Framework Question Recommended Direction Is the resource clearly defined? Use a stable resource-oriented contract Does the operation change state? Define idempotency and retry behavior Can the collection become large? Add pagination and filtering Will clients have different release cycles? Design for backward compatibility Is the endpoint public or high traffic? Add rate limiting and abuse protection Is the operation security-sensitive? Require explicit authorization

23. Key Takeaways • APIs are contracts between independently evolving components. LearnForge • Engineering Handbook • 2026 107 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE • Resource boundaries should reflect meaningful business concepts. • Backward compatibility is a core API design concern. • Authentication and authorization are separate responsibilities. • Input validation must happen at trust boundaries. • Errors should be consistent, useful, and safe. • Pagination and rate limiting protect system resources. • Idempotency makes retries safer for state-changing operations. • Documentation and observability are part of production API design. 24. Quick Review 1. What makes an API a contract? 2. When should an API be versioned? 3. Why are additive changes usually safer than destructive changes? 4. What is the difference between authentication and authorization? 5. Why should validation occur at the API boundary? 6. Why are pagination and rate limiting important? 7. How does idempotency protect state-changing operations? 8. What telemetry should a production API expose? 25. Architect's Checklist • Are API resources and responsibilities clearly defined? • Is the public contract documented? • Are breaking and non-breaking changes understood? • Is backward compatibility tested? • Are authentication and authorization enforced server-side? • Are inputs validated at trust boundaries? • LearnForge • Engineering Handbook • 2026 108 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE Are collection endpoints paginated where necessary? • Are rate limits appropriate for traffic patterns? • Are retry and idempotency semantics documented? • Can engineers monitor latency, errors, traffic, and saturation? 26. Final Architecture Principle Architect's Rule A good API is not merely an endpoint that works. It is a stable, explicit, secure, observable contract that remains understandable as clients, services, teams, and business requirements evolve. LearnForge • Engineering Handbook • 2026 109 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

Chapter 14 Database Architecture & Data Modeling Database architecture is not simply about choosing a database engine. It is about designing data ownership, consistency, access patterns, transactions, performance, resilience, and evolution so that the data layer remains reliable as the system grows. What You Will Learn • How to choose between relational and non-relational databases. • How data ownership and access patterns influence schema design. • How normalization, denormalization, and indexing affect performance. • How transactions and isolation protect data correctness. • How replication, sharding, caching, and connection pooling improve scalability. • How migrations and operational practices keep databases maintainable.

1. Why Database Architecture Matters The database is often one of the most important stateful components in a production system. Application code can frequently be replaced or scaled horizontally, but business data must remain correct, durable, and accessible. A poor database design can therefore become a long-term constraint on the entire architecture. Database decisions should begin with business requirements and access patterns rather than popularity. The correct question is not simply "Which database is best?" but "Which data model and operational behavior best fit this system?" Architect's Rule Choose a database according to data characteristics, consistency requirements, query patterns, scale, and operational capability—not because a particular database is fashionable.

LearnForge • Engineering Handbook • 2026 110 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

2. Relational vs Non-Relational Databases Relational databases organize information into tables with defined relationships. They are particularly strong when the system requires structured data, joins, constraints, and transactional consistency. Non-relational databases include document, key-value, wide-column, and graph models. They can be useful when the application's data access patterns or scale characteristics do not fit naturally into a relational model.

Characteristic	Relational	Non-Relational
Data structure	Tables and relationships	Documents, keys, graphs, or columns
Schema	Usually explicit	Often more flexible
Joins	Strong native support	Often limited or avoided
Transactions	Strong transactional model	Varies by product
Best fit	Structured business data	Specific high-scale or flexible access patterns

3. Data Ownership In a well-designed architecture, ownership of data should be explicit. A component or service should have clear responsibility for creating, changing, validating, and protecting the data within its boundary.

Domain	Owner	Example Data
Identity	Identity service	Users, roles, credentials
Catalog	Catalog service	Products, categories, prices
Orders	Order service	Orders, line items, status
Payments	Payment service	Transactions, payment state

LearnForge • Engineering Handbook • 2026 111 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

Clear ownership reduces accidental coupling. Other components should normally interact with owned data through a stable contract instead of directly modifying another component's tables.

4. Schema Design A schema defines the structure and constraints of stored data. Good schema design makes invalid states difficult to create and common queries efficient.

```
CREATE TABLE courses (
  id UUID PRIMARY KEY,
  title VARCHAR(200) NOT NULL,
  description TEXT,
  status VARCHAR(30) NOT NULL,
  created_at TIMESTAMP NOT NULL,
  updated_at TIMESTAMP NOT NULL
);
```

The schema should communicate important business rules. Required fields, unique constraints, foreign keys, and appropriate data types move correctness closer to the data boundary.

5. Primary Keys Every important entity should have a stable identity. Primary keys provide that identity and allow other records to reference the entity.

Key Type	Strength	Trade-off
Integer sequence	Compact and efficient	Can expose ordering information
UUID	Globally unique	Larger storage and index footprint
Natural key	Meaningful to business	May change over

time Common Mistake Using a business attribute as the permanent identity of an entity without considering whether that attribute can change. Email addresses, usernames, and product codes may have business meaning but are not always stable identifiers. LearnForge • Engineering Handbook • 2026 112 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

6. Normalization

Normalization reduces unnecessary duplication by separating data into logical structures and representing relationships explicitly. For example, storing customer information repeatedly inside every order can create update anomalies. A normalized design can keep customer data in one table and reference it from orders.

Problem Effect Normalization Benefit Duplicate data Higher storage and inconsistency risk Centralized representation Update anomaly One fact must be changed in many places Single source of truth Insert anomaly Cannot store a fact independently Independent entities Delete anomaly Deleting one record removes unrelated information Separated responsibilities

7. Denormalization

Denormalization intentionally duplicates or precomputes information to make important read operations faster or simpler. It should be driven by measured access patterns rather than speculation. Architect's Rule Normalize for correctness first. Denormalize when a demonstrated performance or operational requirement justifies the additional consistency burden.

8. Indexing

An index provides a data structure that helps the database locate matching rows without scanning the entire table. `CREATE INDEX idx_courses_status_created ON courses (status, created_at DESC);` LearnForge • Engineering Handbook • 2026 113 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

Indexes can dramatically improve reads, but they are not free. Every index requires storage and can increase the cost of inserts, updates, and deletes.

Index Useful For Cost Single-column Simple filters Additional storage and write work Composite Multi-column queries More complex maintenance Unique Enforcing uniqueness Write validation overhead Partial Frequently queried subset Database-specific behavior

9. Query Performance

Database performance should be investigated using real query behavior. Developers should inspect query plans, identify expensive scans, measure latency, and understand how indexes are being used. `EXPLAIN ANALYZE SELECT id, title FROM courses WHERE status = 'published' ORDER BY created_at DESC LIMIT 20;` A query that appears simple at the application level can still become expensive when executed against millions of rows. Performance testing should therefore use realistic data volumes.

10. Transactions

A transaction groups related database operations into a controlled unit. The classic ACID properties describe important transactional guarantees: atomicity, consistency, isolation, and durability. LearnForge • Engineering Handbook • 2026 114 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

Property Meaning Atomicity All required operations succeed or the transaction is rolled back Consistency Rules and constraints remain satisfied Isolation Concurrent transactions do not produce unacceptable interference Durability Committed data survives expected failures

11. Transaction Example

await db.transaction(async (tx) => { const order = await tx.orders.create({ userId, status:

"pending" }); await tx.orderItems.createMany({ orderId: order.id, items }); await tx.inventory.reserve({ items }); }); The exact transaction API varies by database library, but the architectural principle remains the same: operations that must preserve one business invariant should share an appropriate transactional boundary.

12. Isolation Levels Isolation controls how concurrent transactions can observe each other's changes. Stronger isolation can improve correctness but may increase contention and reduce concurrency. LearnForge • Engineering Handbook • 2026 115

LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE Level General Idea Read Uncommitted Allows the weakest visibility guarantees Read Committed Reads committed data Repeatable Read Provides stronger repeatability within a transaction Serializable Provides the strongest ordering guarantees Production Tip Do not select the strongest isolation level automatically. Choose the weakest level that safely preserves the business invariants required by the operation, then verify the behavior with concurrency tests.

13. Concurrency Control Concurrent requests can attempt to modify the same business state at the same time. Without appropriate controls, lost updates and inconsistent state can occur. Optimistic concurrency uses a version or timestamp to detect whether another process changed a record before an update was applied. UPDATE courses SET title = 'Advanced Architecture', version = version + 1 WHERE id = :id AND version = :expectedVersion; If zero rows are updated, the application can detect that the record changed and require the caller to reload or resolve the conflict.

14. Replication Replication maintains copies of data on multiple database instances. Replication can improve availability, read scalability, and disaster recovery capabilities. LearnForge • Engineering Handbook • 2026 116

LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE Figure 14.1 — Replication can distribute read traffic and provide additional availability and recovery options. A critical consequence is that replicas may lag behind the primary. Applications must understand whether a workflow requires the latest data or can tolerate eventual consistency.

15. Read Replicas and Consistency Read replicas are useful for workloads dominated by queries. However, a request that writes data and immediately reads from a lagging replica may observe stale information. Requirement Possible Strategy Immediately read own write Read from primary or use session consistency Analytics can be stale Read from replicas Critical financial state Prefer stronger consistency High-volume dashboards Use replicas or derived data

16. Sharding Sharding partitions data across multiple database nodes. It can allow a system to scale beyond the capacity of a single database instance, but it also introduces significant complexity. Figure 14.2 — Sharding distributes data across partitions but introduces routing, rebalancing, and cross-shard query complexity.

17. Choosing a Shard Key A shard key determines how records are distributed. A poor key can create hot partitions where one node receives disproportionately high traffic. LearnForge • Engineering Handbook • 2026 117

LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE Shard-Key Property Why It Matters High cardinality Provides many possible partitions Even distribution Prevents hot shards

Common query alignment Reduces cross-shard operations Stable value Avoids expensive record movement

18. Caching

Caching stores frequently accessed data closer to the application so that repeated reads do not always reach the primary database.

```

async function getCourse(id: string) {
  const cached = await cache.get(`course:${id}`);
  if (cached) {
    return JSON.parse(cached);
  }
  const course = await db.courses.findById(id);
  await cache.set(`course:${id}`, JSON.stringify(course), { ttl: 60 });
  return course;
}

```

Caching introduces a new problem: stale data. The architecture must define when cached values expire, when they are invalidated, and whether stale responses are acceptable.

LearnForge • Engineering Handbook • 2026 118

LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

19. Cache Strategies

Strategy	Description	Common Use	Application
Cache-aside	General read-heavy workloads	Explicitly reads and writes the cache	
Read-through	Cache loads missing values automatically	Frequently accessed data	
Write-Strong	Cache freshness	Writes update cache and persistent store	through requirements
Write-behind	Cache writes are persisted asynchronously	Specific high-throughput workloads	

20. Connection Pooling

Opening a new database connection for every request can be expensive and can exhaust database resources. Connection pools maintain a controlled number of reusable connections.

```

const pool = createPool({
  max: 20,
  min: 5,
  idleTimeoutMillis: 30000
});
const result = await pool.query(
  "SELECT id, title FROM courses WHERE status = $1",
  ["published"]
);

```

Pool sizing should consider application concurrency, database capacity, query duration, and the number of application instances. More connections do not automatically mean better performance.

21. Database Migrations

Database schemas evolve as software evolves. Migrations provide a controlled history of schema changes and allow environments to move from one known schema state to another.

LearnForge • Engineering Handbook • 2026 119

LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

```

ALTER TABLE courses ADD COLUMN published_at TIMESTAMP NULL;
CREATE INDEX idx_courses_published_at ON courses (published_at);

```

Production migrations should be designed carefully. Large table rewrites, long locks, and incompatible schema changes can cause significant downtime.

Production Tip Prefer expand-and-contract migrations for risky changes: introduce the new schema, deploy compatible application code, migrate or backfill data, and only then remove the old structure.

22. Soft Deletes

A soft delete marks a record as deleted instead of physically removing it. This can support recovery, auditing, and historical reporting.

```

ALTER TABLE users ADD COLUMN deleted_at TIMESTAMP NULL;
SELECT * FROM users WHERE deleted_at IS NULL;

```

Soft deletes are not automatically appropriate. They can complicate unique constraints, queries, storage, and data retention. The system should define whether deleted data must eventually be permanently removed.

23. Audit Trails

Important business systems often need to answer who changed what and when. An audit trail records significant state transitions or administrative actions.

LearnForge • Engineering Handbook • 2026 120

LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

Field Purpose

actor_id Who performed the action action What happened entity_id Which object changed timestamp When it occurred metadata Additional context

24. Database Failure Handling A production application must assume that the database can become slow, unavailable, overloaded, or temporarily unreachable. Figure 14.3 — Database failure handling should use bounded timeouts, controlled retries, and explicit degraded behavior.

25. Database Architecture Checklist

- Is the database model appropriate for the data and access patterns?
- Is data ownership clearly defined?
- Are primary keys stable and appropriate?
- Are important business constraints enforced?
- Are indexes based on real query patterns?
- Are transactions used around critical invariants?
- Are concurrency conflicts handled explicitly?
- Is replication consistency understood?
- Are caches allowed to become stale where appropriate?

LearnForge • Engineering Handbook • 2026 121 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

- Are migrations tested before production deployment?
- Are database failures and overload scenarios tested?

26. Key Takeaways

- Database architecture should begin with business data and access patterns.
- Clear data ownership reduces coupling and accidental corruption.
- Normalization improves correctness; denormalization should have a measured reason.
- Indexes improve reads but increase storage and write costs.
- Transactions protect business invariants within appropriate boundaries.
- Replication and sharding improve scale but introduce consistency and operational complexity.
- Caching improves performance but creates freshness and invalidation problems.
- Connection pools protect database resources under application concurrency.
- Migrations are part of application architecture, not merely deployment scripts.
- Database failure must be treated as a normal production scenario.

27. Quick Review

1. What factors should influence database selection?
2. Why is clear data ownership important?
3. What problem does normalization solve?
4. When is denormalization justified?
5. Why can too many indexes hurt performance?
6. What do the ACID properties represent?
7. Why are isolation levels important?
8. What problem does replication solve, and what new issue can it introduce?
9. Why is choosing a shard key difficult?
10. What consistency problems can caching create?
11. Why should production migrations be designed carefully?

LearnForge • Engineering Handbook • 2026 122 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

28. Architect's Checklist

- Are database requirements documented before selecting technology?
- Are schema constraints aligned with business invariants?
- Have high-value queries been identified and measured?
- Are transaction boundaries explicit?
- Are concurrency scenarios tested?
- Is replica lag monitored where replicas are used?
- Is sharding avoided until single-node or simpler scaling options are insufficient?
- Are cache invalidation and TTL policies documented?
- Is connection-pool capacity appropriate for the database?
- Are rollback and recovery strategies defined for schema changes?
- Are backup and restore procedures regularly tested?

29. Final Architecture Principle Architect's Rule A database is not merely a storage component. It is a correctness boundary, a performance boundary, and an operational

dependency. Good architecture makes its guarantees, limitations, ownership, and failure behavior explicit. LearnForge • Engineering Handbook • 2026 123

LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

Chapter 15 Database Architecture

Database architecture is not simply about choosing SQL or NoSQL. It is about designing data ownership, consistency, transactions, indexing, scalability, availability, and recovery around the actual requirements of the business. What You Will Learn • How to choose between relational and non-relational databases. • Why data modeling is an architectural decision. • How indexes improve reads while creating write and storage costs. • How transactions and isolation protect business correctness. • How replication, partitioning, caching, and read replicas scale systems. • How database ownership and migrations reduce architectural risk.

1. Why Database Architecture Matters

The database is often the most durable component of an application. Application code can be replaced or redeployed quickly, but business data may need to survive for years. A poor database design can therefore become a long-term architectural constraint. Database architecture determines how information is represented, protected, queried, replicated, migrated, backed up, and recovered. These decisions affect performance and correctness as well as the ability of teams to evolve the system.

Architect's Rule

Choose database technology according to workload, consistency requirements, access patterns, and operational capability—not because a technology is currently fashionable.

2. Relational Databases

Relational databases organize information into tables connected through well-defined relationships. SQL provides expressive querying, transactions, constraints, and mature tooling.

LearnForge • Engineering Handbook • 2026 124

LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

Strength Why It Matters

Transactions

Multiple related changes can succeed or fail together. Constraints The database can enforce important integrity rules. SQL Complex relationships and analytical queries are well supported. Mature tooling Monitoring, backups, migrations, and administration are widely supported.

3. NoSQL Databases

NoSQL is a broad category rather than one specific database model. Document, key-value, wide-column, and graph databases optimize for different access patterns and workloads.

Model	Typical Use	Important Consideration
Document	Flexible application records	Data duplication may be intentional.
Key-value	Fast lookup by key	Access patterns should be predictable.
Wide-column	Large-scale distributed workloads	Data modeling is often query-driven.
Graph	Highly connected relationships	Traversal patterns drive the design.

4. Choosing SQL or NoSQL

Requirement	Often Favors
Strong relational integrity	Relational database
Complex joins	Relational database
Flexible document structure	Document database
Extremely high key-based throughput	Key-value systems
Highly connected graph traversal	Graph database

LearnForge • Engineering Handbook • 2026 125

LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

The important question is not which category is universally better. The important question is which data model naturally represents the application's business rules and dominant access patterns.

5.

Data Modeling Data modeling describes the entities, attributes, relationships, constraints, and lifecycle of business information. Good modeling starts from business concepts and expected queries rather than from arbitrary tables. Figure 15.1 — A simple relational model expresses ownership and relationships between business entities.

6. Normalization Normalization reduces unnecessary duplication by separating data into related structures. It can improve consistency because one fact has a controlled location. However, normalization is not an absolute rule. Read-heavy workloads may benefit from carefully controlled denormalization when the performance gain justifies the additional complexity. Common Mistake Denormalizing prematurely. First understand the workload and measure the actual bottleneck. Duplication creates synchronization responsibilities.

7. Transactions A transaction groups related database operations into a logical unit. The classic ACID properties describe important transactional guarantees: atomicity, consistency, isolation, and durability. LearnForge • Engineering Handbook • 2026 126 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE Property Meaning Atomicity All required changes succeed together or are rolled back. Consistency Transactions preserve defined integrity rules. Isolation Concurrent operations are controlled according to the isolation model. Durability Committed data survives appropriate failures.

8. Transaction Example

```
async function createOrder(db: Database, order: Order) {
  return db.transaction(async tx => {
    await tx.insert("orders", order);
    await tx.insert("order_items", {
      order_id: order.id,
      product_id: order.productId,
      quantity: order.quantity
    });
    return order;
  });
}
```

The transaction boundary should match the business operation being protected. It should not be made unnecessarily large because long transactions can hold locks and consume database resources.

9. Isolation Levels Concurrent transactions can observe or interfere with each other in different ways. Database systems therefore provide isolation levels that balance correctness, concurrency, and performance. LearnForge • Engineering Handbook • 2026 127 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE Isolation Level General Idea

Isolation Level	General Idea
Read Uncommitted	Allows the weakest visibility guarantees.
Read Committed	Prevents reading data that has not been committed.
Repeatable Read	Provides stronger repeatability for reads within a transaction.
Serializable	Provides the strongest isolation at higher concurrency cost. The exact behavior varies between database engines. Architects should consult the documentation of the selected database rather than assuming that names alone imply identical behavior.

10. Indexes Indexes allow databases to locate rows more efficiently for supported query patterns. They can dramatically improve read performance but consume storage and add work to inserts, updates, and deletes.

```
CREATE INDEX idx_orders_customer_created ON orders(customer_id, created_at);
```

A composite index should reflect actual query patterns. The order of indexed columns matters because the database optimizer may only be able to efficiently use certain prefixes of the index.

11. Query Performance Performance tuning should begin with evidence. Slow-query logs, execution plans, latency metrics, and production workload measurements

are more useful than guessing. LearnForge • Engineering Handbook • 2026 128

LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE Signal Possible Cause Investigation High query latency Missing or ineffective index Inspect execution plan High CPU Expensive queries Profile query workload Lock contention Long or conflicting transactions Inspect locks and transaction duration Connection exhaustion Too many concurrent clients Review pooling and concurrency

12. **Connection Pooling** Opening a new database connection for every request can create unnecessary overhead. Connection pools maintain a controlled number of reusable connections. `const pool = new Pool({ max: 20, idleTimeoutMillis: 30_000, connectionTimeoutMillis: 2_000 });` Pool size should be based on database capacity, workload, query duration, application concurrency, and the number of application instances. More connections do not automatically produce more throughput.

13. **Read Replicas** Read replicas allow some read workloads to be served from copies of the primary database. This can reduce pressure on the primary and improve read scalability. Figure 15.2 — Replication can distribute read workloads while preserving a designated write authority.

14. **Replication Lag** Read replicas may not contain the newest committed data immediately. This creates replication lag and therefore possible stale reads. LearnForge • Engineering Handbook • 2026 129

LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE Production Tip Do not send every read to replicas automatically. User flows that require read-after-write consistency may need to read from the primary or use another consistency mechanism.

15. **Caching** Caching stores frequently accessed data closer to the application or user. A cache can reduce database load and latency, but introduces another copy of data that must eventually become invalid or expire. `async function getProduct(id: string) { const cached = await cache.get(`product:${id}`); if (cached) { return JSON.parse(cached); } const product = await db.query( "SELECT * FROM products WHERE id = $1", [id] ); await cache.set( `product:${id}`, JSON.stringify(product), { ttl: 300 } ); return product; }` The central cache question is not only how to store data. It is how and when cached data becomes invalid. LearnForge • Engineering Handbook • 2026 130

LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

16. **Cache Invalidation Strategy** Advantage Risk TTL Simple automatic expiration Data may remain stale Explicit invalidation Faster freshness after changes More application complexity Write-through Cache updated with writes Additional write path complexity Cache-aside Simple and widely applicable Requires careful invalidation

17. **Partitioning** Partitioning divides a large dataset into smaller logical or physical partitions. The goal may be improved scalability, maintenance, or query performance. A partition key should distribute workload effectively. A poor key can create hot partitions where a disproportionate amount of traffic reaches one partition. Architect's Rule Choose partition keys using real access patterns. A technically valid key that concentrates traffic can become a scalability bottleneck.

18. **Sharding** Sharding distributes data across independent database nodes. It can provide horizontal capacity beyond a single database instance, but it also makes

operations, queries, transactions, and migrations more complicated. Benefit Trade-off Horizontal capacity More operational complexity Distributed storage Cross-shard queries become harder Potential workload isolation Rebalancing can be difficult

LearnForge • Engineering Handbook • 2026 131 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

19. Database per Service In a microservice architecture, database ownership should normally follow service ownership. This does not necessarily mean every service needs a different database technology. It means that one service should not casually modify another service's internal tables. Common Mistake Sharing database tables across independent services. This creates hidden coupling because schema changes become coordination requirements across teams.

20. Schema Migrations Database schemas evolve continuously. Production migrations should be backward-compatible whenever old and new application versions may coexist. `ALTER TABLE users ADD COLUMN display_name VARCHAR(120); UPDATE users SET display_name = name WHERE display_name IS NULL;` A safer deployment may add the new field first, deploy code that can work with both schemas, backfill data, and only later remove obsolete structures.

21. Expand and Contract Phase Action Expand Add new schema structures without breaking old code. Migrate Move or backfill existing data. Switch Deploy application code using the new structure. Contract Remove obsolete structures after they are no longer used.

LearnForge • Engineering Handbook • 2026 132 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

22. Backups and Recovery A backup strategy is incomplete if restoration has never been tested. Organizations need to understand how much data they can afford to lose and how quickly service must be restored. Concept Question RPO How much recent data loss is acceptable? RTO How quickly must service be restored? Backup retention How long must recoverable copies remain available? Restore testing Can the team actually recover the system?

23. Database Security Database security begins with least privilege. Applications should receive only the permissions they require. Credentials should be managed through appropriate secret-management mechanisms rather than embedded in source code. `const databaseConfig = { host: process.env.DB_HOST, user: process.env.DB_USER, password: process.env.DB_PASSWORD, database: process.env.DB_NAME };` Encryption in transit, encryption at rest, auditing, access controls, credential rotation, and network isolation should be evaluated according to the sensitivity and regulatory requirements of the data.

24. Database Observability • Latency: how long queries and transactions take. • Throughput: how much work the database processes. • Error rate: how frequently operations fail.

LearnForge • Engineering Handbook • 2026 133 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE • Connections: whether connection pools are exhausted. • Locks: whether transactions are blocking one another. • Storage: whether capacity is approaching limits.

25. Practical Database Decision Framework Question Architectural Direction Are strong relationships and transactions Prefer a relational model. central? Are access patterns primarily key-based? Evaluate key-value storage.

Is flexible document structure important? Evaluate document storage. Is read traffic much larger than write traffic? Consider replicas and caching. Is one database reaching capacity? Measure first, then evaluate partitioning or sharding. Invest in automated migrations and compatibility. Are schema changes frequent? practices.

26. Database Architecture Checklist

- Is the data model aligned with business concepts?
- Are integrity constraints enforced appropriately?
- Are transaction boundaries clearly defined?
- Are important queries supported by appropriate indexes?
- Is connection pooling configured according to measured capacity?
- Are replica consistency characteristics understood?
- Is cache invalidation behavior documented?

LearnForge • Engineering Handbook • 2026 134 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

Are partition keys based on actual access patterns?

- Is database ownership clear between services?
- Are migrations backward-compatible where required?
- Are backups regularly tested through restoration?
- Are RPO and RTO requirements defined?
- Is database access protected by least privilege?
- Are database latency, errors, locks, and capacity monitored?

27. Key Takeaways

- Database architecture is a business and systems-design decision, not merely a technology choice.
- Relational databases remain powerful when relationships, integrity, and transactions matter.
- NoSQL databases are useful when their data models match the workload.
- Indexes improve reads but add storage and write costs.
- Replication improves scalability but can introduce stale reads.
- Caching reduces latency and database load but creates invalidation complexity.
- Partitioning and sharding can increase capacity while increasing operational complexity.
- Database ownership should be explicit in service-oriented architectures.
- Backups are valuable only when restoration is reliable and tested.
- Security, observability, migrations, and recovery are part of database architecture.

28. Quick Review

1. When is a relational database usually a strong architectural choice?
2. What trade-offs are introduced by indexes?
3. Why are transaction boundaries important?
4. What problem does connection pooling solve?

LearnForge • Engineering Handbook • 2026 135 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

5. Why can read replicas produce stale data?
6. What makes cache invalidation difficult?
7. When should partitioning or sharding be considered?
8. Why should services avoid directly modifying another service's tables?
9. What are RPO and RTO?
10. Why must database restoration be tested?

29. Architect's Final Checklist

- Have database requirements been derived from actual business workflows?
- Are consistency requirements explicitly documented?
- Are critical queries measured under realistic workload?
- Are indexes reviewed using execution plans?
- Are transactions bounded and observable?
- Are replication and failover behaviors understood?
- Are cache failure and invalidation scenarios designed?
- Are migrations tested before production deployment?
- Are backup restoration procedures automated or regularly rehearsed?
- Can the team explain how the database behaves during partial failure?

LearnForge • Engineering Handbook • 2026 136 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

Chapter 16 Security

Architecture Secure architecture is not a collection of isolated security features. It is a system-wide discipline that reduces attack surface, protects identities and data, limits the impact of compromise, and makes security behavior explicit across every architectural boundary.

What You Will Learn

- How to design security as an architectural concern rather than an afterthought.
- The difference between authentication, authorization, and accountability.
- How sessions, tokens, OAuth, and service identities work together.
- How to protect APIs, secrets, data, and service-to-service communication.
- How threat modeling and defense-in-depth reduce architectural risk.
- How to design security controls that remain practical in production.

1. Security Is an Architectural Concern Security cannot be added successfully at the end of a system's development. Architectural decisions determine where trust exists, how data moves, which components can access sensitive resources, and how much damage can occur after a compromise. A secure architecture therefore considers security at the same level as performance, reliability, maintainability, and scalability. Every boundary between users, applications, services, databases, networks, and external providers should have an explicit security model. Architect's Rule Never define a security boundary implicitly. Every important boundary should clearly define who is trusted, what is allowed, what is protected, and what happens when authorization fails.

2. The Security Objectives A useful starting point is to identify what the architecture is trying to protect. Traditional security design commonly focuses on confidentiality, integrity, and availability.

LearnForge • Engineering Handbook • 2026 137 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

Objective	Meaning	Example
Prevent unauthorized access to	Protect user records from unauthorized	Confidentiality
information	users	
Integrity	Prevent unauthorized modification	Prevent alteration of payment records
Availability	Keep important systems usable	Protect APIs against resource exhaustion
Accountability	Determine who performed an action	Audit administrative changes

3. Authentication vs Authorization Authentication answers the question: "Who are you?" Authorization answers a different question: "What are you allowed to do?" Confusing these responsibilities creates security vulnerabilities. A valid identity does not automatically have permission to access every resource.

Figure 16.1 — Authentication establishes identity; authorization determines what that identity is permitted to access.

4. Least Privilege Least privilege means granting an identity only the permissions required to perform its legitimate responsibilities. A database account used by a reporting service should not automatically have permission to delete production data. Likewise, an API endpoint should not return administrative information merely because the caller is authenticated.

LearnForge • Engineering Handbook • 2026 138 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

Identity	Required Permission	Unnecessary Permission
Read-only analyst	Read reports	Modify production data
Payment service	Create and query payments	Manage user credentials
Notification worker	Read notification jobs	Delete customer accounts
Administrator	Explicit administrative actions	Unrestricted access without auditing

Production Tip Review permissions periodically. Permissions that were appropriate six months ago may no longer be necessary after the system evolves.

5. Password Security

Passwords should never be stored as plaintext. Applications should use a password hashing algorithm designed specifically for password storage, with appropriate work factors and unique salts. The application should also avoid exposing password-related information in logs, error messages, analytics, or monitoring systems.

```
import argon2 from "argon2";
async function hashPassword(password: string) {
  return argon2.hash(password);
}
async function verifyPassword(password: string, storedHash: string) {
  return argon2.verify(storedHash, password);
}
```

Common Mistake Using a fast general-purpose hash such as a basic SHA-256 digest directly for password storage. Password hashing should be intentionally expensive and should use a dedicated password-hashing algorithm.

LearnForge • Engineering Handbook • 2026 139

LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

6. Session-Based Authentication

In session-based authentication, the server maintains authentication state and gives the client a session identifier. The identifier should be protected using secure cookie settings.

Cookie Attribute Purpose

Attribute	Purpose
Secure	Send the cookie only over HTTPS
HttpOnly	Reduce direct JavaScript access to the cookie
SameSite	Reduce cross-site request risks

```
res.cookie("session", sessionId, {
  httpOnly: true,
  secure: true,
  sameSite: "lax",
  maxAge: 1000 * 60 * 60
});
```

7. Token-Based Authentication

Token-based authentication allows a client to present a signed credential with subsequent requests. JSON Web Tokens are one common implementation, although tokens are not automatically more secure than server-side sessions. The architecture should define token lifetime, issuer, audience, signing algorithm, revocation behavior, and the permissions represented by the token.

```
interface AccessTokenClaims {
  sub: string;
  aud: string;
  iss: string;
  exp: number;
  scope: string[];
}
```

Architect's Rule LearnForge • Engineering Handbook • 2026 140

LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

Treat access tokens as credentials. Do not place secrets or unnecessary sensitive information inside them, and keep their lifetime appropriate for the risk of the operation.

8. OAuth and Delegated Authorization

OAuth allows an application to obtain delegated access to resources without requiring the application to receive the user's primary credentials. The important architectural idea is separation of identity, authorization, and resource ownership. Modern deployments should use well-defined flows appropriate to the client type rather than inventing custom authentication protocols.

Component Responsibility

Component	Responsibility
Resource Owner	Controls the protected resource
Client	Requests delegated access
Authorization Server	Issues authorization credentials
Resource Server	Protects the requested resource

9. Role-Based Access Control

Role-Based Access Control, or RBAC, assigns permissions through roles. Instead of granting every user individual permissions, the architecture defines roles such as administrator, instructor, analyst, or student.

```
type Role = | "student" | "instructor" | "admin";
const permissions: Record<Role, string[]> = {
  student: ["course:read"],
  instructor: ["course:read", "course:write"],
  admin: ["course:read", "course:write"],
};
```

"user:manage"] }]; LearnForge • Engineering Handbook • 2026 141 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE 10. Attribute-Based Authorization RBAC can become difficult when authorization depends on resource ownership, organization membership, location, risk level, or other attributes. Attribute-Based Access Control evaluates these properties directly. Attribute Example Rule User identity User may access their own profile Organization Employee may access resources belonging to their organization Resource owner Owner may edit the resource Risk level Sensitive operation requires stronger verification 11. API Security APIs should assume that every request may be malicious or malformed. Authentication, authorization, validation, rate limiting, logging, and safe error handling should be implemented at appropriate boundaries. Control Purpose Authentication Identify the caller Authorization Enforce permissions Validation Reject malformed input Rate limiting Control abusive traffic Audit logging Record important security events Safe errors Avoid leaking internal information LearnForge • Engineering Handbook • 2026 142 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE 12. Input Validation Input validation should happen at trust boundaries. Never assume that data from browsers, mobile applications, external APIs, message queues, or other services is automatically trustworthy. interface CreateUserInput { email: string; displayName: string; } function validateCreateUser(input: unknown): CreateUserInput { if (typeof input !== "object" || input === null || typeof (input as any).email !== "string" || typeof (input as any).displayName !== "string") { throw new Error("Invalid request"); } return input as CreateUserInput; } Validation should also enforce business constraints such as maximum lengths, allowed values, numeric ranges, and required relationships between fields. 13. SQL Injection and Parameterized Queries Applications should never construct SQL queries by directly concatenating untrusted input. Parameterized queries separate SQL structure from values. const result = await db.query("SELECT id, email FROM users WHERE email = \$1", [email]); Common Mistake Assuming that frontend validation is sufficient. Client-side validation improves user experience but cannot replace server-side validation. LearnForge • Engineering Handbook • 2026 143 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE 14. Secrets Management API keys, database passwords, signing keys, and service credentials should not be embedded directly into source code or committed to version control. Production systems should use a dedicated secrets-management mechanism or a secure environment configuration strategy appropriate to the deployment platform. Secret Unsafe Practice Better Practice Database password Hard-coded in repository Managed secret configuration API key Committed to source control Secret store or protected environment Signing key Shared in chat messages Controlled key management 15. Encryption in Transit Sensitive communication should be protected in transit. HTTPS and modern TLS configurations prevent intermediaries from trivially reading or modifying traffic. Encryption in transit should cover external client connections and, where

appropriate, internal service-to-service communication. Figure 16.2 — Encryption in transit protects communication across architectural boundaries.

16. Encryption at Rest Data stored in databases, object storage, backups, and other persistent systems may require encryption at rest. Encryption reduces the impact of unauthorized access to physical or storage infrastructure. Encryption does not replace authorization. A compromised application with legitimate decryption access may still be able to read sensitive data. LearnForge • Engineering Handbook • 2026 144 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

17. Security Headers Web applications can reduce common browser-based risks through appropriate security headers and browser policies.

Header or Policy	Security Purpose
Content-Security-Policy	Restrict permitted content sources
Strict-Transport-Security	Encourage HTTPS-only access
X-Content-Type-Options	Reduce MIME-sniffing behavior
Referrer-Policy	Control referrer information

18. Rate Limiting Rate limiting protects resources by restricting how frequently a client or identity can perform an operation.

```
const limiter = rateLimit({ windowMs: 60_000, limit: 100, standardHeaders: true, legacyHeaders: false });
```

Rate limits should reflect the business operation. Login attempts, password reset requests, expensive searches, and ordinary reads may require different limits. Production Tip Do not use one global rate limit for every endpoint. Design limits around resource cost, abuse potential, and legitimate user behavior.

19. Service-to-Service Security Internal services should not automatically trust every other internal component. Internal networks can be compromised, misconfigured, or exposed through another vulnerability. LearnForge • Engineering Handbook • 2026 145 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

Service identities, mutual TLS where appropriate, scoped credentials, and explicit authorization policies can reduce lateral movement after a compromise.

20. Audit Logging Security-sensitive actions should generate structured audit records. These records help detect suspicious behavior and reconstruct important events during incident investigation.

```
const auditEvent = { actorId, action: "USER_ROLE_CHANGED", targetId: userId, timestamp: new Date().toISOString(), requestId };
Event Audit Value Login failure Detect credential attacks Permission change Track privilege changes Account deletion Reconstruct destructive actions Security configuration change Identify administrative changes
```

21. Threat Modeling Threat modeling is a structured process for identifying what can go wrong before attackers discover the weaknesses in production. A practical process is to identify assets, trust boundaries, actors, entry points, potential threats, and mitigations. Figure 16.3 — Threat modeling connects protected assets, threats, controls, and validation. LearnForge • Engineering Handbook • 2026 146 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

22. Defense in Depth Defense in depth means using multiple independent layers of protection. No single control should be assumed to stop every attack.

Layer	Example Control
Identity	Strong authentication
Application	Authorization and input validation
Network	Segmentation and encrypted communication
Data	Encryption and access controls
Operations	Monitoring and

incident response 23. Security Testing Security should be validated continuously rather than only before release. Different tests identify different classes of problems.

- Static analysis: identify suspicious source-code patterns.
- Dependency scanning: identify known vulnerable dependencies.
- Dynamic testing: evaluate running application behavior.
- Penetration testing: investigate realistic attack paths.
- Configuration review: detect insecure infrastructure settings.

24. Dependency and Supply-Chain Security Modern applications depend heavily on third-party packages, build tools, container images, plugins, and external services. A vulnerability in a dependency can become a vulnerability in the application. Teams should maintain dependency inventories, update important packages, review security advisories, and protect the build pipeline itself. LearnForge • Engineering Handbook • 2026 147

LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE Common Mistake Treating dependency security as an optional maintenance task. The software supply chain is part of the application's attack surface.

25. Incident Response Architecture Even strong preventive controls cannot guarantee that incidents will never occur. Architecture should therefore support detection, containment, investigation, recovery, and learning.

Phase Architectural Capability Detection Monitoring, alerts, audit logs Containment Credential rotation, isolation, access restriction Investigation Correlated logs and traces Recovery Backups, restoration procedures, redeployment Learning Post-incident review and architectural improvements

26. Practical Security Architecture Checklist

- Are authentication and authorization explicitly separated?
- Does every sensitive resource have an authorization policy?
- Are permissions based on least privilege?
- Are passwords stored using an appropriate password-hashing algorithm?
- Are sessions and tokens protected with appropriate lifetimes and controls?
- Are all important trust boundaries identified?
- Is untrusted input validated at server-side boundaries?
- Are database queries parameterized?

LearnForge • Engineering Handbook • 2026 148

LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE Are secrets kept outside source code?

- Is sensitive communication encrypted in transit?
- Are important security events recorded in audit logs?
- Are rate limits applied to abuse-sensitive operations?
- Are dependencies monitored for known vulnerabilities?
- Has threat modeling been performed for critical workflows?
- Can compromised credentials be revoked or rotated?

27. Key Takeaways

- Security is an architectural property, not a final development phase.
- Authentication establishes identity while authorization controls access.
- Least privilege limits the damage caused by compromised identities.
- Passwords must be protected with dedicated password-hashing mechanisms.
- APIs must validate input and enforce authorization on the server side.
- Secrets should be managed outside source code and protected throughout their lifecycle.
- Encryption protects data in transit and at rest but does not replace authorization.
- Threat modeling helps identify security weaknesses before production incidents.
- Defense in depth reduces dependence on any single security control.
- Security architecture must include detection, response, recovery, and continuous improvement.

28. Quick

Review 1. What is the difference between authentication and authorization? 2. Why is least privilege important? 3. Why should passwords never be stored as plaintext? 4. What risks do poorly protected sessions create? LearnForge • Engineering Handbook • 2026 149 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE 5. Why should APIs validate input on the server? 6. What is the purpose of rate limiting? 7. Why should secrets not be committed to source control? 8. What is threat modeling? 9. What does defense in depth mean? 10. Why is incident response part of security architecture? 29. Architect's Checklist • Have all major assets and trust boundaries been documented? • Is every sensitive operation explicitly authorized? • Are privileged identities minimized? • Are authentication credentials protected throughout their lifecycle? • Are service-to-service permissions explicitly defined? • Are security controls observable and auditable? • Are security failures designed to fail safely? • Have realistic attack scenarios been tested? • Can the organization detect and contain a compromised credential? • Is the security architecture reviewed whenever major system boundaries change? 30. Final Perspective The strongest security architecture is not the architecture with the largest number of security products. It is the architecture in which trust, permissions, data protection, failure behavior, monitoring, and recovery are deliberately designed. Security should therefore be treated as a continuous architectural practice. As systems grow, new services, dependencies, users, integrations, and data flows create new risks. Good LearnForge • Engineering Handbook • 2026 150 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE architecture makes those risks visible, constrains their impact, and gives the engineering team practical mechanisms for preventing, detecting, and recovering from security failures. Production Tip Review security architecture whenever a major trust boundary, data flow, authentication mechanism, external integration, or privileged capability is introduced. Security review is most effective before the boundary becomes expensive to change. LearnForge • Engineering Handbook • 2026 151 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

Chapter 17 Deployment & Infrastructure Architecture

Good software architecture must survive the transition from source code to production. Deployment architecture defines how applications are packaged, released, scaled, monitored, secured, and recovered across environments. What You Will Learn • How deployment architecture differs from application architecture. • How environments, configuration, and secrets should be separated. • Why containers improve deployment consistency. • How CI/CD pipelines reduce deployment risk. • How blue-green, rolling, and canary deployments work. • How health checks, autoscaling, and rollback strategies improve reliability. • How infrastructure should be designed for observability and recovery.

1. What Is Deployment Architecture?

Deployment architecture describes how software moves from development into running production infrastructure. It includes application packaging, servers, containers, networks, databases, configuration, secrets, deployment pipelines, monitoring, scaling, and recovery mechanisms. A well-designed application can still fail

operationally if deployment is manual, configuration is inconsistent, secrets are exposed, or rollback is difficult. Architect's Rule A system is not production-ready merely because the application works. Production readiness includes repeatable deployment, controlled configuration, observability, security, scaling, and recovery.

LearnForge • Engineering Handbook • 2026 152 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

2. Environment Separation

Most production systems use multiple environments. Development is optimized for rapid experimentation, while production is optimized for reliability and controlled change.

Environment	Purpose	Typical Characteristics
Development	Local engineering	Fast feedback and flexible configuration
Testing	Automated validation	Repeatable and isolated
Staging	Production-like verification	Similar infrastructure and configuration
Production	Real users and business workloads	Strict security and reliability controls

Environment separation reduces the probability that experimental changes affect real users. However, environments should not become completely different systems because large differences create deployment surprises.

3. Configuration Management

Configuration includes values that change between environments without changing application logic. Examples include database endpoints, service URLs, feature flags, timeout values, and operational limits.

```
interface AppConfig {
  port: number;
  databaseUrl: string;
  paymentServiceUrl: string;
  requestTimeoutMs: number;
}
const config: AppConfig = {
  port: Number(process.env.PORT ?? 3000),
  databaseUrl: process.env.DATABASE_URL!,
  paymentServiceUrl: process.env.PAYMENT_SERVICE_URL!,
  requestTimeoutMs: Number(process.env.REQUEST_TIMEOUT_MS ?? 1500)
};
```

Common Mistake

LearnForge • Engineering Handbook • 2026 153 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

Hard-coding production URLs, credentials, or environment-specific values inside source code.

Configuration should be externally supplied and managed through controlled deployment mechanisms.

4. Secrets Management

Passwords, API keys, private certificates, and database credentials are secrets. They should not be committed to source control or embedded directly in application code.

Secret	Risk	Better Approach
Database password	Unauthorized database access	Secret manager
API key	Unauthorized API usage	Managed secret storage
Private key	Identity compromise	Restricted secret store
Session signing key	Session forgery	Rotatable protected secret

Architect's Rule Treat secrets as replaceable credentials. Design systems so credentials can be rotated without rebuilding application logic.

5. Containers

Containers package an application together with its runtime dependencies. This reduces the difference between environments and makes application artifacts easier to transport.

LearnForge • Engineering Handbook • 2026 154 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

```
FROM node:22-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY dist ./dist
USER node
EXPOSE 3000
CMD ["node", "dist/server.js"]
```

Containers do not automatically make an architecture scalable or secure. They are a packaging and isolation mechanism. Poor application design remains poor.

when placed inside a container.

6. Immutable Deployment Artifacts

An immutable artifact is built once and then promoted through environments without rebuilding it for every environment. Figure 17.1 — A repeatable pipeline creates one validated artifact that can be promoted across environments.

7. CI/CD Architecture

Continuous Integration validates changes frequently. Continuous Delivery automates the preparation and controlled release of validated artifacts. Together, they reduce manual deployment work and shorten feedback loops.

LearnForge • Engineering Handbook • 2026 155 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

Pipeline Stage	Purpose	Failure Action
Lint	Detect code-quality issues	Stop pipeline
Unit tests	Validate local behavior	Stop pipeline
Build	Create deployable artifact	Stop pipeline
Security scan	Detect vulnerabilities	Block according to policy
Integration tests	Validate boundaries	Stop or quarantine release
Deployment	Release artifact	Rollback or halt

8. Health Checks

Health checks allow infrastructure and orchestration systems to determine whether an application is alive and capable of serving traffic.

```
app.get("/health/live", (_req, res) => { res.status(200).json({ status: "alive" }); });
app.get("/health/ready", async (_req, res) => {
  const databaseReady = await database.ping();
  if (!databaseReady) { return res.status(503).json({ status: "not-ready" }); }
  return res.status(200).json({ status: "ready" }); });
```

Liveness and readiness are different. Liveness asks whether the process should be restarted. Readiness asks whether the process should receive traffic.

Production Tip Do not make liveness checks depend on every external service. A temporary database failure should not necessarily cause the process itself to restart. Use readiness checks to control traffic instead.

LearnForge • Engineering Handbook • 2026 156 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

9. Rolling Deployments

A rolling deployment gradually replaces old application instances with new instances. It reduces the need for a complete service shutdown. Figure 17.2 — Rolling deployments replace instances gradually rather than switching the entire fleet at once.

10. Blue-Green Deployment

Blue-green deployment maintains two production environments. One environment serves traffic while the other receives the new release and is validated before traffic is switched.

Strategy Advantage Trade-off

Rolling	Efficient resource usage	Old and new versions coexist	Blue-green	Fast rollback	Requires additional capacity	Canary	Small exposure before full release	More sophisticated traffic control
---------	--------------------------	------------------------------	------------	---------------	------------------------------	--------	------------------------------------	------------------------------------

11. Canary Releases

A canary release sends a small percentage of traffic to the new version. Engineers monitor error rate, latency, resource consumption, and business metrics before increasing traffic.

Architect's Rule Deployment strategy should match failure impact. High-risk changes deserve smaller exposure, stronger monitoring, and easier rollback.

12. Rollback Strategy

A deployment is incomplete without a recovery strategy. If a release causes unexpected behavior, the team must be able to return to a known-good version.

LearnForge • Engineering Handbook • 2026 157 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

```
# Example conceptual release flow
deploy artifact-v42
if health_checks_fail; then
  rollback artifact-v41
fi
```

Rollback becomes more complicated

when database schemas are changed. Backward-compatible migrations are therefore important for safe application rollbacks.

13. Database Deployment Compatibility Application and database changes should not require an instantaneous switch between incompatible versions. Phase Action Expand Add new schema elements without removing old ones Deploy Release application compatible with both versions Migrate Move or backfill data Contract Remove obsolete schema elements after verification

14. Autoscaling Autoscaling changes application capacity according to workload. Scaling decisions can use CPU, memory, request rate, queue depth, or application specific metrics. Scaling more instances does not automatically solve every performance problem. If the database or another shared dependency is already saturated, adding application instances may increase the pressure. Common Mistake Assuming that horizontal scaling fixes every bottleneck. Always identify the actual constrained resource before increasing capacity.

LearnForge • Engineering Handbook • 2026 158 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

15. Infrastructure as Code Infrastructure as Code represents infrastructure configuration in versioned files rather than relying exclusively on manual console operations.

```
const service = { name: "course-api", replicas: 3, cpuLimit: "500m", memoryLimit: "512Mi", healthPath: "/health/ready" };
```

The exact tool may vary, but the architectural principle is consistent: infrastructure changes should be reviewable, repeatable, and auditable.

16. Network Architecture Production services should not automatically have unrestricted network access. Network boundaries can reduce the blast radius of compromised components and accidental traffic. Layer Control Goal Internet Load balancer / firewall Control external access Application Service policies Restrict service communication Database Private network rules Prevent direct public access Administration Restricted management path Protect operational interfaces

17. Observability Deployment architecture must expose enough telemetry to determine whether a release is healthy.

- Metrics: latency, throughput, error rate, saturation.
- Logs: structured application and infrastructure events.

LearnForge • Engineering Handbook • 2026 159 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

- Traces: request paths across distributed components.
- Deployment markers: correlation between releases and incidents.

```
{ "service": "course-api", "version": "42", "environment": "production", "latencyMs": 143, "status": "success" }
```

18. Resource Limits Resource limits protect shared infrastructure from a single workload consuming excessive capacity. Resource Why Limit It? CPU Prevent noisy-neighbor behavior Memory Reduce uncontrolled memory consumption Connections Protect databases and network services Queue depth Prevent unbounded backlog

19. Disaster Recovery Disaster recovery defines how a system returns to service after major infrastructure or data failures. Two important concepts are Recovery Point Objective (RPO) and Recovery Time Objective (RTO). RPO describes how much data loss is acceptable, while RTO describes how quickly service should be restored.

LearnForge • Engineering Handbook • 2026 160 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

Concept Question RPO How much recent data can

we afford to lose? RTO How quickly must the service recover? Production Tip A backup that has never been restored is an assumption, not a proven recovery mechanism. Regularly test restoration procedures.

20. Deployment Security

Deployment systems have powerful privileges and therefore become high-value security targets. CI/CD credentials should have the minimum permissions required for their job.

- Protect build credentials.
- Restrict deployment permissions.
- Scan dependencies and artifacts.
- Sign or verify important artifacts where appropriate.
- Audit production changes.

21. Deployment Decision Framework

Question Recommended Consideration Is downtime acceptable? Choose strategy accordingly Is rollback critical? Prefer easily reversible releases Is capacity limited? Rolling deployment may be preferable Is the change high risk? Use canary or staged exposure Is schema changing? Use backward-compatible migration

LearnForge • Engineering Handbook • 2026 161 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

22. Production Readiness Checklist

- Are development, testing, staging, and production appropriately separated?
- Is configuration externalized?
- Are secrets protected and rotatable?
- Is the deployment artifact reproducible?
- Are automated tests executed before deployment?
- Are health and readiness checks implemented?
- Is rollback tested?
- Are database migrations backward compatible?
- Are resource limits defined?
- Is production observability available?
- Are backups regularly tested?

23. Key Takeaways

- Deployment architecture determines how software becomes reliable production infrastructure.
- Configuration and secrets should remain outside application source code.
- Immutable artifacts improve deployment consistency.
- CI/CD makes releases repeatable and auditable.
- Health checks must distinguish process health from traffic readiness.
- Rolling, blue-green, and canary releases provide different risk profiles.
- Rollback and database compatibility must be designed before deployment.
- Autoscaling must consider the actual system bottleneck.
- Disaster recovery requires tested restoration, not merely backups.

LearnForge • Engineering Handbook • 2026 162 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

24. Quick Review

1. What is deployment architecture?
2. Why should configuration be separated from application code?
3. What is an immutable deployment artifact?
4. What is the difference between liveness and readiness?
5. How does a rolling deployment work?
6. When is a canary release useful?
7. Why are backward-compatible database migrations important?
8. What do RPO and RTO measure?

25. Architect's Checklist

- Can the entire deployment be reproduced?
- Can the previous release be restored quickly?
- Are deployment credentials minimally privileged?
- Can the team detect a bad release quickly?
- Can infrastructure changes be reviewed and audited?
- Have failure and recovery scenarios been tested?
- Are critical dependencies protected from resource exhaustion?
- Does the deployment strategy match the business risk?

LearnForge • Engineering Handbook • 2026 163 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

Chapter 18 Scalability & Performance Architecture

Scalable architecture is not simply about adding more servers. It is the disciplined design of

systems that maintain acceptable latency, throughput, reliability, and cost as workload increases. What You Will Learn • How scalability differs from performance optimization. • How to identify bottlenecks before scaling blindly. • When to use vertical and horizontal scaling. • How caching improves performance and where it introduces complexity. • How load balancing distributes traffic. • Why database architecture often becomes the scaling constraint. • How queues, backpressure, batching, and asynchronous processing improve throughput. • How to measure performance using meaningful engineering metrics.

1. What Is Scalability?

Scalability is the ability of a system to handle increasing workload while maintaining acceptable performance and reliability. Workload may increase because of more users, more requests, larger datasets, more background jobs, or more complex computations. A system that works perfectly for one thousand requests per minute may fail at one hundred thousand requests per minute if its architecture contains a non-scalable bottleneck.

Architect's Rule Do not scale the entire system when only one resource is constrained. Identify the bottleneck first, then increase capacity where it produces the greatest benefit.

LearnForge • Engineering Handbook • 2026

164 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

2. Performance vs Scalability

Performance and scalability are related but different concepts. Performance describes how efficiently a system handles a workload. Scalability describes how that behavior changes as workload increases.

Concept	Main Question	Example Metric
Latency	How long does one request take?	p95 response time
Throughput	How much work can be completed?	Requests per second
Scalability	How does capacity grow with resources?	Throughput vs instances
Efficiency	How much resource does work consume?	CPU per request

3. Find the Bottleneck First

A bottleneck is the resource that limits overall system capacity. Common bottlenecks include CPU, memory, database connections, disk I/O, network bandwidth, locks, external APIs, and queue workers.

Figure 18.1 — Scaling unrelated components does not solve the actual system bottleneck.

4. Vertical Scaling

Vertical scaling increases the capacity of an individual machine or instance. More CPU, memory, storage, or network capacity can improve performance without changing the application topology.

Advantage Simple operational model
Hardware has practical limits
Often requires fewer architectural changes
Large instances can become expensive
Useful for databases and stateful systems
Can preserve a single point of failure

LearnForge • Engineering Handbook • 2026

165 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

5. Horizontal Scaling

Horizontal scaling adds more instances of a service. It is particularly effective for stateless application servers because requests can be distributed across multiple instances.

```
const instances = [ "api-01", "api-02", "api-03", "api-04" ];
function chooseInstance(index: number) {
  return instances[index % instances.length];
}
```

Real production load balancers use more sophisticated algorithms and health signals, but the architectural idea is simple: distribute workload across available capacity.

Architect's Rule Horizontal scaling is easiest when application instances are stateless. Persistent state should

live in systems designed to manage shared state.

6. Load Balancing

A load balancer distributes incoming traffic among healthy application instances. It can also terminate TLS, perform health checks, enforce traffic policies, and route requests.

Strategy Behavior Typical Use Round robin Rotate requests across instances Similar-capacity servers Least connections Prefer less-busy instances Variable request duration Weighted Send different traffic percentages Canary releases Hash-based Route consistently by key Session or cache affinity

LearnForge • Engineering Handbook • 2026 166 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

7. Stateless Application Design

Stateless application instances do not depend on local memory for durable user state. This makes replacement and horizontal scaling easier. For example, authentication sessions can be represented using signed tokens or stored in a shared session system rather than being tied to one application instance.

```
interface Session {
    userId: string;
    expiresAt: number;
}
function validateSession(session: Session) {
    return session.expiresAt > Date.now();
}
```

Common Mistake Keeping important state only in process memory and then expecting horizontal scaling to work correctly. When an instance disappears, local memory disappears with it.

8. Caching

Caching stores frequently requested data closer to the component that needs it. A cache can reduce database load and lower response latency. Figure 18.2 — A cache can absorb repeated reads while the database remains the authoritative source.

9. Cache Invalidation

Caching introduces a difficult problem: keeping cached data sufficiently fresh. The classic challenge is cache invalidation.

LearnForge • Engineering Handbook • 2026 167 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

Strategy Behavior Trade-off TTL Expire entries after a period Simple but potentially stale Write-through Update cache with writes More consistent but more complex Cache-aside Application loads missing entries Flexible and common Explicit invalidation Delete/update cache on changes Requires reliable invalidation logic

Production Tip Cache only data whose freshness requirements are understood. A fast incorrect result can be worse than a slower correct result.

10. Database Scaling

Databases frequently become the primary scaling constraint because many applications depend on a relatively small amount of shared state. Possible techniques include indexing, query optimization, connection pooling, read replicas, partitioning, sharding, archival, and caching.

Technique Primary Goal Indexes Reduce query work Connection pooling Reuse database connections Read replicas Scale read workloads Partitioning Divide large datasets Sharding Distribute data across database nodes

11. Query Optimization

Before changing database architecture, inspect the queries. An inefficient query executed millions of times can create more load than an entire new feature.

LearnForge • Engineering Handbook • 2026 168 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

```
EXPLAIN ANALYZE SELECT id, title FROM courses WHERE published = true ORDER BY created_at DESC LIMIT 20;
```

Indexes should be based on real query patterns rather than created arbitrarily. Every index also introduces storage and write-maintenance cost.

12. Connection Pooling

Opening a new database connection

for every request can create unnecessary overhead and can exhaust database connection limits.

```
const pool = new DatabasePool({
  maxConnections: 20,
  idleTimeoutMs: 30000
});
async function getCourse(id: string) {
  const connection = await pool.acquire();
  try {
    return await connection.query(
      "SELECT * FROM courses WHERE id = ?",
      [id]
    );
  } finally {
    connection.release();
  }
}
```

Architect's Rule Connection pools must be sized according to database capacity, application concurrency, and workload. More connections do not automatically mean more throughput.

13. Asynchronous Processing

Not every operation needs to complete during the user's request. Expensive background work can be moved to queues and workers.

LearnForge • Engineering Handbook • 2026 169

LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

```
await queue.publish({
  type: "GENERATE_REPORT",
  reportId,
  userId
});
return {
  status: "accepted",
  reportId
};
```

This allows the API to respond quickly while workers process the expensive operation independently.

14. Queue-Based Scaling

Queues allow producers and consumers to operate at different speeds. Workers can be increased when backlog grows and reduced when demand falls.

Figure 18.3 — Queues decouple producers from workers and allow worker capacity to scale independently.

15. Backpressure

When consumers cannot process work quickly enough, producers must slow down or reject additional work. This prevents queues and resources from growing without bound.

Common Mistake Treating queue capacity as infinite. A queue is a buffer, not a substitute for capacity planning.

16. Batching

Batching combines multiple small operations into a larger operation. This can reduce network round trips, transaction overhead, and per-request processing costs.

LearnForge • Engineering Handbook • 2026 170

LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

```
const users = await userRepository.findMany([
  "u101", "u102", "u103", "u104"
]);
```

Batching must be bounded. Extremely large batches can increase memory usage, latency, and failure impact.

17. Rate Limiting

Rate limiting protects services from excessive traffic and helps enforce fair resource allocation.

Algorithm Concept Fixed window Count requests within fixed intervals Sliding window Measure traffic over a moving interval Token bucket Allow controlled bursts using accumulated tokens Leaky bucket Process requests at a controlled rate

18. Performance Budgets

A performance budget defines acceptable limits for latency, resource usage, payload size, or other performance characteristics.

LearnForge • Engineering Handbook • 2026 171

LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

Metric Example Budget

API p95 latency < 300 ms Error rate < 1% Payload size < 500 KB Database query time < 100 ms for critical reads

Budgets should be based on user experience and business requirements rather than arbitrary numbers.

19. Latency Percentiles

Average latency can hide serious slow requests. Percentiles provide a better view of tail behavior.

- p50: median request latency.
- p95: latency below which 95% of requests complete.
- p99: latency below which 99% of requests complete.

A service with a good average but poor p99 latency may still provide a poor experience for a meaningful number of users.

20. Load Testing

Load testing measures system

behavior under controlled workload. It can reveal saturation points, memory leaks, slow queries, connection exhaustion, and unexpected scaling behavior. # Conceptual load-test command `loadtest \ --requests 10000 \ --concurrency 100 \ https://api.example.com/courses` Load tests should represent realistic request distributions. Testing only a single simple endpoint can produce misleading conclusions.

LearnForge • Engineering Handbook • 2026 172 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE 21. Capacity Planning Capacity planning estimates how much infrastructure is required for expected and unexpected workload. Input Why It Matters Peak requests Determines required request capacity Request cost Determines CPU and memory demand Growth rate Predicts future capacity Failure margin Provides resilience during instance loss Business events Predicts temporary traffic spikes

22. Cost-Aware Scaling Performance improvements have economic costs. A faster architecture is not automatically better if the additional infrastructure provides little business value. Architects should consider the relationship between latency, reliability, capacity, and infrastructure cost. Architect's Rule Optimize for business value, not benchmark numbers alone. A ten percent performance improvement that doubles infrastructure cost may be a poor engineering decision.

LearnForge • Engineering Handbook • 2026 173 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE 23. Scalability Failure Modes Failure Mode Why It Happens Mitigation Indexes, caching, replicas, Database saturation Shared state becomes bottleneck partitioning Cache stampede Many requests rebuild expired data Jitter, locking, request coalescing Queue overload Producers exceed consumer capacity Backpressure and autoscaling Connection Too many concurrent connections Pooling and limits exhaustion Clients retry overloaded Retry storm Backoff and retry budgets dependencies

24. Practical Scalability Checklist • Has the actual bottleneck been measured? • Can application instances scale horizontally? • Is important state stored outside individual instances? • Is traffic distributed across healthy instances? • Is caching used where freshness requirements permit it? • Are database queries indexed and measured? • Are connection pools bounded? • Can expensive work be processed asynchronously? • Is queue growth monitored? • Are rate limits and backpressure implemented? • LearnForge • Engineering Handbook • 2026 174 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE Are p95 and p99 latency monitored? • Has realistic load testing been performed?

25. Key Takeaways • Scalability begins with identifying bottlenecks, not adding servers blindly. • Horizontal scaling works best with stateless application instances. • Load balancers distribute traffic and remove unhealthy instances from service. • Caching can improve latency and reduce database load but introduces freshness concerns. • Databases often become the limiting resource in large systems. • Queues and asynchronous processing separate request latency from background work. • Backpressure protects systems from uncontrolled workload growth. • Percentile latency is more informative than averages for tail behavior. • Performance must be evaluated together with reliability and infrastructure cost.

26. Quick Review 1. What

is the difference between performance and scalability? 2. Why should architects identify bottlenecks before scaling? 3. What is the difference between vertical and horizontal scaling? 4. Why are stateless services easier to scale? 5. What problem does caching solve? 6. Why can caching introduce correctness problems? 7. How do queues improve scalability? 8. Why are p95 and p99 useful? 9. What is backpressure? 10. Why should scalability decisions consider cost? 27. Architect's Checklist • LearnForge • Engineering Handbook • 2026 175 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

Is capacity planning based on measured workload? • Are bottlenecks continuously monitored? • Can critical services scale without redesigning the entire platform? • Are shared dependencies protected from overload? • Are caches monitored for hit rate and staleness? • Are queues bounded and observable? • Are performance budgets documented? • Are scalability assumptions validated with load tests? • Is the cost of scaling understood? LearnForge • Engineering Handbook • 2026 176 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

Chapter 19 Reliability & Resilience Architecture

Reliable systems are not systems that never fail. They are systems designed to continue delivering their most important capabilities when components fail, dependencies become slow, traffic increases, or unexpected conditions occur. What You Will Learn • The difference between reliability, availability, and resilience. • How to identify failure domains and critical dependencies. • How timeouts, retries, backoff, and circuit breakers work together. • How redundancy and graceful degradation improve availability. • How SLOs, error budgets, and reliability metrics guide architecture. • How to design systems that recover safely from operational failures.

1. Reliability Is a System Property

Reliability is not created by adding one monitoring tool or one backup mechanism. It emerges from the interaction of application code, infrastructure, databases, networks, dependencies, deployment processes, and human operations. A system may contain highly reliable individual components and still be unreliable as a whole if those components form fragile dependency chains. Reliability therefore has to be designed at the system level.

Architect's Rule

Do not ask only whether individual components are reliable. Ask what happens to the complete system when each critical dependency becomes slow, unavailable, inconsistent, or overloaded. LearnForge • Engineering Handbook • 2026 177 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

2. Reliability vs Availability vs Resilience

Concept Meaning Architectural Focus

Reliability	Ability to perform correctly over time	Correctness and dependable operation
Availability	Ability to remain accessible when	Uptime and service accessibility needed
Resilience	Ability to withstand and recover from	Failure handling and recovery failure
Recoverability	Ability to restore service after	Backups, restoration, and operational
Disruption procedures	These concepts overlap but are not identical.	A system can be highly available while temporarily returning incorrect information, or it can be correct but unavailable during a major infrastructure failure.

3. Failure Domains

A failure domain is a group of resources that can fail together because they share

infrastructure, configuration, geography, or another dependency. Failure Domain Example Failure Protection Process Application crash Process supervision and restart Host Machine failure Multiple instances Zone Infrastructure outage Multi-zone deployment Region Regional disruption Multi-region strategy Provider External platform outage Alternative providers or graceful degradation Production Tip Map the failure domains of every critical component. Redundancy is valuable only when redundant instances do not share the same hidden failure domain. LearnForge • Engineering Handbook • 2026 178 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE 4. Redundancy Redundancy means maintaining multiple resources capable of performing the same critical function. If one instance fails, another can continue serving requests. Figure 19.1 — Redundant instances allow traffic to continue when one instance fails. 5. Health Checks Health checks allow infrastructure or orchestration systems to determine whether an application is capable of receiving traffic. `app.get("/health/live", (_req, res) => { res.status(200).json({ status: "alive" }); }); app.get("/health/ready", async (_req, res) => { const databaseReady = await database.ping(); if (!databaseReady) { return res.status(503).json({ status: "not-ready" }); } res.status(200).json({ status: "ready" }); });` Liveness and readiness should not always mean the same thing. A process may be alive while temporarily unable to serve production traffic. 6. Timeouts Timeouts prevent a dependency from consuming resources indefinitely. Every network operation should have a deliberately chosen upper bound. LearnForge • Engineering Handbook • 2026 179 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE `async function callInventory(productId: string) { const response = await fetch( `https://inventory.internal/products/${productId}`, { signal: AbortSignal.timeout(1200) } ); if (!response.ok) { throw new Error("Inventory request failed"); } return response.json(); }` Common Mistake Using extremely large timeouts because they appear safer. Large timeouts can allow slow dependencies to consume application resources and amplify an incident. 7. Retry with Exponential Backoff Retries can recover from temporary failures, but they should be bounded and spaced apart. Exponential backoff reduces the probability that many clients will immediately retry an already overloaded dependency. LearnForge • Engineering Handbook • 2026 180 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE `async function retry(operation: () => Promise, attempts = 3): Promise { let lastError: unknown; for (let attempt = 0; attempt < attempts; attempt++) { try { return await operation(); } catch (error) { lastError = error; if (attempt === attempts - 1) { throw error; } const delay = 200 * 2 ** attempt; await new Promise(resolve => setTimeout(resolve, delay)); } } throw lastError; }` 8. Retry Safety Not every operation should be retried. A retry is safe only when repeating the operation cannot create unintended side effects or when idempotency mechanisms make repetition safe. Typical Retry Operation Reason Safety Read request Usually safer Normally does not create a new state change Duplicate execution can create financial Create payment Requires protection damage Send notification Depends on

provider Duplicate messages may be undesirable Update with idempotency Safer Repeated request can be recognized key LearnForge • Engineering Handbook • 2026 181 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE 9. Circuit Breakers A circuit breaker prevents an application from repeatedly calling a dependency that is known to be unhealthy. State Behavior Goal Closed Requests flow normally Normal operation Open Requests fail quickly Protect resources Half-open Limited requests test recovery Determine whether service recovered 10. Graceful Degradation Graceful degradation means reducing secondary functionality while preserving the most important business capabilities. Failed Dependency Primary Function Degraded Behavior Recommendation service Product browsing Show popular products Analytics service Core transaction Queue analytics events Email provider Account operation Store notification for later Search index Content access Use acceptable fallback search Architect's Rule When everything cannot remain available, protect the business-critical path first. Secondary features should fail safely rather than taking the entire system down. 11. Bulkheads Bulkheads isolate resources so that failure in one workload does not consume capacity required by another workload. LearnForge • Engineering Handbook • 2026 182 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE Figure 19.2 — Bulkhead isolation prevents one workload from consuming all resources required by another. 12. Backpressure Backpressure prevents an application from accepting unlimited work when downstream capacity is constrained. Queues, concurrency limits, rate limits, and bounded buffers are common backpressure mechanisms. The objective is to preserve system stability rather than maximize incoming work at any cost. 13. Rate Limiting Rate limiting controls how much traffic a client or workload can generate during a period of time. Strategy Strength Typical Use Fixed window Simple Basic API limits Sliding window More accurate traffic control Public APIs Token bucket Allows controlled bursts Production APIs Concurrency limit Limits simultaneous work Expensive operations 14. Error Budgets An error budget represents the amount of unreliability that is acceptable within a defined service-level objective. For example, if a service has a 99.9% monthly availability target, a small amount of downtime is tolerated. The purpose is not to encourage failure but to create an explicit balance between reliability work and feature delivery. Production Tip LearnForge • Engineering Handbook • 2026 183 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE Use error budgets to make reliability measurable. When the budget is being consumed too quickly, prioritize reliability improvements before adding additional operational risk. 15. Service-Level Indicators SLI Measurement Example Availability Successful requests / total requests 99.95% Latency Request response time p95 < 300 ms Durability Data successfully preserved Backup restoration success Freshness Age of available data Events processed within target 16. Recovery Objectives Term Meaning Architectural Question RTO Recovery Time Objective How quickly must service return? RPO Recovery Point Objective How much data loss is acceptable? RTO and RPO should be business-driven. A critical financial system may

require significantly stronger recovery objectives than a low-priority internal reporting tool.

17. Backup Is Not Recovery A backup that has never been restored is an assumption, not a verified recovery mechanism. Recovery testing should verify that backups can actually be restored, that the restored data is usable, and that the team understands the operational steps required to return the service to production. Common Mistake Assuming that successful backup creation proves disaster recovery capability. A backup system must be paired with restoration procedures and periodic recovery testing. LearnForge • Engineering Handbook • 2026 184 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

18. Deployment Resilience Deployments are one of the most common sources of production incidents. Reliable architecture therefore includes safe release strategies. Strategy Approach Risk Control Rolling deployment Replace instances gradually Reduces simultaneous impact Blue-green Maintain two environments Fast rollback Canary Release to a small percentage first Early detection Feature flags Separate deployment from activation Controlled exposure

19. Failure Injection Resilience cannot be validated only during normal operation. Teams should test what happens when important dependencies fail. • Terminate an application instance. • Introduce network latency. • Return dependency errors. • Exhaust a controlled resource pool. • Simulate database unavailability. • Test restoration from backup. Failure testing should be controlled, observable, and performed within an appropriate environment and risk boundary.

20. Observability Reliable systems require telemetry that helps engineers detect, understand, and recover from failures. LearnForge • Engineering Handbook • 2026 185 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE Signal Purpose Example Metrics Detect trends and anomalies Error rate, latency, saturation Logs Understand individual events Request failure details Traces Follow distributed workflows Cross-service latency Alerts Notify operators SLO violation

21. Incident Response Architecture and incident response are closely related. A system that is technically recoverable but impossible for operators to understand is not operationally resilient. Incident procedures should identify ownership, escalation paths, safe mitigation actions, rollback procedures, and communication responsibilities.

22. Graceful Shutdown Applications should shut down in a controlled manner. Abrupt termination can drop requests, interrupt background work, or leave temporary state in an inconsistent condition. `process.on("SIGTERM", async () => { server.close(); await worker.stop(); await database.close(); process.exit(0); });`

23. Resilience Patterns Working Together Figure 19.3 — Resilience is strongest when multiple complementary patterns protect the system rather than relying on one mechanism. LearnForge • Engineering Handbook • 2026 186 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

24. Practical Reliability Checklist • Are all critical dependencies identified? • Does every remote dependency have a bounded timeout? • Are retries limited and protected by backoff? • Are non-idempotent operations protected from duplicate execution? • Are critical services redundant across appropriate failure domains? • Are readiness and liveness checks correctly

separated? • Are circuit breakers or equivalent protection used where appropriate? • Are degraded modes explicitly defined? • Are backups regularly restored in testing? • Are RTO and RPO requirements documented? • Are important workflows observable across service boundaries? • Are deployment rollback procedures tested? 25. Key Takeaways • Reliability must be designed as a system property. • Redundancy is effective only when hidden shared failure domains are avoided. • Timeouts prevent slow dependencies from consuming unlimited resources. • Retries require backoff, limits, and idempotency awareness. • Circuit breakers, bulkheads, and backpressure reduce cascading failures. • Graceful degradation protects critical business functions. • SLOs and error budgets make reliability measurable. • Backups are useful only when restoration is verified. • Resilience requires controlled failure testing and strong observability. LearnForge • Engineering Handbook • 2026 187 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE 26. Quick Review 1. What is the difference between reliability and resilience? 2. Why are failure domains important? 3. Why can redundancy fail to improve availability? 4. Why should timeouts be bounded? 5. When is a retry unsafe? 6. What problem does a circuit breaker solve? 7. How does graceful degradation protect critical functionality? 8. What are RTO and RPO? 9. Why should backups be tested through restoration? 10. How does observability improve incident response? 27. Architect's Checklist • Have reliability requirements been defined from business impact? • Have critical failure domains been mapped? • Are redundancy and recovery mechanisms aligned with those domains? • Does every critical dependency have an explicit failure policy? • Are retry, timeout, and circuit-breaker policies documented? • Can the system degrade without losing its critical business path? • Are RTO, RPO, SLO, and error-budget targets measurable? • Are disaster-recovery procedures tested periodically? • Can operators diagnose failures quickly using telemetry? • Have realistic failure scenarios been exercised safely? LearnForge • Engineering Handbook • 2026 188 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE Chapter 20 Observability & Monitoring Architecture Observability turns complex system behavior into actionable evidence. A reliable architecture must make it possible to understand what happened, where it happened, why it happened, and which users or business operations were affected. What You Will Learn • Why observability is an architectural capability rather than only a monitoring tool. • How logs, metrics, traces, and events complement each other. • How to design useful telemetry without creating excessive noise. • How correlation IDs and distributed tracing connect service activity. • How SLI, SLO, and error-budget concepts support engineering decisions. • How alerting should focus on actionable user and business impact. • How to design observability for failures, performance problems, and security events. 1. What Is Observability? Observability is the ability to understand the internal state and behavior of a system by examining the information it produces. In modern distributed systems, that information commonly includes logs, metrics, traces, events, profiles, and structured diagnostic data. Monitoring usually asks whether known conditions are healthy.

Observability goes further by helping engineers investigate unexpected conditions. A useful observability architecture should help answer questions that were not necessarily predicted when the system was originally designed. Architect's Rule Design telemetry around questions engineers and operators need to answer, not around the number of dashboards or log lines the platform can generate. LearnForge • Engineering Handbook • 2026 189 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

2. The Three Core Signals Signal Best For Example Logs Detailed event context Payment request rejected Metrics Trends and aggregation HTTP error rate = 2.4% Traces Request flow across boundaries API → Order → Payment These signals are complementary. Metrics can reveal that a problem exists, logs can explain individual events, and traces can show where latency or failure occurred across a distributed request. Figure 20.1 — Logs, metrics, and traces provide complementary views of system behavior.

3. Structured Logging Logs become more useful when important fields are structured rather than embedded only inside free-form text. Structured logs can be searched, aggregated, filtered, and correlated automatically. `logger.info({ event: "payment.completed", paymentId, orderId, userId, amount, currency, durationMs, traceId });` A production log should contain enough context to investigate an event without exposing secrets or unnecessary personal information. LearnForge • Engineering Handbook • 2026 190 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

Field Purpose timestamp When the event occurred service Which component produced the event level Severity or importance traceId Connect event to a request trace requestId Identify an individual request event Stable machine-readable event name Common Mistake Logging secrets, access tokens, passwords, full payment credentials, or unnecessary sensitive data simply because the application has access to it. Operational visibility must not become a data-leak mechanism.

4. Log Levels Log levels help distinguish normal diagnostic information from conditions that require attention. Exact levels vary by platform, but a common model includes debug, info, warn, and error. Level Typical Use DEBUG Detailed development or troubleshooting information INFO Important normal application events WARN Unexpected condition that does not necessarily indicate failure ERROR Operation failed or requires investigation Production systems should avoid generating enormous volumes of low-value logs. Excessive logging increases storage costs and makes important events harder to find. LearnForge • Engineering Handbook • 2026 191 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

5. Metrics Metrics are numerical measurements collected over time. They are especially effective for identifying trends, capacity problems, and changes in system behavior. Metric Type Example Useful Question Counter HTTP requests How many occurred? Gauge Queue depth What is the current value? Histogram Request latency How is latency distributed? Latency distributions are often more informative than simple averages. An average response time may look healthy while a small but important percentage of users experience extremely slow requests.

6. The Four Golden Signals A practical starting point for service monitoring is the four golden signals: latency,

traffic, errors, and saturation. Signal Question Example Latency How long does work take? p95 API latency Traffic How much demand exists? Requests per second Errors How often does work fail? HTTP 5xx rate Saturation How close are resources to capacity? CPU or queue utilization Production Tip Start service dashboards with a small set of high-value signals. Add deeper diagnostics after the primary health picture is clear. LearnForge • Engineering Handbook • 2026 192 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

7. Distributed Tracing A trace represents a logical request as it travels through multiple components. Each individual operation within that trace is represented by a span. Figure 20.2 — Distributed tracing shows how one request consumes time across multiple services.

8. Correlation IDs A correlation identifier connects related operations. In a distributed system, the identifier can travel through HTTP requests, messages, jobs, and service boundaries.

```
const correlationId = request.headers.get("x-correlation-id") ??
crypto.randomUUID(); logger.info({ event: "request.started", correlationId, path:
request.url });
```

Correlation IDs are particularly useful during incident investigation because an engineer can search for one identifier and reconstruct the path of a specific operation across multiple components.

9. Trace Context Propagation Tracing only becomes valuable across services when trace context is propagated correctly. The originating service should pass the relevant trace context to downstream services and message consumers. LearnForge • Engineering Handbook • 2026 193 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

Boundary Propagation Method HTTP Trace context headers Message queue Message metadata Background job Job context Scheduled task Generated root trace

10. Service-Level Indicators A Service-Level Indicator, or SLI, is a quantitative measurement of a service behavior that matters to users or the business. Examples include successful request percentage, request latency, payment completion rate, or availability of a critical workflow. SLI Measurement Availability $\frac{\text{Successful valid requests}}{\text{total valid requests}}$ Latency Percentage of requests below a defined threshold Checkout success $\frac{\text{Completed checkouts}}{\text{checkout attempts}}$ Message processing $\frac{\text{Successfully processed messages}}{\text{received messages}}$

11. Service-Level Objectives A Service-Level Objective, or SLO, defines the target for an SLI over a specified period. For example, an engineering team might define an objective that 99.9 percent of valid API requests should succeed during a monthly measurement period. The exact target should reflect business requirements rather than arbitrary numbers. Architect's Rule LearnForge • Engineering Handbook • 2026 194 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

An SLO should describe an outcome that matters. A dashboard full of internal resource metrics is not a substitute for measuring whether users can successfully complete important operations.

12. Error Budgets An error budget represents the amount of unreliability permitted by an SLO. If a service has a 99.9 percent availability objective, the remaining fraction represents the allowed failure budget during the measurement period. Error budgets create a practical connection between reliability and delivery speed. If the budget is being

consumed rapidly, the team may need to prioritize reliability work instead of introducing additional operational risk.

Condition Possible Engineering Response
 Budget healthy Continue planned delivery
 Budget declining quickly Investigate reliability risks
 Budget exhausted Prioritize stabilization

13. Alerting Architecture

An alert should represent a condition that requires human or automated action. If an alert does not lead to a meaningful response, it may be better represented as a dashboard metric or informational notification.

Alert Quality Reason
 Critical checkout failures rising High value Direct business impact
 CPU reached 70% Context dependent
 May not require action
 One debug log appeared Low value
 Usually not actionable
 Common Mistake LearnForge

- Engineering Handbook • 2026 195

LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

Creating alerts for every abnormal metric. Alert fatigue causes engineers to ignore notifications, including important ones. Prefer fewer alerts with clear ownership and documented actions.

14. Alert Severity

Severity should reflect impact and urgency, not simply technical abnormality. A minor internal warning may have little operational importance, while a small failure rate on a critical payment workflow may deserve immediate attention.

Severity Typical Meaning
 Critical Major user or business impact
 High Significant degradation requiring prompt action
 Medium Important issue that can be investigated during working hours
 Low Informational or preventive condition

15. Dashboard Design

A useful dashboard should support a specific operational decision. Avoid building one enormous dashboard containing every metric in the platform. A service dashboard can begin with traffic, success rate, latency, saturation, dependency health, and recent deployments. Deeper dashboards can then provide infrastructure or component-specific diagnostics.

Figure 20.3 — Dashboards should move from high-level service health toward deeper diagnostic evidence.

16. Monitoring Dependencies

A service can appear healthy while an important dependency is failing. For this reason, observability should include dependency latency, error rates, timeouts, saturation, and availability.

- LearnForge • Engineering Handbook • 2026 196

LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

Dependency Useful Signals
 Database Latency, connection pool, errors, saturation
 Message broker Queue depth, processing latency, consumer failures
 External API Latency, status codes, timeout rate
 Cache Hit ratio, memory, evictions, latency

17. Deployment Correlation

Many production incidents begin shortly after a deployment. Observability systems should therefore make deployments visible alongside application metrics. When engineers can see that error rate increased immediately after a specific release, the investigation can become significantly faster.

Production Tip Annotate dashboards and telemetry with deployment versions, commit identifiers, feature flags, and environment information where appropriate.

18. Health Checks

Health checks provide a standardized way to determine whether an application or dependency is operational.

```
app.get("/health/live", (_req, res) => {
  res.status(200).json({ status: "ok" });
});
app.get("/health/ready", async (_req, res) => {
  const databaseReady = await database.ping();
  res.status(databaseReady ? 200 :
```

503).json({ status: databaseReady ? "ready" : "not-ready" }); }); LearnForge • Engineering Handbook • 2026 197 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE Liveness and readiness should be distinguished. A process can be alive while being temporarily unable to serve traffic safely. 19. Synthetic Monitoring Synthetic monitoring periodically performs controlled workflows to detect problems before users report them. For example, a synthetic test may load a public page, authenticate using a dedicated test account, or execute a safe checkout simulation. Synthetic checks should never create real financial or destructive operations unless the environment is explicitly designed for that purpose. 20. Profiling and Resource Diagnosis Metrics can reveal that CPU or memory consumption is abnormal, but profiling can help explain why. CPU profiles, memory profiles, and database query analysis can expose inefficient code paths. Problem Useful Diagnostic High CPU CPU profile Memory growth Heap or memory profile Slow database Query analysis High latency Distributed trace 21. Observability and Security Observability data often contains operational details and may contain user identifiers, request information, or other sensitive information. Logging systems must therefore follow security and privacy requirements. • Never log authentication secrets. • Minimize sensitive identifiers. LearnForge • Engineering Handbook • 2026 198 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE • Restrict access to telemetry platforms. • Encrypt telemetry in transit and at rest where required. • Define retention periods. • Audit access to sensitive operational data. 22. Cost Control Observability can become expensive at scale. High-cardinality metrics, verbose logs, long retention periods, and full-fidelity traces can generate large volumes of data. Cost control should not mean eliminating important telemetry. Instead, engineers should classify data by value and retention requirements. Data Possible Strategy Critical errors Longer retention Debug logs Shorter retention or sampling High-volume traces Intelligent sampling Rare security events Careful retention and access control 23. Observability During Incidents During an incident, the primary goal is not to produce perfect explanations immediately. The goal is to establish impact, stabilize the system, and collect evidence that supports safe decisions. LearnForge • Engineering Handbook • 2026 199 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE Incident Question Useful Evidence Are users affected? Error rate and business SLIs When did it begin? Time-series metrics and deployment events Where is the failure? Distributed traces and dependency metrics What changed? Deployments and configuration history Is recovery occurring? Live SLIs and error rates Architect's Rule During incidents, prioritize signals that support decisions. More telemetry is not automatically better telemetry. 24. Practical Observability Checklist • Are important services producing structured logs? • Are logs free of secrets and unnecessary sensitive information? • Are latency, traffic, errors, and saturation measured? • Are critical requests traceable across service boundaries? • Are correlation or trace identifiers propagated? • Are critical business workflows represented by meaningful SLIs? • Are SLOs defined for important user-facing operations? • Are alerts actionable and owned by a responsible

team? • Are deployments visible in operational dashboards? • Are important dependencies monitored? • Are health checks separated into liveness and readiness where appropriate? • LearnForge • Engineering Handbook • 2026 200

LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

Are telemetry retention and cost policies defined? 25. Key Takeaways • Observability is the ability to understand system behavior from produced evidence. • Logs, metrics, and traces provide complementary perspectives. • Structured logging makes operational evidence easier to search and correlate. • Metrics are particularly effective for trends, capacity, and service health. • Distributed tracing connects activity across service boundaries. • SLIs and SLOs should represent outcomes that matter to users or the business. • Alerts should be actionable rather than merely interesting. • Telemetry must be designed with security, privacy, retention, and cost in mind. 26. Quick Review 1. What is the difference between monitoring and observability? 2. Why are logs, metrics, and traces complementary? 3. What makes a log entry useful during an incident? 4. Why are latency distributions often more useful than averages? 5. What is the purpose of a correlation ID? 6. What is an SLI? 7. What is an SLO? 8. Why can excessive alerts create operational risk? 9. Why should deployments be correlated with telemetry? 10. How can observability data create security or privacy risks? 27. Architect's Checklist • Can engineers determine whether users are affected? • Can the team identify when a problem started? • LearnForge • Engineering Handbook • 2026 201

LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

Can one request be followed across critical service boundaries? • Can engineers distinguish application failure from dependency failure? • Are important business operations measurable? • Are alert thresholds connected to meaningful operational outcomes? • Are telemetry costs monitored as the system scales? • Are sensitive telemetry fields controlled and protected? • Can the team investigate incidents without relying on guesswork? • Is observability tested as part of reliability engineering? LearnForge • Engineering Handbook • 2026 202

LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

Chapter 21 Testing & Quality

Architecture

Testing is not merely a final verification activity. It is an architectural capability that provides confidence, protects boundaries, exposes design problems, and allows software to evolve safely as complexity increases. What You Will Learn • How to design a balanced testing strategy. • When to use unit, integration, contract, and end-to-end tests. • How testability influences software architecture. • How mocks, stubs, fakes, and spies should be used responsibly. • How CI quality gates prevent defective software from reaching production. • How to test reliability, security, performance, and failure behavior. 1. Testing as an Architectural Capability

Testing is often treated as a separate activity performed after implementation. In a mature architecture, testing is part of the design itself. Components should have clear responsibilities, predictable interfaces, and controllable dependencies so that their behavior can be verified efficiently. A system that is difficult to test is frequently difficult to change. Tight coupling, hidden global state, unpredictable external

dependencies, and large responsibilities increase the cost of verification. Architect's Rule Design important components so that their behavior can be verified without requiring the entire production system to run.

2. What Does a Good Test Prove? A useful test should provide evidence about a meaningful behavior. The goal is not to maximize the number of test files or lines of test code. The goal is to reduce uncertainty about important system behavior.

LearnForge • Engineering Handbook • 2026 203 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

Test Question

Useful Evidence Example Does the business rule work? Unit test Discount calculation Do components integrate? Integration test Repository with database Does an API contract remain compatible? Contract test Consumer/provider agreement Does the user workflow work? End-to-end test Login and checkout

3. The Testing Pyramid

A common testing strategy uses many fast, focused tests at the lower levels and fewer expensive tests at the higher levels. This is often represented as a testing pyramid.

Figure 21.1 — A balanced testing strategy emphasizes fast tests while maintaining enough higher-level coverage for important workflows.

4. Unit Testing

Unit tests verify small pieces of behavior in isolation. A unit may be a function, class, or focused domain component depending on the architecture.

```
function calculateTotal( price: number, quantity: number ): number { if (quantity <= 0) { throw new Error('Quantity must be positive'); } return price * quantity; } test('calculates total price', () => { expect(calculateTotal(25, 4)).toBe(100); });
```

The test is fast because it does not require a database, network, payment provider, or browser. This makes unit tests useful for rapid developer feedback.

LearnForge • Engineering Handbook • 2026 204 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

5. Testing Business Rules

Business rules are among the highest-value targets for automated tests. They represent decisions that directly affect the behavior of the product.

Business Rule Example

Test Free users cannot access premium content Premium request is rejected Orders above a threshold receive a discount Discount is calculated correctly Cancelled orders cannot be shipped Shipping operation is rejected Expired subscriptions lose premium access Access becomes unavailable after expiration

Production Tip

Prioritize tests around business rules that can cause financial loss, security problems, legal exposure, or serious user impact.

6. Testability and Dependency Injection

Dependency injection improves testability because components can receive their dependencies instead of constructing them internally.

```
interface PaymentGateway { charge(amount: number): Promise; } class CheckoutService { constructor( private readonly payments: PaymentGateway ) {} async checkout(amount: number) { return this.payments.charge(amount); } }
```

During testing, a controlled implementation can be supplied instead of a real payment provider.

LearnForge • Engineering Handbook • 2026 205 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

7. Mocks, Stubs, Fakes, and Spies

Test doubles replace real dependencies with controlled alternatives. They should be used to isolate behavior, not to reproduce every implementation detail of the production dependency.

Double Purpose

Typical Use Stub Provides predetermined data Return a

known user Mock Verifies expected interaction Check payment call Fake Working simplified implementation In-memory repository Spy Records actual calls Verify logging or events Common Mistake Mocking every dependency and asserting implementation details can create brittle tests. Prefer verifying meaningful behavior and stable contracts.

8. Integration Testing

Integration tests verify that multiple real components work together. They are especially valuable at architectural boundaries such as databases, queues, HTTP clients, and authentication systems.

Boundary Integration Test Failure It Can Detect Database Repository test

Incorrect SQL or schema assumptions HTTP API Endpoint test Serialization or routing errors Message broker Consumer test Incorrect event handling Authentication Authorization test Incorrect access policy

LearnForge • Engineering Handbook • 2026 206

LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE 9. Database Testing

Database behavior should be tested against an environment that is sufficiently similar to production. Mocking a database can hide SQL syntax problems, constraints, transaction behavior, and indexing assumptions.

```
test('creates an order transactionally', async () => { const order = await repository.create({ customerId: 'customer-1', amount: 120 }); expect(order.amount).toBe(120); expect(order.customerId).toBe('customer-1'); });
```

For important transactional behavior, tests should verify both successful operations and rollback behavior.

10. Contract Testing

Contract testing verifies that independently developed components agree on the shape and meaning of their communication.

Figure 21.2 — Contract tests protect communication boundaries between independently evolving components.

11. End-to-End Testing

End-to-end tests exercise the system through a realistic user or client workflow. They provide high confidence but are slower and more expensive to maintain than lower-level tests.

LearnForge • Engineering Handbook • 2026 207

LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

Workflow Example

Priority Authentication Login and logout High Purchase Cart to payment confirmation Critical Content publishing Create, publish, view High Rare administration feature Low-frequency configuration Lower

12. Avoiding the E2E Test Trap

A large suite of browser-based tests can become slow, flaky, and expensive. When every small behavior requires a full application workflow, feedback becomes unnecessarily slow. Use E2E tests for critical paths rather than attempting to prove every business rule through a browser.

Architect's Rule

Test business rules at the lowest practical level and reserve E2E tests for important cross- system workflows.

13. Test Isolation

Tests should not depend on execution order. A test that passes only because a previous test created data is unreliable. Isolation can be achieved through transactions, temporary databases, controlled fixtures, unique identifiers, or explicit cleanup.

Problem Symptom Solution

Shared database state Tests pass individually but fail together Reset or isolate data Shared global state Order-dependent failures Reset state between tests Fixed timestamps Time-sensitive tests fail Inject a clock Random external dependency Flaky results Control the dependency

LearnForge • Engineering Handbook • 2026 208

LEARNFORGE SYSTEM ARCHITECTURE &

CLEAN CODE 14. Deterministic Testing A deterministic test produces the same result when the relevant inputs and environment are unchanged. Time, randomness, network state, and external services are common sources of nondeterminism.

```
interface Clock { now(): Date; }
class SystemClock implements Clock { now() { return new Date(); } }
class FakeClock implements Clock { constructor(private current: Date) {} now() { return this.current; } }
```

Injecting a clock allows time-dependent behavior to be tested without waiting for real time to pass.

15. Property-Based Thinking Traditional tests check selected examples. Property-based testing focuses on rules that should remain true across many generated inputs.

Property Example Non-negative total Total should never be below zero Idempotency Repeating the same operation should not duplicate state Ordering Sorted output remains ordered Round-trip Encode then decode preserves meaning

LearnForge • Engineering Handbook • 2026 209 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

16. Mutation Testing Mutation testing evaluates whether a test suite can detect intentional changes to production logic. A mutation might replace a greater-than comparison with a less-than comparison or remove a validation condition. If the mutation survives, the tests may not adequately protect that behavior. Mutation testing can therefore reveal weaknesses that simple coverage numbers do not show.

Production Tip Use mutation testing selectively on critical business modules rather than running it indiscriminately across every part of a large system.

17. Code Coverage Code coverage measures which portions of the code were executed by tests. It is useful as a signal, but it is not proof of correctness.

Coverage Type Question Line coverage Which lines executed? Branch coverage Which decision paths executed? Function coverage Which functions executed? Statement coverage Which statements executed?

Common Mistake Treating a high coverage percentage as proof that software is correct. Poorly designed tests can execute code without checking meaningful outcomes.

18. Testing Error Paths Production failures frequently occur in error-handling paths rather than ordinary success paths. Tests should deliberately exercise validation errors, timeouts, dependency failures, authorization failures, and malformed input.

LearnForge • Engineering Handbook • 2026 210 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

```
test('rejects unavailable payment provider', async () => {
  const payments = { charge: async () => { throw new Error('Provider unavailable'); } };
  const service = new CheckoutService(payments);
  await expect(service.checkout(100)).rejects.toThrow('Provider unavailable');
});
```

19. Security Testing Security testing should verify authentication, authorization, input validation, session behavior, secret handling, and protection against common attack classes.

Area Test Example Authentication Invalid credentials are rejected Authorization User cannot access another user's resource Input validation Malformed input is rejected safely Session management Expired sessions lose access Secrets Credentials are not exposed in responses or logs

20. Performance Testing Performance tests determine how a system behaves under expected and extreme

load. They should measure latency, throughput, resource consumption, and failure behavior. LearnForge • Engineering Handbook • 2026 211 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE Test Purpose Load test Expected traffic Stress test Behavior beyond normal capacity Spike test Sudden traffic increase Soak test Long-running stability

21. Testing Distributed Failure Distributed systems require tests that intentionally simulate dependency failure. A system that has only been tested under perfect conditions has not been tested for many of its most important production risks. Figure 21.3 — Failure injection validates resilience behavior before real production incidents expose the weakness.

22. CI Quality Gates Continuous integration should automatically execute important quality checks before changes are merged or deployed.

Gate Purpose	Typical Decision	Type	checking
Detect type errors	Block on failure	Linting	Detect code-quality issues
Block critical violations	Unit tests	Verify business logic	Block failures
Integration tests	Verify boundaries	Block critical failures	Security scan
Detect known vulnerabilities	Block severe findings	Build	Verify deployable artifact
Block failure	LearnForge • Engineering Handbook • 2026	212	LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

23. Test Execution Pipeline

```

quality:
  steps:
    - install
    - typecheck
    - lint
    - unit-tests
    - integration-tests
    - security-scan
  build
  deployment:
    depends_on:
      - quality
  condition:
    success
  
```

The exact CI syntax differs between platforms, but the architectural principle is stable: software should pass defined quality gates before it is allowed to progress toward production.

24. Flaky Tests A flaky test produces inconsistent results without a corresponding change in the tested code. Flakiness can be caused by timing, concurrency, shared state, network dependencies, random data, or insufficient cleanup.

Cause Mitigation

- Timing dependency Use deterministic clocks and explicit waits
- Shared state Isolate test data
- Randomness Control seeds
- External network Use controlled integration environments
- Concurrency race Synchronize explicitly and test race conditions

Architect's Rule Never normalize flaky tests as harmless. A test suite that cannot be trusted will eventually be ignored, reducing the value of the entire quality system.

LearnForge • Engineering Handbook • 2026 213 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

25. Test Data Management Test data should be realistic enough to expose important behavior while remaining deterministic, isolated, and safe. Production personal data should not be copied into test environments without appropriate controls.

- Use factories for repeatable test objects.
- Use unique identifiers when tests run concurrently.
- Keep fixtures understandable and minimal.
- Reset state between tests where necessary.
- Never expose real secrets through test data.

26. Quality Beyond Automated Tests Automated testing is powerful but does not replace architecture review, security review, usability evaluation, operational testing, and human judgment. A mature quality strategy combines automated verification with code review, design review, observability, incident learning, and controlled production validation.

27. Testing Strategy by System Risk

Risk	Recommended Testing
Financial correctness	Unit + integration
E2E	Security boundary
Authorization	+ security testing
External API	

compatibility Contract testing High traffic Load + stress testing Distributed failure Failure injection Complex business rules Focused unit and property tests LearnForge • Engineering Handbook • 2026 214 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE 28. Practical Testing Checklist • Are critical business rules covered by focused tests? • Are unit tests fast and deterministic? • Are important database boundaries integration-tested? • Are service contracts verified? • Are critical user workflows covered by E2E tests? • Are error and timeout paths tested? • Are security boundaries tested? • Are performance requirements measured? • Are distributed failure scenarios tested? • Are flaky tests investigated and removed? 29. Key Takeaways • Testing is an architectural capability, not merely a final verification step. • Fast unit tests provide rapid feedback on business logic. • Integration tests protect important architectural boundaries. • Contract tests reduce compatibility risk between independent services. • E2E tests should focus on critical workflows. • High coverage does not guarantee high-quality tests. • Error paths, security boundaries, and failure scenarios deserve deliberate testing. • CI quality gates prevent known defects from progressing through the delivery pipeline. • Flaky tests damage trust in the entire engineering system. LearnForge • Engineering Handbook • 2026 215 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE 30. Quick Review 1. Why is testability an architectural concern? 2. What is the purpose of the testing pyramid? 3. When should an integration test be preferred over a unit test? 4. What is the difference between a mock, stub, fake, and spy? 5. Why are contract tests valuable in microservice architectures? 6. Why should E2E tests be limited to important workflows? 7. What makes a test deterministic? 8. Why is code coverage not proof of correctness? 9. Why should distributed failure scenarios be tested intentionally? 10. What makes a CI quality gate valuable? 31. Architect's Checklist • Is the testing strategy aligned with business and technical risk? • Can important domain logic be tested without infrastructure? • Are architectural boundaries protected by integration or contract tests? • Are critical workflows represented by reliable E2E tests? • Are dependencies injectable or otherwise controllable during testing? • Are retries, timeouts, and failure paths explicitly tested? • Are security and authorization rules verified automatically? • Are performance expectations measurable and repeatable? • Are CI quality gates enforced before deployment? • Can the team trust the test suite enough to use it during rapid change? LearnForge • Engineering Handbook • 2026 216 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE Chapter 22 CI/CD & Release Architecture Continuous integration and continuous delivery are not merely automation practices. They are architectural capabilities that reduce deployment risk, shorten feedback loops, and make software delivery repeatable, observable, and reversible. What You Will Learn • How CI/CD pipelines transform source code into safely releasable software. • How to design reliable build, test, security, and deployment stages. • Why artifact immutability and reproducible builds matter. • How blue-green, rolling, and canary releases differ. • How feature flags, rollback strategies, and progressive delivery reduce risk. • How to

design pipelines that are secure, observable, and maintainable.

1. What CI/CD Actually Means Continuous Integration (CI) is the practice of frequently integrating code changes into a shared codebase while automatically validating those changes. The purpose is to discover integration problems early, when they are cheaper to fix. Continuous Delivery means keeping software in a releasable state through automated build, test, validation, and packaging processes. Continuous Deployment goes one step further by automatically releasing validated changes to production.
2. The Delivery Pipeline A mature pipeline normally moves software through a sequence of increasingly expensive validation stages. Cheap checks should run early, while expensive integration and deployment tests should run after basic failures have been eliminated.
3. Fast Feedback A pipeline is valuable only if developers receive useful feedback quickly. A ten-minute pipeline that catches a failure immediately can be more useful than a one-hour pipeline that performs unnecessary work before reporting a basic syntax or unit-test failure.
4. Build Once, Deploy Many Times A strong release architecture separates building software from deploying software. The application should ideally be built once and the resulting artifact promoted through environments. Rebuilding separately for development, staging, and production can produce different binaries even when the source commit is identical. This makes debugging and release verification harder.
5. Reproducible Builds A reproducible build is one in which the same source and build inputs produce

LearnForge • Engineering Handbook • 2026 217 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE Architect's Rule A deployment pipeline should make the safe path easier than the unsafe path. If developers must manually perform many fragile steps to release software, the architecture is carrying unnecessary operational risk.

Figure 22.1 — A CI/CD pipeline progressively validates a change before and after deployment.

Architect's Rule Prefer immutable artifacts. Build a specific commit into a versioned artifact, validate that artifact, and promote the same artifact between environments.

```
interface ReleaseArtifact {
  version: string;
  commitSha: string;
  imageDigest: string;
}
const artifact: ReleaseArtifact = {
  version: "2.8.0",
  commitSha: "8f31c2a",
  imageDigest: "sha256:abc123..."
};
console.log(`Promoting ${artifact.version}`);
```

the same expected artifact. Reproducibility improves trust because engineers can determine exactly what was built and deployed. Dependencies should be explicitly versioned. Build environments should be controlled. Hidden dependencies on a developer's local machine should be avoided. LearnForge • Engineering Handbook • 2026 219 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE Risk Weak Practice Better Practice Dependency drift Unbounded versions Lock files and controlled updates Environment drift Different build machines Standardized build environment Unknown artifact Manual packaging Automated immutable artifact Untraceable release Manual file copying Commit and artifact metadata

6. Quality Gates

A quality gate is an explicit condition that must be satisfied before a pipeline can continue. Quality gates prevent known problems from moving into later environments.

```
interface QualityGate { testsPassed: boolean; coverageAccepted: boolean; securityScanPassed: boolean; artifactCreated: boolean; } function canRelease(gate: QualityGate): boolean { return ( gate.testsPassed && gate.coverageAccepted && gate.securityScanPassed && gate.artifactCreated ); }
```

Quality gates should measure meaningful risk rather than arbitrary numbers. For example, a coverage threshold may be useful, but coverage alone cannot prove that critical business behavior is correct.

7. Automated Testing in CI

CI should run tests that provide confidence appropriate to the change. Different test levels serve different purposes. LearnForge • Engineering Handbook • 2026 220 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE Test Type Speed Primary Value Unit Fast Local behavior Integration Moderate Component interaction Contract Moderate API compatibility End-to-end Slow Critical user workflows Common Mistake Relying entirely on end-to-end tests. A large end-to-end suite can become slow, flaky, and difficult to diagnose. A balanced test pyramid usually provides faster and more precise feedback.

8. Security in the Pipeline

Security should not be a final manual review performed immediately before production. CI/CD provides repeated opportunities to identify vulnerable dependencies, leaked secrets, insecure configuration, and dangerous code patterns.

- Dependency scanning: identify vulnerable packages.
- Secret scanning: detect credentials committed to source.
- Static analysis: identify suspicious code patterns.
- Container scanning: inspect packaged runtime images.
- Configuration validation: detect unsafe infrastructure settings.

Architect's Rule Never place production credentials directly in source code or repository configuration. Secrets should be supplied through an appropriate secret management mechanism at runtime or deployment time.

9. Pipeline Security

The CI/CD system itself is part of the production security boundary. A compromised pipeline can modify application artifacts, access credentials, or deploy malicious code. LearnForge • Engineering Handbook • 2026 221 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE Threat Example Control Short-lived credentials and secret Credential theft Exposed deployment token management Pipeline Unauthorized workflow change Code review and branch protection modification Artifact tampering Modified image Signing and immutable registries Build job has

production administrator Excessive privilege Least privilege access

10. Environment Promotion Many organizations use multiple environments such as development, staging, and production. Each environment should have a clear purpose.

Environment	Purpose	Typical Users
Development	Rapid feature development	Developers
Test	Automated integration validation	CI system
Staging	Production-like verification	Engineering and QA
Production	Real user traffic	Customers

The important principle is not the number of environments. The important principle is that each environment reduces a specific type of uncertainty. Adding environments without adding meaningful validation creates maintenance cost without equivalent value.

11. Database Changes During Deployment Database migrations are one of the most important release risks. Application code and database schemas do not always change at exactly the same moment. LearnForge • Engineering Handbook • 2026

222 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE A safer approach is to use backward-compatible migration patterns. A new schema can be introduced before application code begins depending on it. After the application is migrated, obsolete structures can be removed later.

Figure 22.2 — Expand-and-contract migrations reduce incompatibility between old and new application versions.

12. Rolling Releases A rolling release gradually replaces old application instances with new instances. During the transition, both versions may temporarily exist.

Advantage	Risk	Uses
existing infrastructure	Old and new versions coexist	Gradual replacement
Rollback may require replacing instances again	No full duplicate environment required	Backward compatibility becomes important

13. Blue-Green Deployment Blue-green deployment maintains two production-like environments. One serves traffic while the other receives the new release. Traffic can then be switched between environments.

Figure 22.3 — Blue-green deployment separates the active environment from the candidate release and enables controlled traffic switching.

14. Canary Releases A canary release exposes a new version to a small percentage of traffic before expanding the release. The production environment becomes part of the validation process, but exposure is deliberately limited.

LearnForge • Engineering Handbook • 2026

223 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

Stage	Traffic	Decision
Canary	1-5%	Check errors and latency
Expansion	10-25%	Compare behavior
Major rollout	50%	Validate stability
Complete	100%	Finish release

The exact percentages depend on system risk and traffic volume. A canary strategy should define measurable promotion and rollback criteria before the release begins.

15. Feature Flags Feature flags separate deployment from feature activation. Code can be deployed while a feature remains disabled. The feature can then be activated for selected users or environments.

```
interface FeatureContext {
  userId: string;
  environment: string;
}
function isEnabled( feature: string, context: FeatureContext ): boolean {
  if (feature === "new-checkout") {
    return context.environment === "staging";
  }
  return false;
}
```

Production Tip Feature flags create operational state. Every long-lived flag should have an owner, purpose, expiration plan, and removal task. Forgotten flags increase code complexity.

LearnForge • Engineering Handbook • 2026 224 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE 16. Rollback Strategy A release strategy is incomplete without a rollback strategy. Before deploying, the team should know how to stop or reverse the change. Rollback Method Useful When Important Constraint Previous artifact Application regression Schema compatibility Traffic switch Blue-green deployment Both environments must work Feature disable Flag-controlled feature Feature must be isolated Canary stop Progressive delivery Monitoring must be fast Common Mistake Treating database rollback as identical to application rollback. Database changes may be irreversible or destructive. Prefer forward-compatible migration strategies and tested recovery procedures. 17. Deployment Verification Deployment success should not be defined only as "the process completed." The application must also behave correctly after deployment. • Health checks succeed. • Error rates remain within acceptable limits. • Latency remains within service objectives. • Critical business transactions succeed. • Logs and metrics remain healthy. • No unexpected resource exhaustion occurs. 18. Health Checks Health checks allow infrastructure and deployment systems to determine whether an application instance can receive traffic. LearnForge • Engineering Handbook • 2026 225 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

```
app.get("/health/live", (_req, res) => { res.status(200).json({ status: "alive" }); });
app.get("/health/ready", async (_req, res) => { const databaseReady = await checkDatabase(); if (!databaseReady) { return res.status(503).json({ status: "not-ready" }); } return res.status(200).json({ status: "ready" }); });
```

Liveness and readiness checks serve different purposes. A process can be alive while temporarily unable to serve traffic safely. 19. Deployment Observability Release systems should be observable like production applications. Engineers need to know which version is running and whether a deployment changed system behavior. Signal Useful Question Deployment event When did the release occur? Version metadata Which build is running? Error rate Did failures increase? Latency Did performance regress? Business metrics Did user behavior change unexpectedly? LearnForge • Engineering Handbook • 2026 226 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE 20. Deployment Frequency and Delivery Metrics Delivery performance should be measured using outcomes rather than vanity metrics. Useful indicators include deployment frequency, lead time for changes, change failure rate, and time to restore service. Metric Meaning Why It Matters Deployment How often changes are released Indicates delivery flow frequency Measures feedback and delivery Lead time Time from change to production speed Percentage of releases causing incidents or Change failure rate Measures release quality rollback Recovery time Time required to restore normal service Measures resilience 21. Pipeline as Code Pipeline definitions should generally be version-controlled alongside application code or within a controlled infrastructure repository. This makes pipeline changes reviewable and reproducible. LearnForge • Engineering Handbook • 2026 227 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE name: CI on: push: branches: [main] pull_request:

jobs: validate: runs-on: ubuntu-latest steps: - uses: checkout - run: npm ci - run: npm run lint - run: npm test - run: npm run build The exact syntax depends on the CI platform. The architectural principle is more important than the specific tool: the delivery process should be automated, versioned, reviewable, and repeatable.

22. Dependency and Artifact Caching

Caching can dramatically reduce pipeline duration, but incorrect cache invalidation can produce confusing failures.

Cache Benefit	Risk
Package dependencies	Faster installs
Stale dependency data	Build outputs
Faster compilation	Incorrect reuse
Docker layers	Faster image builds
Unexpected stale layers	Cache keys should include the inputs that determine the cached result.

When inputs change, the cache should naturally become invalid. LearnForge • Engineering Handbook • 2026 228 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

23. Release Approvals

Not every production deployment requires manual approval. The appropriate level of control depends on risk.

Change Type	Suggested Control
Low-risk routine change	Automated validation and deployment
High-risk infrastructure change	Additional review or approval
Emergency security fix	Expedited controlled process
Destructive migration	Explicit review and recovery plan

Architect's Rule Do not add manual approvals simply because automation feels dangerous. Instead, automate evidence collection and place human approval only where human judgment genuinely adds value.

24. Release Management and Ownership

A release pipeline should have clear ownership. Teams should know who owns the service, who receives deployment alerts, who can stop a rollout, and who is responsible for recovery.

- Service owner is documented.
- Deployment permissions follow least privilege.
- Rollback procedures are documented.
- Alerts have responsible teams.
- Production changes are traceable to source commits.

25. Disaster Recovery for Delivery Systems

The CI/CD system itself can become a dependency. If the pipeline is unavailable during an incident, teams may be unable to deploy fixes. LearnForge • Engineering Handbook • 2026 229 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

Important pipeline configuration, infrastructure definitions, and deployment credentials should therefore have appropriate backup and recovery strategies. The recovery process should be tested rather than assumed.

26. Release Anti-Patterns

Anti-Pattern	Problem	Better Approach
Manual production copying	Unrepeatable and error-prone	Automated artifact deployment
Build separately per environment	Artifact drift	Build once and promote
No rollback plan	Long incident recovery	Tested rollback strategy
All tests only after deployment	Late feedback	Shift appropriate validation left
Permanent feature flags	Growing code complexity	Flag ownership and cleanup

27. A Practical Release Workflow

A practical production workflow can combine the principles discussed in this chapter into a controlled sequence.

1. Developer creates a small change.
2. Pull request triggers automated validation.
3. Linting and unit tests run first.
4. Build creates a versioned immutable artifact.
5. Security and integration checks validate the artifact.
6. The artifact is deployed to staging.
7. Smoke tests verify critical behavior.
8. The same artifact is promoted toward production.
9. A canary or controlled rollout begins.
10. Monitoring determines

whether the release continues. 11. Failed releases are stopped or rolled back according to predefined rules. LearnForge • Engineering Handbook • 2026 230 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE 28. Key Takeaways • CI/CD is an architectural capability, not simply a collection of scripts. • Fast feedback reduces the cost of discovering integration problems. • Build once and promote immutable artifacts whenever practical. • Quality gates should measure meaningful risk. • Security controls belong throughout the delivery pipeline. • Database changes require backward-compatible deployment strategies. • Canary, blue-green, and rolling releases provide different risk profiles. • Every release needs a tested rollback or mitigation strategy. • Deployment success must include post-release behavioral validation. • Delivery pipelines themselves require security, ownership, and recovery planning. 29. Practical CI/CD Checklist • Are changes automatically built after integration? • Are unit and integration tests executed automatically? • Are security checks included in the pipeline? • Is the production artifact immutable and traceable? • Can the same artifact be promoted between environments? • Are database migrations backward compatible? • Is there a defined deployment strategy? • Is there a tested rollback or mitigation procedure? • Are production deployments observable? • Are deployment permissions protected by least privilege? • Are feature flags owned and eventually removed? LearnForge • Engineering Handbook • 2026 231 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE • Can the CI/CD platform itself be recovered during an incident? 30. Quick Review 1. What is the difference between continuous integration, delivery, and deployment? 2. Why is building once and deploying the same artifact valuable? 3. What makes a build reproducible? 4. Why should cheap tests usually run before expensive tests? 5. What is the purpose of a quality gate? 6. How does a canary release reduce deployment risk? 7. What is the difference between rolling and blue-green deployment? 8. Why are feature flags useful? 9. Why can database migrations make rollback difficult? 10. What should deployment observability measure? 31. Architect's Checklist • Is the delivery pipeline treated as production infrastructure? • Are all pipeline changes version-controlled and reviewed? • Are build inputs deterministic and dependencies controlled? • Are artifacts immutable, versioned, and traceable to commits? • Are deployment permissions minimized? • Are secrets kept outside source code? • Are release promotion criteria measurable? • Are canary or progressive rollout mechanisms available for high-risk services? • Can the organization rapidly stop a harmful release? • LearnForge • Engineering Handbook • 2026 232 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE Has the rollback procedure been exercised rather than merely documented? • Are schema migrations compatible with mixed application versions? • Is there an owner for every production service and pipeline? • Can the delivery platform recover during a major infrastructure incident? LearnForge • Engineering Handbook • 2026 233 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE Chapter 23 Architecture Governance & Decision Making Good architecture governance creates clarity without unnecessary

bureaucracy. It makes important technical decisions explicit, reviewable, measurable, and aligned with business goals while allowing teams to evolve the system safely over time. What You Will Learn • Why architecture governance matters in growing engineering organizations. • How Architecture Decision Records make important decisions explicit. • How to evaluate architectural trade-offs systematically. • How architecture reviews can improve systems without blocking delivery. • How to manage technical debt intentionally. • How standards, ownership, and documentation support architectural consistency. • How to design governance that enables teams rather than creating bureaucracy.

1. What Is Architecture Governance?

Architecture governance is the set of principles, decision processes, standards, and review mechanisms used to keep a software system aligned with its technical and business objectives. Governance does not mean that one architect controls every implementation detail. Effective governance establishes boundaries within which teams can make decisions independently while ensuring that decisions affecting the whole system remain visible and consistent. Architect's Rule Govern decisions that create long-term consequences, not every line of implementation code. Teams need freedom for local decisions and alignment for system-wide decisions. LearnForge • Engineering Handbook • 2026 234 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

2. Why Governance Becomes Important as Systems Grow

A small application can often rely on direct communication between developers. As the organization and system grow, the number of services, teams, databases, APIs, infrastructure components, and architectural decisions increases.

System Stage	Typical Challenge	Governance Need
Small	Limited coordination	Lightweight conventions
Growing	Multiple teams make overlapping decisions	Shared principles and documentation
Large	Platform Many dependencies and services	Formal decision records and reviews
Enterprise	High architectural and regulatory complexity	Clear ownership, standards, controls, and system complexity governance

3. Governance Is Not Bureaucracy

Poor governance creates meetings, approval queues, documents that nobody reads, and rules that slow engineering teams without improving system quality. Good governance does the opposite: it reduces repeated debates, makes constraints visible, and helps teams make decisions faster.

Figure 23.1 — Effective governance connects team autonomy with system-wide architectural consistency.

4. Architecture Principles

Architecture principles provide stable guidance for recurring decisions. They should be concise enough to remember and specific enough to influence design choices. LearnForge • Engineering Handbook • 2026 235 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

Principle	Meaning	Example
Each capability has clear Services	avoid direct writes to another	Own your data responsibility for its state
service's database	Prefer explicit	Dependencies should be visible
Use documented APIs or events	boundaries Automate	Reduce manual operational risk
Automated deployments and testing	repeatable work	Design for failure
Failures are expected	Timeouts, retries, fallbacks, monitoring	

5. Architecture Decision Records

An Architecture Decision Record, commonly called an

ADR, captures an important architectural decision together with its context, alternatives, and consequences. The value of an ADR is not merely documentation. It preserves the reasoning behind a decision so future engineers can understand why the system was designed that way. # ADR-023: Use PostgreSQL for Transactional Order Data ## Status Accepted ## Context Order data requires transactions, constraints, and reliable consistency. ## Decision Use PostgreSQL as the primary transactional database. ## Alternatives - Document database - Distributed key-value store - Relational database ## Consequences We gain strong transactional guarantees and mature SQL tooling. Horizontal scaling requires deliberate partitioning and read-scaling strategies as traffic increases. LearnForge • Engineering Handbook • 2026 236 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

6. The Structure of a Good ADR Section

Purpose Title Clearly identify the decision
Status Show whether the decision is proposed, accepted, superseded, or rejected
Context Explain the problem and constraints
Decision State what will be done
Alternatives Record meaningful options considered
Consequences Describe benefits, costs, and risks
Production Tip Write ADRs for decisions that are difficult or expensive to reverse. Do not create an ADR for every ordinary implementation choice.

7. Decision Reversibility

Not every technical decision deserves the same level of governance. A useful classification is based on reversibility.

Decision Type	Example	Governance Level	Easy to reverse	Internal helper library	Team-level	Moderately difficult	Framework selection	Team review	Expensive to reverse	Database technology	Architecture review	+ ADR	Very difficult to reverse	Public API contract	Formal review	+ ADR	+ compatibility plan
Architect's Rule	The harder a decision is to reverse, the more evidence and architectural review it deserves.																

LearnForge • Engineering Handbook • 2026 237 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

8. Trade-off Analysis

Architecture decisions rarely have one universally correct answer. A design that improves scalability may increase operational complexity. A design that improves consistency may increase latency.

Option	Benefit	Cost	Risk
Synchronous workflow	Immediate result	Tighter coupling	Cascading failure
Asynchronous	Loose coupling	Eventual consistency	Complex debugging workflow
Lower operational complexity	Migration	Managed database	Vendor dependency burden
Self-managed	Operational	Maintenance	Greater control
database responsibility	failures		

9. A Practical Decision Function

Teams can make architectural reasoning more explicit by evaluating candidate solutions against consistent criteria.

```
interface ArchitectureOption {
    name: string;
    reliability: number;
    scalability: number;
    complexity: number;
    cost: number;
    reversibility: number;
}

function score(option: ArchitectureOption): number {
    return (
        option.reliability * 0.30 +
        option.scalability * 0.25 +
        option.reversibility * 0.20 -
        option.complexity * 0.15 -
        option.cost * 0.10
    );
}
```

LearnForge • Engineering Handbook • 2026 238 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

The purpose of such scoring is not mathematical precision. It forces the team to make assumptions visible and discuss why one option is preferred.

10. Architecture Reviews

An architecture review evaluates whether a

proposed change is consistent with system principles, constraints, operational requirements, and long-term direction. Reviews should focus on risk and system impact rather than style preferences. A review should help the team discover important problems before those problems become expensive production incidents.

11. Lightweight Review Process Figure 23.2 — A lightweight architecture review turns a problem into an explicit decision with documented consequences.

12. Architecture Ownership Every important architectural area should have clear ownership. Ownership does not mean one person performs all work. It means somebody is accountable for maintaining the decision, standards, and operational understanding.

Area	Example Owner	Responsibility
API standards	Platform team	Contracts, versioning, conventions
Database platform	Data/platform team	Reliability and operational standards
Security architecture	Security team	Threat models and security controls
Service domain	Product team	Business behavior and domain decisions

LearnForge • Engineering Handbook • 2026 239 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

13. Technical Debt Technical debt is the future cost created by technical shortcuts, outdated designs, missing automation, or deliberate compromises. Not all technical debt is bad. Sometimes taking a shortcut is rational when the business needs speed and the consequences are understood. Architect's Rule Technical debt becomes dangerous when it is invisible, unmanaged, or more expensive to repay than the organization can afford.

14. Managing Technical Debt

Debt Type	Example	Action	Known	Acceptable
Temporary implementation shortcut	Document and monitor	Operational	Manual deployment process	Automate gradually
Architectural	Tight service coupling	Refactor	boundary	Security
Outdated dependency	Prioritize according to risk	Structural	Unmaintainable module	Refactor or replace incrementally

15. Governance Metrics Governance should be evaluated by outcomes rather than the number of meetings or documents produced.

Metric	What It Indicates
Architecture-related incidents	Whether important design risks are being controlled
Decision lead time	Whether governance blocks delivery
ADR reuse	Whether previous decisions provide useful guidance
Technical debt trend	Whether structural problems are increasing or decreasing
Change failure rate	Whether architectural and delivery practices support safe change

16. Standards and Policies Standards are useful when they solve recurring problems. Examples include API naming conventions, authentication requirements, logging formats, deployment requirements, observability standards, and dependency-management policies. A standard should have a clear reason for existing. Teams should understand what problem the rule solves and when exceptions are appropriate.

17. Exceptions to Standards Rigid standards can become harmful when unusual business or technical requirements require a different solution. A healthy governance model supports controlled exceptions.

Step	Question	Identify Why	Assess What	Document Why	Review When
Identify	Why does the standard not fit?	Why does the standard not fit?	What risks does the exception introduce?	Why was the exception approved?	When should the exception be reconsidered?

LearnForge • Engineering Handbook • 2026 241 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE 18. Documentation as an Architectural Tool Architecture documentation should help engineers understand how the system works and why important decisions were made. Useful documentation describes boundaries, ownership, dependencies, data flows, deployment assumptions, failure behavior, and important decisions. Documentation that only describes implementation details quickly becomes outdated. interface ServiceOwnership { service: string; team: string; dataOwner: string; operationalOwner: string; criticalDependencies: string[]; } const ownership: ServiceOwnership = { service: "payment-service", team: "commerce", dataOwner: "commerce", operationalOwner: "platform", criticalDependencies: ["payment-provider", "identity-service"] }; 19. Architecture Fitness Functions A fitness function is an automated check that verifies whether an important architectural property remains true as the system evolves. LearnForge • Engineering Handbook • 2026 242 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE function assertNoForbiddenDependency(caller: string, dependency: string) { const forbidden = dependency === "database-internal"; if (forbidden) { throw new Error(`\${caller} contains a forbidden architectural dependency`); } } Fitness functions turn architectural expectations into executable checks. This is especially valuable for large systems where architecture can gradually deteriorate through many individually reasonable changes. 20. Preventing Architecture Erosion Architecture erosion occurs when repeated small changes gradually weaken the original structure. Examples include bypassing APIs, introducing circular dependencies, sharing internal database tables, and adding undocumented cross-service coupling. Common Mistake Assuming that architecture remains healthy because the original design was good. Architecture requires continuous verification as the system evolves. 21. Governance and Team Autonomy A mature organization separates local implementation freedom from global architectural constraints. LearnForge • Engineering Handbook • 2026 243 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE Decision Typical Owner Variable naming Developer Internal helper structure Team Service API contract Owning team + affected consumers Company-wide authentication model Security/platform governance Major data architecture Cross-team architecture review 22. Governance Workflow Figure 23.3 — Architecture governance is continuous: propose, evaluate, decide, monitor, and evolve. 23. Governance Anti-Patterns Anti-Pattern Problem Better Approach Teams become slow and Central approval for everything Delegate local decisions dependent Use evidence and explicit trade- Architecture by opinion Decisions become political offs Documentation after incidents Record important decisions when Reasoning is lost only made Give exceptions owners and review Permanent exceptions Standards gradually disappear dates Architecture becomes Enable teams with principles and Architect as bottleneck centralized tools LearnForge • Engineering Handbook • 2026 244 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE 24. Practical Architecture Governance Checklist •

Are important architectural decisions explicitly documented? • Does each major system component have a clear owner? • Are architecture principles short and understandable? • Are difficult-to-reverse decisions reviewed more carefully? • Are meaningful alternatives recorded in ADRs? • Are technical-debt risks visible? • Can teams make local implementation decisions without unnecessary approval? • Are architecture standards automated where practical? • Are exceptions documented and periodically reviewed? • Are architecture outcomes measured rather than meeting counts?

25. Key Takeaways • Architecture governance should create alignment without unnecessary bureaucracy. • ADR documents preserve the context and reasoning behind important decisions. • Hard-to-reverse decisions deserve stronger review and evidence. • Technical debt should be visible, prioritized, and managed intentionally. • Architecture principles provide reusable guidance for teams. • Fitness functions can continuously verify important architectural properties. • Governance should protect system-wide concerns while preserving team autonomy. • Architecture is continuously evaluated as systems, teams, and business requirements evolve.

LearnForge • Engineering Handbook • 2026 245 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

26. Quick Review

1. What is architecture governance?
2. How is effective governance different from bureaucracy?
3. What is an Architecture Decision Record?
4. Why should difficult-to-reverse decisions receive stronger review?
5. What information should a useful ADR contain?
6. How should technical debt be managed?
7. What are architecture fitness functions?
8. Why are exceptions to standards sometimes necessary?
9. How can governance preserve team autonomy?
10. Why must architecture be continuously monitored?

27. Architect's Checklist • Is the architectural problem clearly defined? • Are business and technical constraints documented? • Have realistic alternatives been considered? • Are trade-offs explicit? • Is the decision reversible or difficult to reverse? • Is ownership clearly assigned? • Is the decision recorded in an ADR when appropriate? • Can important architectural rules be automatically verified? • Are technical-debt consequences understood? • Is there a plan to review the decision as the system evolves?

LearnForge • Engineering Handbook • 2026 246 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

Chapter 24 Evolutionary Architecture & System Modernization

Mature software systems must evolve continuously. Successful modernization reduces risk through incremental change, explicit migration boundaries, measurable outcomes, compatibility strategies, and disciplined removal of obsolete architecture.

What You Will Learn • Why legacy systems should be modernized incrementally. • How to choose between refactoring, rewriting, and replacement. • How the Strangler Fig pattern enables gradual modernization. • How to migrate databases and APIs safely. • How feature flags and branch-by-abstraction reduce migration risk. • How to measure modernization progress. • How to prevent migration projects from creating new technical debt.

1. Why Software Architecture Must Evolve

Software architecture is created under a particular set of business requirements, technical constraints, team structures, and infrastructure conditions.

Those conditions change. A system that was appropriate five years ago may become expensive to operate, difficult to change, or unable to satisfy new scalability and security requirements. Architecture therefore should be treated as an evolving technical asset rather than a permanent structure. Architect's Rule Do not modernize because a technology is old. Modernize when the current architecture creates measurable business, operational, security, or engineering constraints.

2. What Makes a System Legacy? A legacy system is not necessarily an old system. A relatively new application can become legacy when its architecture prevents safe and economical change.

LearnForge • Engineering Handbook • 2026 247 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

Characteristic Typical Consequence High coupling Small changes affect many components Poor automated testing Refactoring becomes risky Unsupported dependencies Security and maintenance risk Manual deployment Slow and error-prone releases Undocumented behavior Engineers fear changing the system Obsolete infrastructure Increasing operational cost

3. Modernization Is a Business Decision Modernization consumes engineering capacity. Therefore the justification should be connected to measurable outcomes such as reduced operational cost, faster delivery, improved reliability, stronger security, or increased scalability.

Driver Possible Measurement Slow delivery Lead time for changes Reliability problems Incident frequency and recovery time High infrastructure cost Cost per transaction or user Security risk Critical vulnerabilities and remediation time Scaling limitations Maximum sustainable throughput

4. Refactoring vs Rewriting Refactoring changes the internal structure of a system while preserving its observable behavior. Rewriting replaces substantial portions of the existing implementation with a new system.

LearnForge • Engineering Handbook • 2026 248 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

Approach Advantage Risk Refactoring Small incremental changes Old constraints may remain Rewriting Opportunity for major redesign Large unknown migration risk Replacement Can remove obsolete technology Integration and migration complexity Common Mistake Assuming a rewrite is faster simply because the existing system is difficult to understand. A rewrite often recreates years of hidden business rules that were embedded in the original implementation.

5. The Strangler Fig Pattern The Strangler Fig pattern replaces an old system gradually. New capabilities are implemented around the legacy system until enough functionality has moved that the original component can eventually be retired.

Figure 24.1 — The Strangler Fig pattern allows a legacy system to be replaced incrementally rather than through one high-risk cutover.

6. Establish a Migration Boundary A modernization effort needs a clear boundary. The boundary may be an API, event stream, module, database table, or business capability. A good boundary reduces the amount of code that must change simultaneously and creates a measurable migration unit.

7. Branch by Abstraction Branch by abstraction introduces an abstraction layer between consumers and the implementation being replaced. The old and new implementations can then coexist while traffic or behavior is gradually migrated.

LearnForge •

Engineering Handbook • 2026 249 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE interface PricingService { calculatePrice(productId: string): Promise; } class LegacyPricing implements PricingService { async calculatePrice(productId: string) { return legacyPricingSystem.getPrice(productId); } } class NewPricing implements PricingService { async calculatePrice(productId: string) { return modernPricingEngine.calculate(productId); } } The abstraction prevents consumers from becoming directly dependent on either implementation.

8. Feature Flags

Feature flags allow teams to change behavior without deploying a completely new release for every experiment or migration step.

```
function selectPricingEngine(userId: string) { if (featureFlags.enabled("new-pricing", userId)) { return new NewPricing(); } return new LegacyPricing(); }
```

Production Tip Feature flags are temporary architecture tools. Every migration flag should have an owner, purpose, monitoring plan, and removal date.

9. Database Modernization

Database migration is often one of the most difficult parts of modernization because application behavior and persistent state are tightly connected.

LearnForge • Engineering Handbook • 2026 250 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

Strategy Use Case Risk

- Big-bang migration
- Small datasets and controlled downtime
- High cutover risk
- Dual write
- Gradual transition
- Consistency problems
- Change data capture
- Synchronizing systems during migration
- Operational complexity
- Backfill + validation
- Large existing datasets
- Long migration window

10. Expand and Contract

The expand-and-contract pattern allows database schemas to evolve without requiring every application version to change simultaneously.

Phase Action

- Expand** Add the new schema without removing the old structure
- Migrate** Deploy code that can work with both representations
- Backfill** Populate the new representation
- Switch** Move reads and writes to the new structure
- Contract** Remove the obsolete schema

11. API Evolution

APIs become architectural contracts. Once external or internal consumers depend on them, changing them carelessly can create widespread failures.

LearnForge • Engineering Handbook • 2026 251 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

```
interface UserResponseV1 { id: string; name: string; } interface UserResponseV2 { id: string; displayName: string; locale: string; }
```

Safe API evolution often uses additive changes, compatibility periods, versioning, consumer testing, and explicit deprecation schedules.

12. Compatibility

Is a Migration Tool

Compatibility allows old and new components to coexist. This is particularly important when teams cannot deploy every component simultaneously.

Compatibility Technique Purpose

- Additive** API fields Allow old consumers to continue working
- Versioned** contracts Support multiple consumer generations
- Adapters** Translate old interfaces into new ones
- Dual-read** logic Support migration between data representations
- Feature flags** Control rollout behavior

13. Migration Observability

A migration without observability is difficult to control. Engineers need to know whether the new implementation behaves correctly and whether users are experiencing regressions.

LearnForge • Engineering Handbook • 2026 252

LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

Signal What to Monitor
 Errors New vs legacy error rates Latency p50, p95, and p99 behavior Business outcomes Successful transactions and conversion Data consistency Differences between old and new results

14. Shadow Traffic Shadow traffic sends a copy of real requests to a new implementation without using its response for the user's actual operation.

```

  async function handleRequest(request: Request) {
    const result = await legacyService.execute(request);
    void newService.execute(request).catch(error => {
      logger.error({ event: "shadow-request-failed", error });
    });
    return result;
  }
  
```

Shadow traffic can reveal performance and behavioral differences before the new implementation receives production authority.

15. Incremental Cutover Instead of switching every user at once, traffic can be moved gradually. A typical rollout may begin with internal users, then a small percentage, then progressively larger groups.

Figure 24.2 — Incremental rollout limits the blast radius of modernization changes.

LearnForge • Engineering Handbook • 2026 253

LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

16. Rollback Planning Every significant migration should define what happens when the new implementation behaves incorrectly.

Architect's Rule A migration is not production-ready until the team knows how to stop, reverse, or safely continue it when reality differs from the migration plan.

17. Migration Risk Matrix

Risk	Probability	Impact	Mitigation
Data inconsistency	Medium	High	Validation and reconciliation
Performance regression	Medium	High	Load testing and gradual rollout
Hidden business rule	High	High	Characterization tests and domain review
Rollback failure	Low	Critical	Test rollback before cutover

18. Characterization Tests Before changing legacy behavior, teams should capture what the existing system actually does. These tests are called characterization tests.

```

  test("legacy pricing behavior is preserved", async () => {
    const result = await legacyPricing.calculatePrice("product-123");
    expect(result).toEqual(1299);
  });
  
```

The purpose is not necessarily to prove that the old behavior is ideal. It creates a safety net that exposes accidental behavior changes during modernization.

LearnForge • Engineering Handbook • 2026 254

LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

19. Modernizing a Monolith A monolith does not need to be replaced with microservices simply because it has grown. Many monoliths can be improved through modularization, better testing, clearer ownership, and selective extraction.

Problem First Modernization Step

Problem	First Modernization Step
Large tangled modules	Create explicit module boundaries
Shared database access	Introduce ownership around data access
Slow tests	Create focused unit and integration tests
Manual deployment	Automate build and deployment
Unclear dependencies	Measure and document dependency graph

20. Cloud Migration Moving an application to cloud infrastructure is not automatically modernization. Simply moving virtual machines without improving operational architecture may preserve the original problems. Cloud modernization may involve managed databases, automated infrastructure, elastic scaling, observability, security automation, and improved deployment processes.

21. Avoiding Migration Big Bangs Large migration projects become risky when too many changes are combined into

one release. Splitting the migration into independently verifiable steps reduces uncertainty. Common Mistake Combining database migration, service extraction, infrastructure replacement, API redesign, and UI changes into one cutover. If the result fails, the team cannot easily determine which change caused the problem.

LearnForge • Engineering Handbook • 2026 255 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

22. Measuring Modernization Progress Measure Useful Question Legacy dependency count Are obsolete components decreasing? Migration coverage How much functionality has moved? Incident rate Is modernization improving reliability? Deployment frequency Can teams deliver changes more easily? Change lead time Has engineering speed improved?

23. Removing the Old System A migration is incomplete when the new system works but the old system continues running indefinitely. The old path creates cost, confusion, and maintenance obligations. Retirement should therefore be treated as an explicit migration phase: disable traffic, confirm consumers are gone, archive required data, remove infrastructure, and delete obsolete code.

24. Practical Modernization Checklist • Is there a measurable reason for modernization? • Is the migration divided into small verifiable steps? • Are important legacy behaviors covered by tests? • Is there a clear migration boundary? • Can old and new implementations coexist safely? • Are APIs and database schemas backward compatible during transition? • Is migration telemetry available?

LearnForge • Engineering Handbook • 2026 256 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

Is rollout gradual where appropriate? • Has rollback been tested? • Is there a defined retirement plan for obsolete components?

25. Key Takeaways • Modernization should solve measurable business or engineering problems. • Incremental migration usually reduces risk compared with big-bang replacement. • The Strangler Fig pattern allows old and new architecture to coexist. • Compatibility is one of the most important tools for safe evolution. • Feature flags, shadow traffic, and gradual rollout reduce migration blast radius. • Database and API migrations require explicit compatibility strategies. • Characterization tests protect important existing behavior. • A migration is incomplete until obsolete architecture is safely retired.

26. Quick Review

1. What makes a system legacy?
2. When should refactoring be preferred over rewriting?
3. How does the Strangler Fig pattern reduce migration risk?
4. What is branch by abstraction?
5. Why are feature flags useful during modernization?
6. What is the expand-and-contract migration pattern?
7. Why is API compatibility important?
8. What purpose does shadow traffic serve?
9. Why should migration rollout be gradual?
10. Why must the old system eventually be retired?

27. Architect's Checklist • LearnForge • Engineering Handbook • 2026 257 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

Is the modernization objective measurable? • Are business-critical behaviors understood? • Are migration boundaries explicit? • Are dependencies and data flows documented? • Are compatibility requirements defined? • Are data consistency risks monitored? • Are rollout and rollback procedures tested? • Are feature flags temporary and owned? •

Are migration outcomes measured? • Is obsolete architecture scheduled for removal?

LearnForge • Engineering Handbook • 2026 258 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

Chapter 25 Architecture Patterns & Practical System Design

Architecture patterns provide reusable ways to structure software systems, but a pattern is valuable only when it solves a real problem without introducing unnecessary complexity. Good system design combines patterns, constraints, trade-offs, and operational realities into a coherent whole.

What You Will Learn

- How to select architecture patterns based on actual system constraints.
- When layered, hexagonal, event-driven, CQRS, and other patterns are useful.
- How to combine patterns without creating unnecessary complexity.
- How to evaluate architecture using business and technical requirements.
- How to turn architecture decisions into practical system designs.

1. Why Architecture Patterns Matter

Architecture patterns are reusable approaches for organizing software systems around recurring structural problems. They provide vocabulary and guidance, but they are not complete solutions that can be copied without adaptation. For example, a layered architecture may work extremely well for a business application with moderate scale and a small engineering team. The same structure may become restrictive when independent deployment, high-volume asynchronous processing, or multiple storage models become dominant concerns.

Architect's Rule

Never choose a pattern because it is popular. Choose it because a specific problem, constraint, or quality attribute makes its trade-offs worthwhile.

LearnForge • Engineering Handbook • 2026 259 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

2. Pattern Selection Begins With Constraints

Architecture starts with constraints rather than diagrams. Before selecting a pattern, identify the system's business goals, expected workload, team structure, reliability requirements, security constraints, and operational capabilities.

Constraint	Important Question	Possible Architectural Influence
Scale	What must scale independently?	Service separation, caching, partitioning
Redundancy	Redundancy, failover, graceful degradation	Availability
Which functions must remain available?	Latency	Which operations require fast responses?
Caching, replication, optimized paths	Team	How many teams own the system?
Module or service boundaries	structure	Data
How strong must consistency be?	Transactional or event-driven models	What can the organization reliably
Operations	Monolith, modular monolith, or services	operate?

3. Layered Architecture

Layered architecture separates software into conceptual levels such as presentation, application, domain, and infrastructure. Each layer has a defined responsibility and dependencies generally move in a controlled direction.

Figure 25.1 — A layered architecture separates responsibilities into presentation, application, domain, and infrastructure concerns. The major advantage is conceptual simplicity. Developers can locate responsibilities more easily and enforce architectural boundaries through project structure and dependency rules.

4. Hexagonal Architecture

Hexagonal architecture, also called ports and adapters architecture, separates business logic from external technologies. The application defines ports, while adapters implement

LearnForge •

Engineering Handbook • 2026 260 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE communication with databases, APIs, message brokers, user interfaces, or other external systems.

```
interface PaymentPort {
  charge(amount: number, currency: string): Promise;
}
class CheckoutService {
  constructor(private readonly payments: PaymentPort) {}
  async checkout(amount: number) {
    return this.payments.charge(amount, "USD");
  }
}
class StripePaymentAdapter implements PaymentPort {
  async charge(amount: number, currency: string) {
    // External payment provider integration
    return `payment-${amount}-${currency}`;
  }
}
```

The application depends on the port rather than directly depending on the external provider. This makes testing easier and allows infrastructure implementations to change without rewriting the core use case.

5. Clean Architecture and Dependency Direction

Clean Architecture emphasizes keeping business rules independent from frameworks, databases, and external delivery mechanisms. The central idea is that dependencies should point toward stable business concepts rather than toward volatile implementation details.

Layer	Primary Responsibility	Typical Dependency	Entities
Core	business rules	Minimal or none	Use Cases
Application	Application-specific workflows	Entities and interfaces	Adapters
Infrastructure	Frameworks, databases, external services	Adapters and interfaces	

LearnForge • Engineering Handbook • 2026 261 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

Architect's Rule

The most important boundary is not the folder structure. It is the direction of dependency. A database should not define the shape of the core business logic merely because the ORM makes that convenient.

6. Event-Driven Architecture

Event-driven architecture allows components to communicate by publishing and consuming events. The producer does not necessarily need to know which consumers will process the event.

```
await eventBus.publish({
  type: "ORDER_CONFIRMED",
  eventId: crypto.randomUUID(),
  orderId,
  occurredAt: new Date().toISOString()
});
```

This model can reduce synchronous coupling and allow new consumers to be added without changing the producer. The cost is additional complexity around event schemas, ordering, duplication, retries, observability, and eventual consistency.

7. CQRS Command Query Responsibility Segregation

separates operations that change state from operations that read state. The command side focuses on business changes while the query side can use models optimized for retrieval.

Side	Purpose	Typical Optimization
Command	Validate and perform state changes	Business rules and transactions
Query	Retrieve information	Read-optimized projections

CQRS is particularly useful when read and write workloads have significantly different requirements. It should not be introduced simply because the application contains both reads and writes; every normal application does.

LearnForge • Engineering Handbook • 2026 262 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE

Common Mistake

Treating CQRS as mandatory for every system. Separating reads and writes can increase operational and consistency complexity, so the benefit must justify the additional architecture.

8. Repository Pattern

The repository pattern provides an

abstraction around persistence operations. It can keep domain or application code independent from a specific database implementation.

```
interface UserRepository {
  findById(id: string): Promise;
  save(user: User): Promise;
}
class UserService {
  constructor(private readonly users: UserRepository) {}
  async getUser(id: string) {
    return this.users.findById(id);
  }
}
Repositories are most useful when they represent meaningful persistence operations. A repository that merely reproduces every ORM method may create an abstraction without providing meaningful architectural separation.


### 9. Strategy Pattern



The Strategy pattern allows interchangeable algorithms to be selected behind a stable interface. It is useful when behavior varies while the surrounding workflow remains the same.



LearnForge • Engineering Handbook • 2026 263 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE



```
interface PricingStrategy {
 calculate(total: number): number;
}
class RegularPricing implements PricingStrategy {
 calculate(total: number) {
 return total;
 }
}
class StudentPricing implements PricingStrategy {
 calculate(total: number) {
 return total * 0.8;
 }
}
function checkout(total: number, strategy: PricingStrategy) {
 return strategy.calculate(total);
}
```



### 10. Adapter Pattern



An adapter converts one interface into another interface expected by the application. It is particularly valuable at architectural boundaries where external systems use different data formats or protocols.



| External System        | Adapter              | Responsibility  | Internal Interface          |
|------------------------|----------------------|-----------------|-----------------------------|
| Payment provider       | Convert provider API | PaymentPort     | Email provider              |
| Convert message format | EmailPort            | Legacy database | Translate persistence model |
| Repository interface   |                      |                 |                             |



### 11. Anti-Corruption Layer



An anti-corruption layer protects a new domain from concepts and models belonging to a legacy or external system. Instead of allowing foreign concepts to spread throughout the application, the boundary translates them.



LearnForge • Engineering Handbook • 2026 264 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE



Figure 25.2 — An anti-corruption layer prevents legacy concepts from polluting a cleaner domain model.



### 12. Strangler Fig Pattern



The Strangler Fig pattern supports gradual modernization. Instead of rewriting an entire legacy system at once, new functionality is introduced around the existing system until parts of the old system can be safely retired. This approach reduces the risk of a large one-time migration and allows the team to learn from production behavior while modernization progresses.



### 13. Backend-for-Frontend



A Backend-for-Frontend, or BFF, provides an API tailored to a particular user interface. A mobile application may require a different data shape and interaction pattern from a web application.



```
app.get("/mobile/home", async (req, res) => {
 const [profile, courses, notifications] = await Promise.all([
 profileService.get(req.user.id),
 courseService.getRecommended(req.user.id),
 notificationService.getUnread(req.user.id)
]);
 res.json({ profile, courses, notifications });
});
```



A BFF can reduce client-side orchestration and provide an interface optimized for the actual presentation layer. It should not become an uncontrolled business-logic dumping ground.



LearnForge • Engineering Handbook • 2026 265 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE



### 14. Modular


```

Monolith as a Pattern A modular monolith applies strong internal boundaries while keeping the application in one deployable unit. It can provide many benefits associated with good service boundaries without immediately introducing distributed deployment. Characteristic Modular Monolith Microservices Deployment Usually one unit Multiple independent units Communication Mostly in-process Network communication Operational complexity Lower Higher Independent scaling Limited Strong Failure boundaries Mostly process-level Service-level Architect's Rule A well-designed modular monolith is not a failed microservice architecture. It is often the safest architecture for a system whose boundaries are still being discovered. LearnForge • Engineering Handbook • 2026 266 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE 15. Choosing Between Patterns Problem Potential Pattern Primary Benefit Primary Cost Complex internal Modular monolith Strong organization Shared deployment boundaries External dependency Hexagonal Testability and More interfaces isolation architecture replaceability Independent event Loose temporal Event-driven Eventual consistency processing coupling Different read/write Synchronization CQRS Specialized models needs complexity Temporary dual Legacy modernization Strangler pattern Incremental migration systems 16. Combining Patterns Real systems rarely use only one architectural pattern. A practical system may use a modular monolith internally, hexagonal boundaries around important integrations, event-driven processing for background work, and caching for high-volume reads. The danger is pattern accumulation. Every additional pattern introduces conceptual vocabulary, code, configuration, and operational responsibilities. Production Tip Document why each major pattern exists. If the team cannot explain the problem that a pattern solves, reconsider whether the pattern is still necessary. LearnForge • Engineering Handbook • 2026 267 LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE 17. Architecture Pattern Trade-offs Pattern Strength Weakness Best Fit Can create rigid Traditional business Layered Simple structure dependencies systems Domain-heavy Hexagonal External isolation More abstractions applications Asynchronous Event-driven Loose coupling Harder debugging workflows High-scale or complex CQRS Read/write specialization Higher complexity domains Modular Strong boundaries with simple Shared runtime Growing applications monolith deployment 18. Architecture Review Questions • What concrete problem does this pattern solve? • Which quality attribute does it improve? • What new complexity does it introduce? • Can the current team operate it reliably? • What happens when the pattern's assumptions stop being true? • How will the architecture evolve if traffic or organizational structure changes? 19. Practical System Design Example Consider an online learning platform containing authentication, courses, payments, notifications, search, and analytics. A practical architecture might begin as a modular monolith with clear domain boundaries. Payments can be isolated behind a port and adapter because the external provider may change. Notifications and analytics can use asynchronous events because they do not need to block course publishing. Search can use a separate projection if query

requirements justify it. LearnForge • Engineering Handbook • 2026 268

LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE Figure 25.3 — A practical architecture can combine modular boundaries, adapters, asynchronous events, and specialized projections without requiring every component to become an independent microservice.

20. Avoiding Pattern Overengineering Overengineering occurs when architecture becomes more complicated than the problem requires. A small application may not need CQRS, event sourcing, service meshes, distributed transactions, or multiple deployment pipelines. Common Mistake Confusing architectural sophistication with architectural quality. More components do not automatically produce a better system.

21. Pattern Evolution Architecture patterns should evolve as evidence changes. A system may begin as a simple layered application, become a modular monolith as the domain grows, and later extract only those capabilities that demonstrate a real need for independent deployment or scaling. This evolutionary approach reduces premature decisions and allows architecture to follow actual system behavior.

22. Pattern Selection Checklist • Is the underlying problem clearly defined? • Is the selected pattern solving that problem directly? • Are the pattern's trade-offs understood? • Does the team have the skills to operate it? • Does the pattern improve an important quality attribute? • Is there a simpler solution? • Can the architecture evolve if requirements change? • LearnForge • Engineering Handbook • 2026 269

LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE Is the decision documented?

23. Key Takeaways • Architecture patterns are tools, not goals. • Pattern selection should begin with constraints and quality attributes. • Hexagonal and clean architectures protect business logic from infrastructure details. • Event-driven architecture reduces some coupling while introducing consistency and operational challenges. • CQRS is valuable only when read/write differences justify its complexity. • Modular monoliths can provide strong boundaries without distributed-system overhead. • Anti-corruption and strangler patterns are powerful tools for modernization. • Good architecture uses the minimum complexity necessary to satisfy real requirements.

24. Quick Review 1. What should determine architecture pattern selection? 2. What is the main idea behind hexagonal architecture? 3. Why can CQRS be unnecessarily complex? 4. What problem does an anti-corruption layer solve? 5. When is a modular monolith a strong architectural choice? 6. How does the strangler pattern reduce modernization risk? 7. Why should architecture patterns be evaluated continuously? 8. What is the danger of pattern overengineering?

25. Architect's Checklist • Are architecture patterns tied to explicit business or technical requirements? • Are the benefits and costs of each major pattern documented? • Are domain boundaries clearer because of the selected patterns? • LearnForge • Engineering Handbook • 2026 270

LEARNFORGE SYSTEM ARCHITECTURE & CLEAN CODE Are infrastructure dependencies isolated where appropriate? • Are asynchronous workflows designed with idempotency and observability? • Has unnecessary distributed complexity been avoided? • Can the system evolve without requiring a

complete rewrite? • Can the engineering team explain why each major pattern exists? Production Tip The best architecture is rarely the one with the most patterns. It is the one whose boundaries, trade-offs, operational requirements, and evolution strategy are clear enough that engineers can make consistent decisions as the system grows. LearnForge • Engineering Handbook • 2026 271